

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN II

THIẾT KẾ MẠCH VI ĐIỀU KHIỂN ATMEGA16
VÀ ỨNG DỤNG GIÁM SÁT NHIỆT ĐỘ, ĐIỀU KHIỂN ĐỘNG CƠ

Giảng viên hướng dẫn: Th.S Ngô Đức Kiên
Th.S Nguyễn Thanh Hải
Học phần: ĐỒ ÁN II
Học kỳ: 2025.3

STT	Họ và tên	MSSV	Lớp
1	Đinh Ngọc Hoàng	20233408	Điện tử 11 – K68
2	Nguyễn Bá Toàn	20238498	Điện tử 11 – K68
3	Đinh Quang Vinh	20233716	Điện tử 11 – K68

Hà Nội, tháng 8 năm 2026

LỜI MỞ ĐẦU

Trong quá trình học tập các học phần chuyên ngành, việc kết hợp kiến thức lý thuyết với quá trình thiết kế và xây dựng một hệ thống điện tử hoàn chỉnh có vai trò quan trọng đối với sinh viên ngành kỹ thuật. Học phần Đồ án II tạo điều kiện để sinh viên vận dụng tổng hợp các kiến thức về vi điều khiển, thiết kế mạch điện tử, lập trình nhúng, mô phỏng và thiết kế mạch in vào một bài toán cụ thể. Qua đó, sinh viên không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện phương pháp phân tích, thiết kế, kiểm thử và hoàn thiện một hệ thống nhúng.

Trong đồ án này, nhóm thực hiện đề tài xây dựng hệ thống giám sát nhiệt độ và độ ẩm, kết hợp điều khiển tốc độ động cơ theo nhiệt độ. Hệ thống sử dụng vi điều khiển ATmega16 làm bộ xử lý trung tâm, cảm biến DHT22 để thu nhận thông tin nhiệt độ và độ ẩm môi trường, LCD 16x2 để hiển thị thông tin, các nút nhấn để thiết lập tham số và giao tiếp UART để truyền nhận dữ liệu với thiết bị bên ngoài. Tín hiệu PWM được tạo từ vi điều khiển nhằm thay đổi mức điều khiển động cơ theo giá trị nhiệt độ đo được.

Quá trình thực hiện đồ án bao gồm nhiều công việc từ phân tích yêu cầu, lựa chọn linh kiện, xây dựng sơ đồ nguyên lý, thiết kế PCB trên Altium Designer, phát triển chương trình điều khiển cho ATmega16 đến mô phỏng và kiểm tra hoạt động của hệ thống trên Proteus. Bên cạnh đó, các tệp Gerber và NC Drill được chuẩn bị từ thiết kế PCB nhằm phục vụ quá trình chế tạo và hoàn thiện phần cứng. Báo cáo trình bày lần lượt cơ sở lý thuyết, quá trình thiết kế phần cứng, phát triển phần mềm, mô phỏng, kiểm thử và đánh giá hệ thống.

Trong quá trình thực hiện Đồ án II, nhóm đã nhận được sự hướng dẫn và góp ý từ các giảng viên hướng dẫn là thầy Ngô Đức Kiên và thầy Nguyễn Thanh Hải. Nhóm xin chân thành cảm ơn các thầy đã dành thời gian hướng dẫn, trao đổi và đưa ra những nhận xét giúp nhóm từng bước hoàn thiện nội dung đồ án. Do thời gian thực hiện và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp để có thể tiếp tục hoàn thiện kiến thức, kỹ năng thiết kế và chất lượng của hệ thống trong các giai đoạn tiếp theo.

Mục lục

LỜI MỞ ĐẦU	1
DANH MỤC HÌNH ẢNH	9
DANH MỤC BẢNG	10
DANH MỤC TỪ VIẾT TẮT	11
1 TỔNG QUAN ĐỀ TÀI	12
1.1 Đặt vấn đề	12
1.2 Mục tiêu của đề án	12
1.3 Yêu cầu chức năng	13
1.4 Yêu cầu phi chức năng và ràng buộc thiết kế	14
1.5 Phạm vi thực hiện	14
1.6 Sơ đồ khối tổng thể của hệ thống	15
1.7 Quy trình thực hiện đề án	15
1.8 Cấu trúc báo cáo	16
2 CƠ SỞ LÝ THUYẾT VÀ LỰA CHỌN LINH KIỆN	18
2.1 Vi điều khiển ATmega16	18
2.1.1 Đặc điểm kiến trúc AVR	18
2.1.2 Hệ thống xung nhịp và mạch dao động	19
2.1.3 Các cổng vào/ra sử dụng trong đề án	19
2.2 Cảm biến nhiệt độ và độ ẩm DHT22	21
2.2.1 Đặc điểm của cảm biến DHT22	21
2.2.2 Nguyên lý giao tiếp với DHT22	22
2.2.3 Cấu trúc khung dữ liệu DHT22	22
2.3 Điều chế độ rộng xung PWM và điều khiển động cơ	23

2.3.1	Nguyên lý PWM	23
2.3.2	Fast PWM trên Timer0	24
2.3.3	Luật điều khiển theo nhiệt độ	25
2.4	Màn hình LCD 16x2	27
2.4.1	Cấu trúc và chân giao tiếp	27
2.4.2	Chế độ giao tiếp 8 bit	27
2.5	Giao tiếp UART	29
2.5.1	Khung truyền UART	29
2.5.2	Cấu hình tốc độ 9600 baud	30
2.5.3	Nhận dữ liệu bằng ngắt	31
2.6	Timer, ngắt và cơ chế đa nhiệm đơn giản	31
2.6.1	Timer1 phục vụ bộ đếm thời gian	32
2.6.2	Timer2 phục vụ quét phím và chống dội	33
2.7	Watchdog Timer	35
2.8	Nút nhấn và chống dội phím	35
2.9	Khởi nguồn và LM7805	36
2.10	Các khối hỗ trợ trên bo mạch	37
2.10.1	Mạch Reset và mạch nạp ISP	37
2.10.2	Đầu nối UART-USB	37
2.10.3	LED đơn và LED 7 đoạn	38
2.10.4	Các đầu nối mở rộng	38
2.11	Công cụ thiết kế và phát triển	38
2.11.1	Altium Designer	39
2.11.2	Proteus Design Suite	39
2.11.3	Atmel Studio/Microchip Studio và AVR-GCC	39
3	THIẾT KẾ PHẦN CỨNG VÀ MẠCH IN PCB	41
3.1	Phân rã hệ thống phần cứng	41
3.2	Phân bổ chân vi điều khiển	42
3.3	Thiết kế khởi nguồn	44

3.3.1	Nguồn vào và bảo vệ	44
3.3.2	Ổn áp 5 V sử dụng LM7805	45
3.3.3	Tụ lọc và LED báo nguồn	45
3.4	Thiết kế khối vi điều khiển ATmega16	46
3.4.1	Nguồn VCC/AVCC và GND	46
3.4.2	Mạch dao động thạch anh	47
3.4.3	Mạch Reset	47
3.4.4	Đầu nối nạp chương trình	48
3.5	Thiết kế khối cảm biến nhiệt độ và độ ẩm DHT22	48
3.6	Thiết kế khối LCD 16x2	49
3.7	Thiết kế khối nút nhấn	49
3.8	Thiết kế khối UART	50
3.9	Thiết kế ngõ ra PWM và giao tiếp động cơ	50
3.10	Các khối LED và LED 7 đoạn mở rộng	51
3.11	Xây dựng thư viện linh kiện	51
3.11.1	Schematic Library	51
3.11.2	PCB Library và footprint	52
3.12	Sơ đồ nguyên lý hoàn chỉnh	52
3.13	Thiết kế PCB	53
3.13.1	Thiết lập Board Shape và Design Rules	53
3.13.2	Bố trí linh kiện	54
3.13.3	Đi dây nguồn và tín hiệu	54
3.13.4	Đổ đồng và xử lý mass	54
3.13.5	Silkscreen và ký hiệu đầu nối	54
3.14	Kiểm tra DRC	55
3.15	Xuất file chế tạo Gerber và NC Drill	56
3.16	Danh sách linh kiện BOM	56
4	THIẾT KẾ PHẦN MỀM VÀ MÔ PHỎNG	58
4.1	Yêu cầu phần mềm	58

4.2	Kiến trúc chương trình	59
4.2.1	Mô hình vòng lặp chính kết hợp ngắt	59
4.2.2	Phân bổ chân I/O theo chương trình	59
4.2.3	Các biến trạng thái dùng chung	60
4.3	Tổ chức mã nguồn theo chức năng	61
4.4	Khởi tạo hệ thống	62
4.4.1	Vô hiệu hóa JTAG và cấu hình GPIO	62
4.4.2	Trình tự khởi tạo ngoại vi	62
4.4.3	Màn hình khởi động	62
4.5	Giao tiếp LCD 16x2	63
4.5.1	Nguyên tắc giao tiếp	63
4.5.2	Chuỗi lệnh khởi tạo LCD	63
4.6	Giao tiếp với cảm biến DHT22	64
4.6.1	Đặc điểm giao thức sử dụng trong chương trình	64
4.6.2	Phát tín hiệu bắt đầu và kiểm tra phản hồi	64
4.6.3	Giải mã 40 bit dữ liệu	65
4.6.4	Kiểm tra checksum và tính nhiệt độ	65
4.6.5	Chu kỳ đọc cảm biến	66
4.7	Thuật toán điều khiển tốc độ động cơ bằng PWM	66
4.7.1	Quan hệ giữa nhiệt độ và giá trị PWM	66
4.7.2	Điều kiện cho phép chạy động cơ	67
4.8	Tạo PWM bằng Timer0	67
4.8.1	Cấu hình Timer0	67
4.8.2	Nguyên lý PWM phần mềm	68
4.9	Xử lý nút nhấn và Timer2	68
4.9.1	Cấu hình Timer2 ở chế độ CTC	68
4.9.2	Thuật toán chống dội	69
4.9.3	Chức năng các nút trong vòng lặp chính	69
4.10	Bộ đếm thời gian bằng Timer1	70

4.10.1	Tính toán chu kỳ Timer1	70
4.11	Giao tiếp UART và điều khiển từ xa	70
4.11.1	Cấu hình tốc độ truyền	70
4.11.2	Định dạng khung truyền	71
4.11.3	Nhận lệnh dừng/chạy động cơ	71
4.11.4	Truyền dữ liệu trạng thái	72
4.12	Hiển thị và cảnh báo trên LCD	72
4.12.1	Giao diện trạng thái chính	72
4.12.2	Giao diện cài đặt ngưỡng	72
4.12.3	Trạng thái mất kết nối cảm biến	72
4.13	Cơ chế Watchdog	73
4.14	Lưu đồ thuật toán chương trình	73
4.14.1	Lưu đồ chương trình chính	73
4.14.2	Lưu đồ đọc DHT22	75
4.15	Phân tích vòng lặp chính từ tổng quát đến chi tiết	76
4.16	Xây dựng mô hình mô phỏng Proteus	76
4.16.1	Thiết lập ATmega16 và nạp chương trình	76
4.16.2	Mô phỏng cảm biến DHT22	77
4.16.3	Mô phỏng LCD 16x2	78
4.16.4	Mô phỏng cụm nút nhấn	78
4.16.5	Mô phỏng PWM và tăng công suất động cơ	79
4.16.6	Mô phỏng giao tiếp UART bằng Virtual Terminal	80
4.16.7	Tổng hợp cấu hình mô phỏng	80
4.17	Các trường hợp kiểm thử phần mềm	81
4.18	Đánh giá và các lưu ý đối với phiên bản phần mềm hiện tại	82
4.19	Kết luận chương	83
5	THỰC NGHIỆM, KIỂM THỬ VÀ ĐÁNH GIÁ	84
5.1	Mục tiêu và phương pháp kiểm thử	84
5.2	Kiểm thử cảm biến DHT22	87

5.2.1	Kiểm tra quá trình nhận dữ liệu	87
5.2.2	Kiểm tra trạng thái mất kết nối	87
5.3	Kiểm thử quan hệ nhiệt độ–PWM	88
5.3.1	Đối chiếu giá trị PWM theo mã nguồn	88
5.3.2	Kiểm tra dạng sóng PWM	89
5.4	Kiểm thử LCD và chức năng cảnh báo	89
5.4.1	Màn hình trạng thái chính	89
5.4.2	Màn hình cài đặt ngưỡng	90
5.5	Kiểm thử nút nhấn và chống dôi	92
5.6	Kiểm thử Timer1 và bộ đếm thời gian	93
5.7	Kiểm thử giao tiếp UART	94
5.7.1	Kiểm tra cấu hình tốc độ truyền	94
5.7.2	Kiểm tra dữ liệu truyền	94
5.7.3	Kiểm tra lệnh điều khiển từ UART	95
5.8	Kiểm thử Watchdog	96
5.9	Kiểm thử tích hợp toàn hệ thống	97
5.10	Các vấn đề và giới hạn phát hiện	98
5.10.1	Thời gian đọc DHT22 làm gián đoạn các ngắt	98
5.10.2	Truyền chuỗi UART bên trong ISR nhận	99
5.10.3	Chu kỳ đọc DHT22	99
5.10.4	Giá trị ngưỡng mặc định chưa thống nhất	99
5.11	Đối chiếu với yêu cầu của hệ thống	99
5.12	Đánh giá chung	100
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	102
6.1	Kết quả đạt được	102
6.2	Mức độ hoàn thành mục tiêu	102
6.3	Hạn chế của hệ thống	103
6.4	Bài học kinh nghiệm	104
6.5	Hướng phát triển	104

6.6	Kết luận chung	105
	TÀI LIỆU THAM KHẢO	106

Danh sách hình vẽ

Hình 4.1	Kiến trúc phần mềm và các luồng ngắt của chương trình	59
Hình 4.2	Lưu đồ chống dội nút nhấn bằng Timer2	69
Hình 4.3	Lưu đồ thuật toán chương trình chính	74
Hình 4.4	Lưu đồ thuật toán đọc và kiểm tra dữ liệu DHT22	75
Hình 4.5	Thiết lập ATmega16 và file chương trình trong Proteus	77
Hình 4.6	Kết nối cảm biến DHT22 với chân PA0 trong Proteus	77
Hình 4.7	Kết nối LCD 16x2 trong mô hình Proteus	78
Hình 4.8	Kết nối các nút UP, DOWN, SET và RESET trong Proteus	79
Hình 4.9	Tăng công suất điều khiển động cơ bằng tín hiệu PWM tại PB4	79
Hình 4.10	Kết nối UART giữa ATmega16 và Virtual Terminal trong Proteus	80
Hình 5.1	Kiểm thử hoạt động của cảm biến DHT22 trên mô hình thực tế	87
Hình 5.2	Trạng thái cảnh báo trên màn hình LCD khi nhiệt độ vượt ngưỡng	90
Hình 5.3	Màn hình LCD ở chế độ cài đặt ngưỡng nhiệt độ	91
Hình 5.4	Kết quả thay đổi giá trị ngưỡng nhiệt độ trên LCD	91
Hình 5.5	Kết quả kiểm thử bộ đếm thời gian sử dụng Timer1	94
Hình 5.6	Dữ liệu trạng thái được truyền qua UART trên Virtual Terminal	95
Hình 5.7	Kết quả gửi lệnh '0' để dừng động cơ qua UART	96
Hình 5.8	Kết quả gửi lệnh '1' để cho phép động cơ hoạt động trở lại	96

Danh sách bảng

Bảng 2.1	Ánh xạ các chân ATmega16 sử dụng trong hệ thống	20
Bảng 2.2	Cấu trúc khung dữ liệu của DHT22	23
Bảng 2.3	Quan hệ giữa nhiệt độ và giá trị PWM	26
Bảng 2.4	Kết nối giữa LCD 16x2 và ATmega16	27
Bảng 3.1	Phân bổ chân ATmega16 trong hệ thống	43
Bảng 3.2	Kết quả kiểm tra DRC của PCB	56
Bảng 3.3	Danh sách linh kiện chính của hệ thống	57
Bảng 4.1	Phân bổ chân I/O trong chương trình	60
Bảng 4.2	Các hàm và chương trình phục vụ ngắt trong mã nguồn	61
Bảng 4.3	Các thông số và kết nối chính trong mô hình Proteus	81
Bảng 4.4	Một số trường hợp kiểm thử chức năng phần mềm	82
Bảng 5.1	Ma trận kiểm thử các chức năng chính	86
Bảng 5.2	Giá trị PWM tính từ thuật toán điều khiển nhiệt độ	88
Bảng 5.3	Kiểm thử các trạng thái hiển thị LCD	92
Bảng 5.4	Các mốc kiểm thử bộ đếm thời gian	93
Bảng 5.5	Kiểm thử lệnh điều khiển qua UART	95
Bảng 5.6	Tổng hợp kết quả đối chiếu chức năng của phần mềm	98
Bảng 5.7	Đối chiếu yêu cầu và chức năng đã triển khai	100

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Ý nghĩa
MCU	Microcontroller Unit	Vi điều khiển
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự – số
PWM	Pulse Width Modulation	Điều chế độ rộng xung
UART	Universal Asynchronous Receiver/Transmitter	Giao tiếp nối tiếp bất đồng bộ
LCD	Liquid Crystal Display	Màn hình tinh thể lỏng
PCB	Printed Circuit Board	Mạch in
DRC	Design Rule Check	Kiểm tra luật thiết kế
BOM	Bill of Materials	Danh sách vật tư linh kiện
WDT	Watchdog Timer	Bộ định thời giám sát
ISR	Interrupt Service Routine	Chương trình phục vụ ngắt

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Trong nhiều hệ thống điện – điện tử hiện nay, việc giám sát nhiệt độ và điều khiển các thiết bị làm mát theo nhiệt độ môi trường là một yêu cầu phổ biến. Đối với các thiết bị điện tử, tủ điều khiển hoặc các hệ thống có phát sinh nhiệt trong quá trình hoạt động, nhiệt độ tăng cao có thể làm giảm độ ổn định, tuổi thọ linh kiện và ảnh hưởng đến khả năng làm việc của toàn hệ thống. Vì vậy, việc xây dựng một hệ thống có khả năng đo nhiệt độ và tự động điều chỉnh tốc độ động cơ hoặc quạt làm mát theo giá trị nhiệt độ đo được có ý nghĩa thực tế.

Bài toán này có thể được triển khai dưới dạng một hệ thống điều khiển nhúng sử dụng vi điều khiển làm bộ xử lý trung tâm. Nhiệt độ môi trường được thu nhận thông qua cảm biến nhiệt độ, sau đó được vi điều khiển xử lý để xác định chế độ hoạt động phù hợp của động cơ. Tốc độ động cơ được điều chỉnh bằng tín hiệu PWM nhằm thay đổi công suất điều khiển theo mức nhiệt độ. Bên cạnh đó, các thông số hoạt động của hệ thống có thể được hiển thị trên LCD, đồng thời các nút nhấn và giao tiếp UART được sử dụng để hỗ trợ quá trình cài đặt, giám sát và kiểm tra hệ thống.

Trong phạm vi học phần Đồ án II, đề tài được thực hiện nhằm vận dụng các kiến thức về vi điều khiển, cảm biến, ADC, PWM, Timer, giao tiếp ngoại vi và thiết kế mạch điện tử vào một hệ thống hoàn chỉnh. Quá trình thực hiện bao gồm thiết kế sơ đồ nguyên lý, xây dựng chương trình điều khiển, mô phỏng hoạt động, thiết kế mạch in PCB và kiểm tra các chức năng của hệ thống. Qua đó, đề tài không chỉ giải quyết bài toán giám sát và điều khiển nhiệt độ mà còn giúp củng cố kỹ năng thiết kế, lập trình và tích hợp một hệ thống nhúng trong thực tế.

1.2 Mục tiêu của đồ án

Mục tiêu của đồ án là thiết kế và xây dựng một hệ thống điều khiển nhúng sử dụng vi điều khiển ATmega16 làm bộ xử lý trung tâm, có khả năng thu thập thông tin nhiệt độ, xử lý dữ liệu và điều khiển tốc độ động cơ theo điều kiện nhiệt độ của môi trường. Hệ thống được xây dựng theo hướng tích hợp cả phần cứng và phần mềm, qua đó hình thành một mô hình hoàn chỉnh từ khâu thiết kế nguyên lý, thiết kế mạch in đến lập trình và mô phỏng hoạt động.

Về phần cứng, đồ án hướng tới xây dựng bo mạch sử dụng ATmega16, bao gồm khối nguồn, khối cảm biến nhiệt độ LM35, LCD 16x2, các nút nhấn điều khiển, khối giao tiếp UART, khối tạo tín hiệu PWM điều khiển động cơ và các mạch phụ trợ cần

thiết. Bo mạch cần bảo đảm khả năng cấp nguồn ổn định, nạp chương trình thuận tiện và có thể mở rộng phục vụ quá trình kiểm thử.

Về phần mềm, chương trình điều khiển được xây dựng bằng ngôn ngữ C cho ATmega16. Vi điều khiển thực hiện đọc giá trị nhiệt độ thông qua bộ chuyển đổi ADC, xử lý và hiển thị kết quả trên LCD 16x2, đồng thời tạo tín hiệu PWM để thay đổi tốc độ động cơ theo nhiệt độ. Hệ thống cũng cho phép người dùng thiết lập ngưỡng nhiệt độ bằng các nút nhấn, truyền nhận dữ liệu qua UART, thực hiện các tác vụ định thời bằng Timer và sử dụng Watchdog nhằm tăng khả năng phục hồi khi chương trình xảy ra trạng thái bất thường.

Thông qua quá trình thực hiện đồ án, các kiến thức về vi điều khiển, thiết kế mạch điện tử, thiết kế PCB, lập trình hệ thống nhúng và mô phỏng được kết hợp trong một hệ thống thống nhất. Đây cũng là cơ sở để đánh giá khả năng tích hợp giữa phần cứng và phần mềm trong quá trình phát triển một sản phẩm điện tử nhúng.

1.3 Yêu cầu chức năng

Để đáp ứng mục tiêu đặt ra, hệ thống cần thực hiện được các chức năng chính sau:

- Thu nhận tín hiệu nhiệt độ từ cảm biến LM35 thông qua một kênh ADC của vi điều khiển ATmega16.
- Chuyển đổi giá trị số thu được từ bộ ADC sang điện áp tương ứng và tính toán giá trị nhiệt độ của môi trường.
- Tạo tín hiệu PWM để điều khiển tốc độ động cơ hoặc quạt làm mát. Độ rộng xung PWM được thay đổi dựa trên giá trị nhiệt độ đo được và thuật toán điều khiển đã xây dựng.
- Hiển thị các thông tin cần thiết như nhiệt độ đo được, giá trị cài đặt và trạng thái hoạt động của hệ thống trên LCD 16x2.
- Cho phép người sử dụng thiết lập hoặc thay đổi ngưỡng nhiệt độ thông qua các nút nhấn SET, UP và DOWN. Nút RESET được sử dụng để đưa hệ thống về trạng thái khởi tạo khi cần thiết.
- Thực hiện giao tiếp UART để truyền dữ liệu trạng thái của hệ thống tới thiết bị bên ngoài, đồng thời nhận các lệnh điều khiển chạy hoặc dừng từ giao diện giao tiếp nối tiếp.
- Sử dụng Timer và cơ chế ngắt của vi điều khiển để thực hiện các tác vụ có yêu cầu về thời gian, hỗ trợ quá trình cập nhật dữ liệu, kiểm tra trạng thái phím và điều phối các chức năng của chương trình.

- Sử dụng bộ Watchdog Timer để giám sát hoạt động của chương trình. Trong trường hợp chương trình bị treo hoặc không thực hiện đúng chu trình dự kiến, Watchdog có khả năng đưa vi điều khiển trở lại trạng thái khởi động nhằm tăng độ tin cậy của hệ thống.

1.4 Yêu cầu phi chức năng và ràng buộc thiết kế

Bên cạnh các yêu cầu về chức năng, hệ thống cần đáp ứng một số yêu cầu phi chức năng và ràng buộc trong quá trình thiết kế. Trước hết, bo mạch sử dụng nguồn một chiều ổn định phù hợp với mức điện áp làm việc của ATmega16 và các linh kiện ngoại vi. Khối nguồn cần hạn chế dao động điện áp và bảo đảm khả năng cấp dòng cho vi điều khiển, LCD, cảm biến và các mạch chức năng khác trong quá trình hoạt động.

ATmega16 phải làm việc với tần số hệ thống được xác định thống nhất giữa phần cứng và chương trình, vì thông số này ảnh hưởng trực tiếp đến quá trình tính toán Baud rate của UART, tần số PWM và các khoảng thời gian được tạo bởi Timer. Các linh kiện liên quan đến mạch dao động, mạch Reset và tụ lọc nguồn cần được bố trí hợp lý để hạn chế ảnh hưởng của nhiễu tới hoạt động của vi điều khiển.

Bo mạch cần có đầu nối phục vụ quá trình nạp chương trình cho ATmega16 mà không cần tháo vi điều khiển khỏi mạch. Các đầu nối tín hiệu và linh kiện ngoại vi cần được bố trí thuận tiện cho việc lắp ráp, kiểm tra và đo đạc.

Trong quá trình thiết kế PCB trên Altium Designer, kích thước linh kiện và footprint phải phù hợp với linh kiện thực tế. Việc bố trí linh kiện và đi dây cần tuân theo các quy tắc thiết kế đã thiết lập. Sau khi hoàn thiện PCB, thiết kế phải được kiểm tra bằng công cụ Design Rule Check (DRC) để phát hiện các lỗi như khoảng cách đường mạch không phù hợp, đường mạch chưa kết nối hoặc các vi phạm về kích thước.

Các tệp Gerber và NC Drill cần được xuất đầy đủ để phục vụ quá trình gia công PCB. Đồng thời, mô hình hệ thống trên Proteus cần cho phép quan sát các tín hiệu quan trọng như giá trị nhiệt độ, tín hiệu PWM, dữ liệu UART và trạng thái LCD nhằm hỗ trợ việc kiểm chứng hoạt động của chương trình trước khi triển khai trên phần cứng thực tế.

1.5 Phạm vi thực hiện

Trong phạm vi của Đề án II, đề tài tập trung vào quá trình thiết kế một hệ thống điều khiển nhiệt độ sử dụng vi điều khiển ATmega16, bao gồm cả phần cứng, phần mềm và mô phỏng.

Đối với phần cứng, đề án thực hiện xây dựng sơ đồ nguyên lý trên Altium Designer, lựa chọn linh kiện, xây dựng hoặc sử dụng các thư viện schematic và footprint phù hợp, bố trí linh kiện trên PCB và thực hiện đi dây cho bo mạch. Thiết kế PCB được kiểm tra

bằng DRC và các tệp phục vụ chế tạo như Gerber và NC Drill được xuất sau khi hoàn thiện.

Đối với phần mềm, chương trình điều khiển cho ATmega16 được phát triển theo từng chức năng, bao gồm đọc ADC, tính toán nhiệt độ, điều khiển PWM, hiển thị LCD, xử lý nút nhấn, giao tiếp UART, sử dụng Timer, ngắt và Watchdog. Các chức năng sau đó được tích hợp thành chương trình điều khiển chung của hệ thống.

Hoạt động của hệ thống được kiểm tra thông qua mô phỏng trên Proteus nhằm đánh giá sự phối hợp giữa vi điều khiển và các khối ngoại vi. Các tín hiệu và trạng thái chính được quan sát để kiểm tra tính đúng đắn của thuật toán điều khiển trước khi triển khai trên phần cứng.

Phạm vi đánh giá của báo cáo được giới hạn trong các nội dung đã thực tế thực hiện trong quá trình đồ án. Đối với những hạng mục chưa được chế tạo hoặc chưa có số liệu đo trên bo mạch thực, báo cáo chỉ đánh giá dựa trên kết quả thiết kế PCB, dữ liệu Gerber và kết quả mô phỏng, không sử dụng kết quả giả định thay cho kết quả thực nghiệm.

1.6 Sơ đồ khối tổng thể của hệ thống

Hệ thống được xây dựng với ATmega16 đóng vai trò là bộ xử lý trung tâm. Các khối chức năng được kết nối với vi điều khiển để thực hiện quá trình thu thập dữ liệu, xử lý và điều khiển.

Khối nguồn có nhiệm vụ tạo điện áp một chiều ổn định cung cấp cho ATmega16 và các linh kiện ngoại vi. Khối cảm biến LM35 thực hiện chuyển đổi nhiệt độ môi trường thành tín hiệu điện áp tương ứng. Tín hiệu này được đưa tới một kênh ADC của ATmega16 để chuyển đổi sang dạng số và xử lý.

Sau khi xác định giá trị nhiệt độ, vi điều khiển thực hiện thuật toán điều khiển và tạo tín hiệu PWM tại ngõ ra để thay đổi tốc độ của động cơ. Giá trị nhiệt độ, ngưỡng cài đặt và trạng thái hoạt động của hệ thống được hiển thị trên LCD 16x2.

Khối nút nhấn cung cấp giao diện điều khiển trực tiếp cho người sử dụng, cho phép thay đổi các tham số cài đặt của hệ thống. Bên cạnh đó, khối UART cho phép ATmega16 trao đổi dữ liệu với thiết bị bên ngoài, phục vụ quá trình giám sát và điều khiển. Các khối Timer, ngắt và Watchdog được tích hợp bên trong vi điều khiển để hỗ trợ quản lý thời gian và nâng cao độ ổn định của chương trình.

1.7 Quy trình thực hiện đồ án

Quá trình thực hiện đồ án được tiến hành theo các bước chính như sau:

- 1. Xác định yêu cầu và lựa chọn linh kiện:** phân tích các chức năng cần thực hiện

của hệ thống, lựa chọn ATmega16 làm vi điều khiển trung tâm và xác định các linh kiện ngoại vi cần thiết như cảm biến nhiệt độ, LCD 16x2, khối giao tiếp UART, nút nhấn và mạch điều khiển động cơ.

2. **Thiết kế sơ đồ nguyên lý:** xây dựng hoặc kiểm tra thư viện schematic và footprint của các linh kiện, sau đó thiết kế sơ đồ nguyên lý của hệ thống trên Altium Designer. Các kết nối nguồn, vi điều khiển và ngoại vi được kiểm tra trước khi chuyển sang bước thiết kế PCB.
3. **Thiết kế mạch in PCB:** bố trí linh kiện trên bo mạch, thực hiện đi dây và thiết lập các quy tắc thiết kế phù hợp. Sau khi hoàn thiện, bo mạch được kiểm tra bằng DRC. Các tệp Gerber và NC Drill được xuất để phục vụ quá trình chế tạo PCB.
4. **Phát triển chương trình điều khiển:** xây dựng chương trình C cho ATmega16 theo từng chức năng riêng biệt như đọc ADC, điều khiển PWM, hiển thị LCD, xử lý nút nhấn, giao tiếp UART, Timer và Watchdog. Các chức năng được kiểm tra từng bước trước khi tích hợp vào chương trình hoàn chỉnh.
5. **Mô phỏng và tích hợp hệ thống:** xây dựng mô hình trên Proteus, kết nối ATmega16 với các linh kiện ngoại vi và nạp chương trình đã biên dịch vào mô hình vi điều khiển. Các chức năng được kiểm tra riêng lẻ trước khi đánh giá hoạt động tổng thể của hệ thống.
6. **Kiểm thử và đánh giá:** theo dõi các thông số đầu vào, đầu ra và trạng thái hoạt động của hệ thống, so sánh kết quả thu được với các yêu cầu đã đặt ra. Trên cơ sở đó tiến hành điều chỉnh chương trình, hoàn thiện thiết kế và xây dựng báo cáo Đồ án II.

1.8 Cấu trúc báo cáo

Nội dung báo cáo được tổ chức thành sáu chương chính. Nội dung cụ thể của từng chương được trình bày như sau:

Chương 1 – Tổng quan đề tài: trình bày vấn đề đặt ra, mục tiêu của đề án, các yêu cầu chức năng và phi chức năng, phạm vi thực hiện, sơ đồ khối tổng thể và quy trình triển khai đề tài.

Chương 2 – Cơ sở lý thuyết và lựa chọn linh kiện: trình bày các kiến thức nền tảng liên quan đến ATmega16, bộ chuyển đổi ADC, cảm biến nhiệt độ, phương pháp điều chế độ rộng xung PWM, LCD 16x2, UART, Timer, ngắt, Watchdog và các linh kiện được sử dụng trong hệ thống.

Chương 3 – Thiết kế phần cứng và PCB: trình bày quá trình xây dựng sơ đồ nguyên lý, thiết kế từng khối chức năng, lựa chọn và bố trí linh kiện, thiết kế PCB trên

Altium Designer, kiểm tra DRC và xuất các tệp Gerber, NC Drill phục vụ chế tạo bo mạch.

Chương 4 – Thiết kế phần mềm và mô phỏng hệ thống: trình bày cấu trúc chương trình điều khiển cho ATmega16, thuật toán xử lý nhiệt độ, điều khiển PWM, LCD, nút nhấn, UART, Timer và Watchdog. Chương này đồng thời trình bày quá trình mô phỏng và tích hợp hệ thống trên Proteus.

Chương 5 – Kiểm thử và đánh giá kết quả: trình bày phương pháp kiểm thử từng chức năng của hệ thống, các kết quả thu được trong quá trình mô phỏng hoặc thực nghiệm, đồng thời đánh giá mức độ đáp ứng của hệ thống so với các yêu cầu đã đặt ra.

Chương 6 – Kết luận và hướng phát triển: tổng hợp những nội dung đã thực hiện, đánh giá kết quả của đồ án, chỉ ra các hạn chế còn tồn tại và đề xuất một số hướng cải tiến hoặc mở rộng hệ thống trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ LỰA CHỌN LINH KIỆN

2.1 Vi điều khiển ATmega16

ATmega16 được lựa chọn làm bộ xử lý trung tâm của hệ thống. Đây là vi điều khiển 8 bit thuộc họ AVR, tích hợp nhiều ngoại vi cần thiết cho các ứng dụng điều khiển nhúng. Trong đề án, ATmega16 đảm nhiệm việc giao tiếp với cảm biến nhiệt độ và độ ẩm DHT22, xử lý dữ liệu thu nhận được, tạo tín hiệu PWM điều khiển động cơ, hiển thị thông tin trên LCD 16x2, xử lý nút nhấn, giao tiếp UART và thực hiện các tác vụ định thời.

Việc sử dụng một vi điều khiển tích hợp nhiều ngoại vi giúp giảm số lượng linh kiện bên ngoài, đồng thời thuận lợi cho quá trình thiết kế mạch, lập trình và mô phỏng hệ thống. Các khối chức năng của ATmega16 được khai thác trực tiếp trong đề án gồm GPIO, Timer/Counter, bộ tạo PWM, USART, hệ thống ngắt và Watchdog Timer.

2.1.1 Đặc điểm kiến trúc AVR

ATmega16 sử dụng kiến trúc AVR 8 bit theo nguyên lý Harvard, trong đó bộ nhớ chương trình và bộ nhớ dữ liệu được tổ chức thành hai không gian riêng biệt. Cấu trúc này cho phép quá trình truy xuất lệnh và dữ liệu được thực hiện hiệu quả, phù hợp với các ứng dụng điều khiển nhúng có yêu cầu xử lý theo thời gian thực.

Vi điều khiển có 32 thanh ghi làm việc 8 bit được kết nối trực tiếp với khối xử lý số học và logic ALU. Phần lớn các lệnh của kiến trúc AVR được thực hiện trong một chu kỳ xung nhịp, nhờ đó ATmega16 đáp ứng tốt yêu cầu xử lý của hệ thống.

Các vùng nhớ chính của ATmega16 bao gồm:

- Bộ nhớ Flash dung lượng 16 KB dùng để lưu chương trình;
- Bộ nhớ SRAM dung lượng 1 KB dùng để lưu biến và dữ liệu trong quá trình chương trình hoạt động;
- Bộ nhớ EEPROM dung lượng 512 byte dùng để lưu các dữ liệu cần duy trì khi nguồn điện bị ngắt;
- 32 đường vào/ra số được tổ chức thành bốn cổng PORTA, PORTB, PORTC và PORTD.

Ngoài các khối nhớ và GPIO, ATmega16 còn tích hợp các bộ Timer/Counter, khả năng tạo tín hiệu PWM, giao tiếp USART, SPI, TWI, bộ chuyển đổi ADC, hệ thống ngắt và Watchdog Timer. Trong phạm vi đề án, các khối GPIO, Timer/Counter, PWM,

USART và Watchdog được sử dụng trực tiếp để thực hiện các chức năng điều khiển của hệ thống.

Cảm biến DHT22 được giao tiếp với vi điều khiển thông qua một chân GPIO theo phương thức truyền dữ liệu số một dây. PORTB được sử dụng cho các nút nhấn và ngõ ra PWM, PORTC được sử dụng làm bus dữ liệu cho LCD 16x2, trong khi PORTD đảm nhiệm các tín hiệu điều khiển LCD và giao tiếp UART.

2.1.2 Hệ thống xung nhịp và mạch dao động

Xung nhịp là tín hiệu thời gian cơ sở quyết định tốc độ thực thi lệnh và hoạt động của các ngoại vi bên trong vi điều khiển. Tần số xung nhịp ảnh hưởng trực tiếp đến tốc độ Baud của UART, tần số PWM, thời gian của các bộ Timer/Counter và quá trình xác định độ rộng xung khi giao tiếp với cảm biến DHT22.

Trong chương trình của hệ thống, tần số hoạt động của vi điều khiển được khai báo:

```
#define F_CPU 8000000UL
```

Do đó, chương trình được xây dựng với giả thiết tần số hoạt động của ATmega16 là 8 MHz. Việc khai báo chính xác giá trị F_CPU đặc biệt quan trọng đối với các hàm tạo trễ trong thư viện util/delay.h, giao tiếp UART, Timer và quá trình tạo các khoảng thời gian cần thiết khi truyền nhận dữ liệu với DHT22.

ATmega16 có thể sử dụng bộ dao động nội hoặc nguồn xung nhịp bên ngoài tùy thuộc vào cấu hình Fuse bit và thiết kế phần cứng. Trong trường hợp sử dụng thạch anh ngoài, thạch anh được kết nối vào hai chân XTAL1 và XTAL2 cùng các tụ tải phù hợp. Các linh kiện thuộc mạch dao động cần được bố trí gần vi điều khiển để giảm chiều dài đường mạch và hạn chế ảnh hưởng của nhiễu.

Tần số xung nhịp thực tế của phần cứng phải thống nhất với giá trị được khai báo trong chương trình. Nếu có sai lệch, tốc độ truyền UART, tần số PWM, khoảng thời gian của Timer cũng như việc xác định các xung dữ liệu của DHT22 có thể không còn chính xác.

2.1.3 Các cổng vào/ra sử dụng trong đồ án

Các cổng vào/ra của ATmega16 được phân chia theo chức năng của từng khối ngoại vi. Ánh xạ các nhóm chân chính của hệ thống được trình bày trong Bảng 2.1.

Các chân kết nối với nút nhấn được cấu hình ở chế độ đầu vào và có thể sử dụng điện trở kéo lên nội của ATmega16. Khi đó, trạng thái không nhấn tương ứng với mức logic cao và trạng thái nhấn tương ứng với mức logic thấp. Chương trình thực hiện kiểm

Bảng 2.1 Ánh xạ các chân ATmega16 sử dụng trong hệ thống

Cổng/chân	Chức năng	Mô tả sử dụng
GPIO kết nối DHT22	DHT22 DATA	Truyền và nhận dữ liệu số giữa ATmega16 và cảm biến DHT22.
PB0	Nút UP	Tăng giá trị ngưỡng nhiệt độ trong chế độ cài đặt.
PB1	Nút DOWN	Giảm giá trị ngưỡng nhiệt độ trong chế độ cài đặt.
PB2	Nút SET	Chuyển đổi giữa chế độ hiển thị thông thường và chế độ cài đặt ngưỡng nhiệt độ.
PB3/OC0	Ngõ ra PWM	Xuất tín hiệu PWM từ Timer0 để điều khiển tốc độ động cơ.
PB4	Nút RESET	Đưa giá trị ngưỡng nhiệt độ về giá trị mặc định.
PC0–PC7	Dữ liệu LCD	Bus dữ liệu 8 bit kết nối với LCD 16x2.
PD0/RXD	UART RX	Nhận dữ liệu hoặc lệnh điều khiển từ thiết bị bên ngoài.
PD1/TXD	UART TX	Truyền nhiệt độ, độ ẩm, giá trị PWM và trạng thái hệ thống.
PD5	LCD RW	Tín hiệu Read/Write của LCD 16x2.
PD6	LCD RS	Tín hiệu Register Select của LCD 16x2.
PD7	LCD EN	Tín hiệu Enable của LCD 16x2.

tra trạng thái các nút và xử lý chống rung bằng phần mềm.

Chân PB3 có chức năng OC0 của Timer0 và được sử dụng để phát tín hiệu PWM. PORTC được cấu hình làm bus dữ liệu 8 bit cho LCD. Do một số chân PORTC có liên quan đến chức năng JTAG của ATmega16, chức năng JTAG cần được vô hiệu hóa nếu muốn sử dụng toàn bộ PORTC như các chân GPIO thông thường.

Giao tiếp UART sử dụng hai chân phần cứng PD0/RXD và PD1/TXD. Các chân PD5, PD6 và PD7 được sử dụng cho các tín hiệu RW, RS và EN của LCD. Việc ánh xạ các chân cần được giữ thống nhất giữa chương trình, sơ đồ nguyên lý trên Altium Designer và mô hình mô phỏng trên Proteus.

2.2 Cảm biến nhiệt độ và độ ẩm DHT22

DHT22 là cảm biến số có khả năng đo đồng thời nhiệt độ và độ ẩm môi trường. Khác với các cảm biến nhiệt độ có đầu ra điện áp tương tự, DHT22 tích hợp sẵn khối cảm biến, mạch xử lý và chuyển đổi tín hiệu bên trong. Kết quả đo được truyền trực tiếp tới vi điều khiển dưới dạng dữ liệu số.

Do đó, khi sử dụng DHT22, ATmega16 không cần sử dụng bộ ADC để chuyển đổi tín hiệu nhiệt độ. Vi điều khiển chỉ cần giao tiếp với cảm biến thông qua một chân GPIO và thực hiện đúng trình tự thời gian của giao thức truyền dữ liệu.

2.2.1 Đặc điểm của cảm biến DHT22

DHT22 có bốn chân kết nối, trong đó các chân chính gồm nguồn cấp, chân DATA và chân GND. Chân DATA được sử dụng để truyền dữ liệu hai chiều giữa cảm biến và vi điều khiển. Đường truyền dữ liệu thường được duy trì ở mức logic cao khi ở trạng thái nghỉ thông qua một điện trở kéo lên.

Cảm biến thực hiện việc đo và xử lý tín hiệu bên trong, sau đó gửi kết quả dưới dạng một khung dữ liệu số. Nhờ đó, giá trị đo không phụ thuộc vào độ phân giải ADC của vi điều khiển và không cần thực hiện phép chuyển đổi từ điện áp sang nhiệt độ như đối với cảm biến tương tự.

DHT22 cung cấp hai đại lượng chính:

- Nhiệt độ môi trường, biểu diễn theo đơn vị °C;
- Độ ẩm tương đối, biểu diễn theo đơn vị %.

Độ phân giải dữ liệu nhiệt độ và độ ẩm của DHT22 là 0,1 đơn vị. Vì vậy, dữ liệu thô nhận được từ cảm biến cần được chia cho 10 để thu được giá trị thực.

2.2.2 Nguyên lý giao tiếp với DHT22

Quá trình truyền dữ liệu giữa ATmega16 và DHT22 được thực hiện trên một đường tín hiệu DATA. Vi điều khiển đóng vai trò thiết bị khởi tạo quá trình truyền, trong khi DHT22 phản hồi và gửi dữ liệu đo.

Ban đầu, đường DATA được duy trì ở mức cao. Khi cần đọc dữ liệu, ATmega16 chuyển chân DATA thành ngõ ra và kéo đường tín hiệu xuống mức thấp trong một khoảng thời gian đủ lớn để tạo tín hiệu bắt đầu. Sau đó, vi điều khiển đưa đường DATA trở lại mức cao và chuyển chân tương ứng sang chế độ đầu vào để chờ phản hồi từ cảm biến.

Sau khi nhận được tín hiệu khởi tạo, DHT22 phát tín hiệu xác nhận và bắt đầu truyền chuỗi dữ liệu. Mỗi bit dữ liệu được mã hóa bằng độ rộng của xung mức cao. Vì vậy, chương trình trên ATmega16 cần xác định thời gian tồn tại của các xung để phân biệt bit logic 0 và bit logic 1.

Quá trình giao tiếp có thể mô tả theo trình tự:

1. ATmega16 phát tín hiệu bắt đầu tới DHT22;
2. Chân DATA được chuyển sang chế độ đầu vào;
3. ATmega16 chờ tín hiệu phản hồi từ cảm biến;
4. DHT22 gửi tuần tự 40 bit dữ liệu;
5. Vi điều khiển đo độ rộng từng xung và xác định giá trị bit;
6. Dữ liệu được ghép thành các byte nhiệt độ, độ ẩm và checksum;
7. Chương trình kiểm tra checksum trước khi sử dụng kết quả đo.

2.2.3 Cấu trúc khung dữ liệu DHT22

Mỗi lần truyền, DHT22 gửi một khung dữ liệu có tổng chiều dài 40 bit, tương ứng với 5 byte. Cấu trúc dữ liệu được trình bày trong Bảng 2.2.

Hai byte đầu tiên được ghép lại để tạo thành giá trị độ ẩm 16 bit. Giá trị độ ẩm được xác định theo:

$$H = \frac{(D_H \times 256 + D_L)}{10}, \quad (2.1)$$

trong đó D_H và D_L lần lượt là byte cao và byte thấp của dữ liệu độ ẩm.

Tương tự, hai byte tiếp theo chứa dữ liệu nhiệt độ. Nếu bỏ qua bit dấu đối với trường hợp nhiệt độ dương, giá trị nhiệt độ được xác định bởi:

Bảng 2.2 Cấu trúc khung dữ liệu của DHT22

Byte	Nội dung	Mô tả
Byte 1	Độ ẩm, 8 bit cao	Chứa 8 bit cao của dữ liệu độ ẩm.
Byte 2	Độ ẩm, 8 bit thấp	Chứa 8 bit thấp của dữ liệu độ ẩm.
Byte 3	Nhiệt độ, 8 bit cao	Chứa bit dấu và các bit cao của dữ liệu nhiệt độ.
Byte 4	Nhiệt độ, 8 bit thấp	Chứa các bit thấp của dữ liệu nhiệt độ.
Byte 5	Checksum	Dùng để kiểm tra tính đúng đắn của dữ liệu nhận được.

$$T = \frac{(T_H \times 256 + T_L)}{10}, \quad (2.2)$$

trong đó T_H và T_L lần lượt là byte cao và byte thấp của dữ liệu nhiệt độ.

Đối với giá trị nhiệt độ âm, bit cao nhất của dữ liệu nhiệt độ được sử dụng làm bit dấu. Chương trình cần kiểm tra bit này trước khi thực hiện phép chuyển đổi giá trị nhiệt độ.

Byte cuối cùng là checksum. Giá trị checksum được xác định từ tổng bốn byte dữ liệu trước đó và lấy 8 bit thấp:

$$Checksum = (Byte_1 + Byte_2 + Byte_3 + Byte_4) \bmod 256. \quad (2.3)$$

Vì điều khiển chỉ sử dụng kết quả nhiệt độ và độ ẩm khi checksum tính toán trùng với byte checksum do DHT22 gửi về. Cơ chế này giúp phát hiện một số lỗi xảy ra trong quá trình truyền dữ liệu.

2.3 Điều chế độ rộng xung PWM và điều khiển động cơ

Sau khi thu nhận giá trị nhiệt độ từ DHT22, ATmega16 sử dụng giá trị này làm đầu vào cho thuật toán điều khiển tốc độ động cơ. Tín hiệu điều khiển được tạo bằng phương pháp điều chế độ rộng xung PWM thông qua Timer0 của ATmega16.

2.3.1 Nguyên lý PWM

PWM (*Pulse Width Modulation*) là phương pháp điều khiển giá trị trung bình của điện áp hoặc công suất cung cấp cho tải bằng cách thay đổi khoảng thời gian tín hiệu ở mức cao trong mỗi chu kỳ.

Một chu kỳ PWM gồm thời gian tín hiệu ở mức cao Ton và thời gian tín hiệu ở mức thấp Toff. Chu kỳ tín hiệu được xác định bởi:

$$T_{\text{PWM}} = T_{\text{on}} + T_{\text{off}}. \quad (2.4)$$

Tần số của tín hiệu PWM được xác định theo:

$$f_{\text{PWM}} = \frac{1}{T_{\text{PWM}}}. \quad (2.5)$$

Hệ số công tác (*duty cycle*) biểu thị tỷ lệ thời gian tín hiệu ở mức cao trong một chu kỳ:

$$D = \frac{T_{\text{on}}}{T_{\text{on}} + T_{\text{off}}} \times 100\%. \quad (2.6)$$

Khi duty cycle tăng, thời gian tín hiệu ở mức cao trong một chu kỳ tăng, từ đó làm tăng công suất trung bình cấp cho động cơ. Ngược lại, khi duty cycle giảm, công suất trung bình giảm và tốc độ động cơ cũng giảm.

Trong trường hợp lý tưởng, điện áp trung bình có thể biểu diễn gần đúng theo:

$$V_{\text{avg}} \approx D \times V_{\text{CC}}, \quad (2.7)$$

trong đó D được biểu diễn trong khoảng từ 0 đến 1.

2.3.2 Fast PWM trên Timer0

Timer0 của ATmega16 được sử dụng ở chế độ Fast PWM để tạo tín hiệu điều khiển tốc độ động cơ. Chân PB3 tương ứng với chức năng OC0 được cấu hình là ngõ ra:

```
DDRB |= (1 << PB3);
```

Thanh ghi điều khiển Timer0 được cấu hình:

```
TCCR0 |= (1 << WGM01) | (1 << WGM00)
        | (1 << COM01) | (1 << CS01);
```

Hai bit WGM01 và WGM00 cùng được đặt bằng 1, do đó Timer0 hoạt động ở chế độ Fast PWM 8 bit. Bộ đếm tăng từ 0 đến 255 và sau đó quay trở lại 0 để bắt đầu chu kỳ mới.

Bit COM01 được đặt bằng 1 và COM00 bằng 0, tương ứng với chế độ PWM không đảo (*non-inverting mode*). Trong chế độ này, tín hiệu tại chân OC0 được phân cứng Timer0 điều khiển dựa trên giá trị của thanh ghi OCR0.

Bit CS01 được đặt bằng 1, tương ứng với hệ số chia tần số:

$$N = 8. \quad (2.8)$$

Với chế độ Fast PWM 8 bit, tần số tín hiệu PWM được xác định theo:

$$f_{\text{PWM}} = \frac{f_{\text{CPU}}}{N \times 256}. \quad (2.9)$$

Với $f_{\text{CPU}}=8$ MHz và $N=8$, ta có:

$$f_{\text{PWM}} = \frac{8 \times 10^6}{8 \times 256} \approx 3906.25 \text{ Hz}. \quad (2.10)$$

Như vậy, tần số PWM của hệ thống xấp xỉ 3.91 kHz.

Độ rộng xung được thay đổi thông qua thanh ghi OCR0. Với Timer0 8 bit, OCR0 có thể nhận giá trị từ 0 đến 255. Duty cycle có thể được biểu diễn gần đúng theo:

$$D \approx \frac{\text{OCR0}}{255} \times 100\%. \quad (2.11)$$

Do việc tạo PWM được thực hiện trực tiếp bằng phần cứng Timer0, vi điều khiển không cần liên tục thay đổi trạng thái chân OC0 bằng phần mềm, giúp giảm tải cho CPU và tạo tín hiệu điều khiển ổn định.

2.3.3 Luật điều khiển theo nhiệt độ

Giá trị nhiệt độ thu nhận từ DHT22 được sử dụng làm đầu vào của thuật toán điều khiển tốc độ động cơ. Trong chương trình, dải nhiệt độ được chia thành ba vùng.

Khi nhiệt độ nhỏ hơn 30 C, động cơ dừng:

$$\text{PWM} = 0, \quad T < 30 \text{ }^{\circ}\text{C}. \quad (2.12)$$

Trong khoảng từ 30 C đến 60 C, giá trị PWM tăng tuyến tính theo nhiệt độ:

$$\text{PWM} = \frac{T - 30}{60 - 30} \times 255 = \frac{T - 30}{30} \times 255, \quad 30 \leq T \leq 60. \quad (2.13)$$

Khi nhiệt độ lớn hơn 60 C, động cơ được điều khiển với PWM cực đại:

$$PWM = 255, \quad T > 60^{\circ}\text{C}. \quad (2.14)$$

Luật điều khiển tổng quát được biểu diễn bởi:

$$PWM(T) = \begin{cases} 0, & T < 30, \\ \frac{T-30}{30} \times 255, & 30 \leq T \leq 60, \\ 255, & T > 60. \end{cases} \quad (2.15)$$

Nếu biểu diễn theo duty cycle, ta có:

$$D(T) = \begin{cases} 0\%, & T < 30, \\ \frac{T-30}{30} \times 100\%, & 30 \leq T \leq 60, \\ 100\%, & T > 60. \end{cases} \quad (2.16)$$

Một số giá trị đặc trưng được trình bày trong Bảng 2.3.

Bảng 2.3 Quan hệ giữa nhiệt độ và giá trị PWM

Nhiệt độ (°C)	Giá trị PWM	Duty cycle xấp xỉ
T<30	0	0%
30	0	0%
35	42	16.7%
40	85	33.3%
45	127	50.0%
50	170	66.7%
55	212	83.3%
60	255	100%
T>60	255	100%

Theo luật trên, động cơ không hoạt động khi nhiệt độ thấp hơn 30 C. Trong vùng từ 30 C đến 60 C, tốc độ động cơ tăng dần theo nhiệt độ. Khi nhiệt độ vượt quá 60 C, động cơ hoạt động với mức PWM cực đại.

Cách điều khiển này giúp công suất làm mát thay đổi theo điều kiện nhiệt độ, tránh việc động cơ luôn phải hoạt động ở tốc độ lớn nhất khi chưa cần thiết.

2.4 Màn hình LCD 16x2

LCD 16x2 được sử dụng làm thiết bị hiển thị trực tiếp của hệ thống. Màn hình có khả năng hiển thị hai dòng, mỗi dòng tối đa 16 ký tự, phù hợp để hiển thị các thông số cơ bản như nhiệt độ, độ ẩm, ngưỡng nhiệt độ và trạng thái hoạt động của hệ thống.

Việc tích hợp LCD cho phép người sử dụng quan sát trực tiếp dữ liệu đo mà không cần kết nối hệ thống với máy tính.

2.4.1 Cấu trúc và chân giao tiếp

LCD 16x2 thông dụng sử dụng bộ điều khiển tương thích HD44780. Các chân giao tiếp chính bao gồm:

- **RS (Register Select):** lựa chọn thanh ghi lệnh hoặc thanh ghi dữ liệu. Khi RS bằng 0, dữ liệu truyền tới LCD được hiểu là lệnh điều khiển; khi RS bằng 1, dữ liệu được hiểu là ký tự cần hiển thị.
- **RW (Read/Write):** lựa chọn thao tác đọc hoặc ghi dữ liệu.
- **EN (Enable):** tạo xung cho phép để LCD chốt dữ liệu trên bus dữ liệu.
- **D0–D7:** tám đường dữ liệu được sử dụng khi LCD hoạt động ở chế độ giao tiếp 8 bit.

Trong chương trình, các chân của LCD được ánh xạ với ATmega16 như Bảng 2.4.

Bảng 2.4 Kết nối giữa LCD 16x2 và ATmega16

Tín hiệu LCD	Chân ATmega16	Chức năng
D0–D7	PC0–PC7	Bus dữ liệu 8 bit.
RS	PD6	Lựa chọn thanh ghi lệnh hoặc dữ liệu.
RW	PD5	Lựa chọn chế độ đọc hoặc ghi.
EN	PD7	Tín hiệu cho phép để LCD chốt dữ liệu.

2.4.2 Chế độ giao tiếp 8 bit

Trong chế độ giao tiếp 8 bit, một byte dữ liệu hoặc một byte lệnh được truyền đồng thời qua tám đường D0–D7. Trong hệ thống, PORTC của ATmega16 được sử dụng làm bus dữ liệu:

```
#define LCD_DATA_PORT PORTC
```

```
#define LCD_DATA_DDR DDRC
```

Các chân điều khiển được bố trí trên PORTD:

```
#define LCD_CTRL_PORT PORTD
```

```
#define LCD_CTRL_DDR DDRD
```

```
#define RS PD6
```

```
#define RW PD5
```

```
#define EN PD7
```

Khi khởi tạo LCD, PORTC được cấu hình là ngõ ra. Các chân RS, RW và EN cũng được cấu hình ở chế độ ngõ ra:

```
LCD_DATA_DDR = 0xFF;
```

```
LCD_CTRL_DDR |= (1 << RS) | (1 << RW) | (1 << EN);
```

Sau khoảng thời gian chờ khởi động, chương trình gửi các lệnh cấu hình cơ bản tới LCD:

```
LCD_Command(0x38);
```

```
LCD_Command(0x0C);
```

```
LCD_Command(0x06);
```

```
LCD_Command(0x01);
```

Trong đó:

- 0x38: thiết lập giao tiếp 8 bit và chế độ hiển thị hai dòng;
- 0x0C: bật màn hình và tắt con trỏ;
- 0x06: tự động tăng vị trí con trỏ sau khi ghi ký tự;
- 0x01: xóa nội dung màn hình.

Khi gửi lệnh tới LCD, chương trình đặt dữ liệu lên PORTC, đưa chân RS về mức thấp và tạo xung tại chân EN. Khi gửi ký tự hiển thị, RS được đưa lên mức cao trước khi tạo xung EN.

Do PORTC của ATmega16 còn liên quan đến chức năng JTAG, chương trình cần vô hiệu hóa JTAG trước khi sử dụng toàn bộ PORTC làm GPIO. Việc này cho phép tám chân PC0–PC7 hoạt động ổn định với vai trò bus dữ liệu của LCD.

Trong chế độ hoạt động bình thường, LCD có thể hiển thị nhiệt độ và độ ẩm thu nhận từ DHT22, đồng thời hiển thị các thông tin như ngưỡng nhiệt độ, trạng thái động cơ hoặc thời gian hoạt động của hệ thống. Khi người dùng chuyển sang chế độ cài đặt, nội dung màn hình được thay đổi để phục vụ quá trình điều chỉnh các tham số bằng nút nhấn.

So với chế độ giao tiếp 4 bit, giao tiếp 8 bit sử dụng nhiều chân GPIO hơn, nhưng cho phép truyền toàn bộ một byte trong một lần ghi và đơn giản hóa quá trình lập trình điều khiển LCD.

2.5 Giao tiếp UART

UART (*Universal Asynchronous Receiver/Transmitter*) được sử dụng để trao đổi dữ liệu giữa ATmega16 và thiết bị bên ngoài mà không cần tín hiệu xung nhịp dùng chung. Trong đồ án, giao tiếp UART có hai nhiệm vụ chính: truyền các thông số hoạt động của hệ thống ra Virtual Terminal hoặc bộ chuyển đổi UART–USB và nhận các lệnh điều khiển động cơ từ thiết bị bên ngoài.

Việc sử dụng UART giúp quá trình kiểm thử và giám sát hệ thống thuận tiện hơn. Người sử dụng có thể theo dõi nhiệt độ, độ ẩm, giá trị PWM và trạng thái động cơ trên máy tính, đồng thời gửi lệnh điều khiển mà không cần thao tác trực tiếp trên bo mạch.

2.5.1 Khung truyền UART

UART truyền dữ liệu theo phương thức nối tiếp không đồng bộ. Do không có đường xung nhịp riêng, phía phát và phía nhận phải được cấu hình cùng tốc độ Baud và cùng định dạng khung truyền.

Trong chương trình, thanh ghi UCSRC được cấu hình:

$$\text{UCSRC} = (1 \ll \text{URSEL}) \mid (1 \ll \text{UCSZ1}) \mid (1 \ll \text{UCSZ0});$$

Hai bit UCSZ1 và UCSZ0 được đặt bằng 1, tương ứng với độ dài dữ liệu 8 bit. Các thông số còn lại sử dụng cấu hình mặc định gồm một bit Stop và không sử dụng bit Parity. Vì vậy, khung truyền UART của hệ thống có dạng 8N1, bao gồm:

- 1 bit Start;
- 8 bit dữ liệu;
- Không sử dụng bit kiểm tra chẵn lẻ;
- 1 bit Stop.

Mỗi ký tự ASCII được truyền dưới dạng một khung UART riêng. Ví dụ, ký tự '0'

và '1' được sử dụng làm các lệnh điều khiển trạng thái động cơ.

2.5.2 Cấu hình tốc độ 9600 baud

Trong đồ án, giao tiếp UART được cấu hình với tốc độ 9600 baud. Hàm khởi tạo UART nhận tốc độ Baud làm tham số và tính giá trị cho thanh ghi UBRR:

```
void UART_Init(unsigned long baud) {  
    unsigned int ubrr = (F_CPU / (16UL * baud)) - 1;  
  
    UBRRH = (unsigned char)(ubrr >> 8);  
    UBRL = (unsigned char)(ubrr);  
  
    UCSRB = (1 << RXEN) | (1 << TXEN) | (1 << RXCIE);  
    UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0);  
}
```

Ở chế độ UART không đồng bộ thông thường, giá trị UBRR được xác định bởi:

$$UBRR = \frac{f_{CPU}}{16 \times Baud} - 1. \quad (2.17)$$

Với $f_{CPU}=8$ MHz và tốc độ truyền Baud=9600, giá trị lý thuyết là:

$$UBRR = \frac{8 \times 10^6}{16 \times 9600} - 1 \approx 51.08. \quad (2.18)$$

Do thanh ghi UBRR nhận giá trị nguyên, chương trình sử dụng:

$$UBRR = 51. \quad (2.19)$$

Khi đó tốc độ Baud thực tế được xác định bởi:

$$Baud_{thuc} = \frac{8 \times 10^6}{16 \times (51 + 1)} \approx 9615.38 \text{ baud}. \quad (2.20)$$

Sai số so với tốc độ 9600 baud là nhỏ và đáp ứng yêu cầu của quá trình truyền nhận dữ liệu trong hệ thống.

2.5.3 Nhận dữ liệu bằng ngắt

Để tránh việc chương trình chính phải liên tục kiểm tra xem UART có dữ liệu mới hay không, chức năng ngắt khi nhận dữ liệu được sử dụng. Trong thanh ghi UCSRB, các bit RXEN, TXEN và RXCIE được kích hoạt:

$$\text{UCSRB} = (1 \ll \text{RXEN}) \mid (1 \ll \text{TXEN}) \mid (1 \ll \text{RXCIE});$$

Trong đó, RXEN cho phép bộ thu, TXEN cho phép bộ phát và RXCIE cho phép ngắt khi nhận hoàn tất một byte dữ liệu.

Chương trình phục vụ ngắt UART được xây dựng theo dạng:

```
ISR(USART_RXC_vect) {  
    char cmd = UDR;  
  
    if (cmd == '0') {  
        trang_thai_dong_co = 0;  
        OCR0 = 0;  
    }  
    else if (cmd == '1') {  
        trang_thai_dong_co = 1;  
    }  
}
```

Khi nhận ký tự '0', biến trạng thái động cơ được đặt về 0 và giá trị OCR0 được đưa về 0 để ngắt PWM, làm động cơ dừng. Khi nhận ký tự '1', hệ thống cho phép động cơ hoạt động trở lại. Giá trị PWM sau đó được xác định theo nhiệt độ đo được từ DHT22.

Ngoài việc nhận lệnh, UART còn được sử dụng để truyền trạng thái hệ thống ra Virtual Terminal trong Proteus hoặc máy tính thông qua bộ chuyển đổi UART–USB. Với phiên bản sử dụng DHT22, chuỗi dữ liệu có thể bao gồm các thông tin như nhiệt độ, độ ẩm, giá trị PWM và trạng thái động cơ, ví dụ:

Temp: 35.2 C | Humi: 62.5 % | PWM: 44 | Motor: RUN

Việc truyền thông tin trạng thái hỗ trợ theo dõi hoạt động của hệ thống và đối chiếu kết quả trong quá trình mô phỏng cũng như thử nghiệm phần cứng.

2.6 Timer, ngắt và cơ chế đa nhiệm đơn giản

Hệ thống không sử dụng hệ điều hành thời gian thực mà tổ chức các tác vụ theo mô hình vòng lặp chính kết hợp với các ngắt định kỳ. Các Timer bên trong ATmega16

được sử dụng để tạo mốc thời gian và kiểm tra trạng thái nút nhấn, trong khi chương trình chính thực hiện các tác vụ có thời gian xử lý lớn hơn như đọc cảm biến, cập nhật LCD, tính toán PWM và truyền dữ liệu UART.

Cách tổ chức này có thể xem là một cơ chế đa nhiệm đơn giản. Các hàm phục vụ ngắt chỉ thực hiện những công việc ngắn và cập nhật biến hoặc cờ trạng thái, sau đó chương trình chính xử lý các sự kiện tương ứng. Nhờ đó, hệ thống có thể thực hiện nhiều chức năng gần như đồng thời mà không cần sử dụng hệ điều hành chuyên dụng.

2.6.1 Timer1 phục vụ bộ đếm thời gian

Timer1 là bộ Timer/Counter 16 bit của ATmega16. Trong hệ thống, Timer1 được sử dụng để tạo mốc thời gian cho bộ đếm giờ, phút và giây.

Chương trình cấu hình Timer1 ở chế độ CTC (*Clear Timer on Compare Match*):

```
TCCR1B |= (1 << WGM12);
TCCR1B |= (1 << CS12);
OCR1A = 10000;
TIMSK |= (1 << OCIE1A);
```

Bit WGM12 được đặt bằng 1 để Timer1 hoạt động ở chế độ CTC. Trong chế độ này, bộ đếm TCNT1 tăng từ 0 đến giá trị được lưu trong OCR1A. Khi TCNT1 bằng OCR1A, sự kiện Compare Match xảy ra, bộ đếm được đưa về 0 và một ngắt được tạo nếu bit OCIE1A đã được kích hoạt.

Bit CS12 bằng 1, trong khi hai bit chọn xung còn lại bằng 0, tương ứng với hệ số chia tần số:

$$N = 256. \quad (2.21)$$

Chu kỳ ngắt Compare Match được xác định theo:

$$T_{\text{Timer1}} = \frac{(OCR1A + 1)N}{f_{\text{CPU}}}. \quad (2.22)$$

Với cấu hình hiện tại:

$$T_{\text{Timer1}} = \frac{(10000 + 1) \times 256}{8 \times 10^6} \approx 0.320 \text{ s}. \quad (2.23)$$

Như vậy, giá trị OCR1A = 10000 chưa tạo được chu kỳ ngắt 1 giây. Nếu mỗi lần ngắt thực hiện tăng biến giây một đơn vị thì bộ đếm thời gian sẽ chạy nhanh hơn thời gian thực.

Để tạo chu kỳ xấp xỉ 1 giây với $f_{CPU}=8\text{ MHz}$ và prescaler bằng 256, giá trị OCR1A cần được xác định theo:

$$OCR1A = \frac{f_{CPU} \times T}{N} - 1. \quad (2.24)$$

Với $T=1\text{ s}$:

$$OCR1A = \frac{8 \times 10^6}{256} - 1 = 31249. \quad (2.25)$$

Do đó, để bộ đếm thời gian hoạt động đúng, chương trình nên sử dụng:

```
OCR1A = 31249;
```

Hàm phục vụ ngắt thực hiện tăng biến giây và cập nhật phút, giờ:

```
ISR(TIMER1_COMPA_vect) {  
    giay++;  
  
    if (giay > 59) {  
        giay = 0;  
        phut++;  
    }  
  
    if (phut > 59) {  
        phut = 0;  
        gio++;  
    }  
  
    if (gio > 23) {  
        gio = 0;  
    }  
}
```

Các giá trị thời gian sau đó có thể được hiển thị trên LCD để theo dõi thời gian hoạt động của hệ thống.

2.6.2 Timer2 phục vụ quét phím và chống dội

Timer2 được sử dụng để tạo ngắt định kỳ phục vụ việc lấy mẫu trạng thái các nút SET, UP, DOWN và RESET. Cách làm này giúp chương trình không phải kiểm tra nút

nhấn liên tục trong vòng lặp chính.

Timer2 được cấu hình ở chế độ CTC với prescaler bằng 1024:

```
TCCR2 = (1 << WGM21)
        | (1 << CS22)
        | (1 << CS21)
        | (1 << CS20);
```

```
OCR2 = 78;
TIMSK |= (1 << OCIE2);
```

Với fCPU=8 MHz, chu kỳ một bước đếm của Timer2 là:

$$T_{\text{tick}} = \frac{1024}{8 \times 10^6} = 128 \mu\text{s}. \quad (2.26)$$

Chu kỳ ngắt Compare Match tương ứng với OCR2 = 78 là:

$$T_{\text{Timer2}} = (78 + 1) \times 128 \mu\text{s} \approx 10.11 \text{ ms}. \quad (2.27)$$

Do đó, trạng thái nút nhấn được lấy mẫu xấp xỉ mỗi 10 ms.

Trong hàm phục vụ ngắt, mỗi nút sử dụng một biến đếm trạng thái riêng. Nút nhấn chỉ được xác nhận khi trạng thái tác động được phát hiện trong nhiều lần lấy mẫu liên tiếp. Ví dụ:

```
if (!(PINB & (1 << PB0))) {
    if (st_up < 3) {
        st_up++;

        if (st_up == 2)
            c_btn_up = 1;
    }
}
else {
    st_up = 0;
}
```

Với chu kỳ lấy mẫu khoảng 10 ms và yêu cầu hai lần phát hiện liên tiếp, trạng thái nhấn được xác nhận sau khoảng 20 ms. Khoảng thời gian này đủ để loại bỏ phần lớn các dao động cơ học ngắn xuất hiện khi nút vừa được nhấn hoặc nhả.

Sau khi một nút được xác nhận, hàm ngắt chỉ đặt cờ tương ứng. Việc thay đổi chế độ hoặc giá trị cài đặt được thực hiện trong vòng lặp chính, nhờ đó thời gian thực thi của chương trình phục vụ ngắt được giữ ngắn.

2.7 Watchdog Timer

Watchdog Timer là bộ định thời hoạt động độc lập, được sử dụng để giám sát quá trình thực thi chương trình. Nếu chương trình bị treo hoặc không thực hiện được luồng xử lý bình thường trong một khoảng thời gian xác định, Watchdog sẽ hết thời gian và tạo tín hiệu Reset cho vi điều khiển.

Trong chương trình, Watchdog được kích hoạt bằng:

```
wdt_enable(WDTO_2S);
```

Tham số WDTO_2S thiết lập khoảng thời gian Watchdog xấp xỉ 2 giây. Trong điều kiện hoạt động bình thường, bộ đếm Watchdog phải được xóa định kỳ trước khi hết khoảng thời gian này.

Lệnh:

```
wdt_reset();
```

được gọi định kỳ trong vòng lặp chính. Khi chương trình hoạt động bình thường, lệnh này liên tục đưa bộ đếm Watchdog về trạng thái ban đầu. Nếu chương trình bị kẹt tại một vị trí bất thường và không thể thực hiện `wdt_reset()` trong thời gian quy định, Watchdog sẽ tự động Reset ATmega16.

Cơ chế này giúp hệ thống có khả năng tự phục hồi sau một số lỗi phần mềm, đặc biệt hữu ích đối với hệ thống nhúng cần hoạt động liên tục mà không có người giám sát.

2.8 Nút nhấn và chống dội phím

Hệ thống sử dụng các nút SET, UP, DOWN và RESET để người dùng thao tác trực tiếp với các tham số cài đặt. Các chân nút nhấn được cấu hình ở chế độ đầu vào:

```
DDRB &= ~( (1 << PB0)
           | (1 << PB1)
           | (1 << PB2)
           | (1 << PB4));
```

Điện trở kéo lên nội của ATmega16 được kích hoạt:

```

PORTB |= (1 << PB0)
        | (1 << PB1)
        | (1 << PB2)
        | (1 << PB4);

```

Do sử dụng điện trở kéo lên, khi nút không được nhấn, chân vào có mức logic 1. Khi nút được nhấn, chân tín hiệu được nối xuống GND và chuyển về mức logic 0. Như vậy, các nút hoạt động theo mức tích cực thấp (*active-low*).

Nút nhấn cơ khí thường xuất hiện hiện tượng dội tiếp điểm trong vài mili giây khi đóng hoặc mở. Nếu đọc trực tiếp trạng thái chân vào, một lần nhấn có thể bị chương trình nhận thành nhiều lần tác động.

Để khắc phục hiện tượng này, Timer2 lấy mẫu trạng thái nút với chu kỳ khoảng 10 ms. Mỗi nút có một bộ đếm riêng và chỉ được xác nhận khi trạng thái mức thấp xuất hiện liên tiếp trong hai lần lấy mẫu. Sau khi xác nhận, chương trình đặt một cờ sự kiện tương ứng.

Các cờ `c_btn_set`, `c_btn_up`, `c_btn_down` và `c_btn_reset` được xử lý trong vòng lặp chính. Phương pháp này vừa thực hiện chống dội, vừa hạn chế thời gian xử lý bên trong hàm phục vụ ngắt.

2.9 Khối nguồn và LM7805

Khối nguồn có nhiệm vụ cung cấp điện áp một chiều ổn định cho ATmega16, cảm biến DHT22, LCD 16x2 và các khối chức năng khác trên bo mạch. Điện áp cấp cho các mạch logic của hệ thống được ổn áp ở mức 5 V.

LM7805 là bộ ổn áp tuyến tính điện áp dương được sử dụng để tạo nguồn 5 V từ nguồn một chiều đầu vào có mức điện áp lớn hơn. Việc sử dụng bộ ổn áp giúp hạn chế sự thay đổi điện áp nguồn ảnh hưởng đến hoạt động của vi điều khiển và các ngoại vi số.

Các tụ lọc được bố trí ở phía đầu vào và đầu ra của LM7805 nhằm giảm nhiễu và cải thiện độ ổn định của điện áp. Ngoài các tụ lọc của khối nguồn, các tụ decoupling nên được đặt gần chân nguồn của vi điều khiển và các linh kiện số để hạn chế nhiễu cao tần trên đường cấp nguồn.

LED báo nguồn cùng điện trở hạn dòng được sử dụng để cho phép quan sát nhanh trạng thái cấp nguồn của bo mạch. Khi đường nguồn 5 V hoạt động, LED sáng cho biết bo mạch đã được cấp điện.

Trong trường hợp phiên bản PCB cuối sử dụng diode bảo vệ tại đầu vào nguồn, diode có nhiệm vụ hạn chế ảnh hưởng khi nguồn được đấu sai cực hoặc ngăn dòng điện chạy theo chiều không mong muốn.

Do LM7805 là bộ ổn áp tuyến tính, phần công suất tiêu tán trên linh kiện có thể xác định gần đúng theo:

$$P_{LM7805} = (V_{in} - 5)I_{load}, \quad (2.28)$$

trong đó V_{in} là điện áp đầu vào và I_{load} là dòng tải. Vì vậy, điện áp đầu vào và dòng tiêu thụ cần được lựa chọn phù hợp để hạn chế nhiệt lượng phát sinh trên bộ ổn áp.

2.10 Các khối hỗ trợ trên bo mạch

Ngoài các khối chức năng chính, bo mạch còn bố trí một số mạch hỗ trợ cho việc nạp chương trình, kiểm thử và mở rộng hệ thống. Các khối này không trực tiếp tham gia vào thuật toán điều khiển nhiệt độ nhưng giúp quá trình phát triển và sử dụng bo mạch thuận tiện hơn.

2.10.1 Mạch Reset và mạch nạp ISP

Mạch Reset cho phép đưa ATmega16 trở về trạng thái khởi tạo khi cần thiết. Chân RESET của vi điều khiển được duy trì ở trạng thái logic cao trong điều kiện làm việc bình thường và được kéo xuống mức thấp khi thực hiện Reset.

Ngoài khả năng Reset bằng phần cứng, bo mạch bố trí đầu nối ISP (*In-System Programming*) để cho phép nạp chương trình mà không cần tháo vi điều khiển khỏi mạch.

Các tín hiệu chính của giao tiếp ISP gồm:

- MOSI;
- MISO;
- SCK;
- RESET;
- VCC;
- GND.

Đầu nối ISP giúp quá trình lập trình, cập nhật firmware và kiểm thử bo mạch thuận tiện hơn trong giai đoạn phát triển.

2.10.2 Đầu nối UART-USB

Đầu nối UART được sử dụng để kết nối ATmega16 với bộ chuyển đổi UART-USB hoặc thiết bị giao tiếp nối tiếp bên ngoài.

Hai tín hiệu chính là:

- TXD của ATmega16 dùng để truyền dữ liệu;
- RXD của ATmega16 dùng để nhận dữ liệu.

Ngoài hai đường dữ liệu, hai thiết bị cần sử dụng chung mức tham chiếu GND. Khi kết nối, đường TX của một thiết bị được nối với RX của thiết bị còn lại.

Khối UART-USB cho phép sử dụng máy tính để quan sát dữ liệu từ hệ thống, gửi lệnh điều khiển và hỗ trợ quá trình gỡ lỗi chương trình.

2.10.3 LED đơn và LED 7 đoạn

Các LED đơn trên bo mạch có thể được sử dụng để báo nguồn hoặc hiển thị nhanh một số trạng thái logic trong quá trình kiểm thử. So với LCD hoặc UART, LED cung cấp phương pháp quan sát đơn giản và trực tiếp khi cần xác định trạng thái của một chân tín hiệu.

LED 7 đoạn là khối hiển thị số cơ bản có thể được sử dụng trong quá trình thử nghiệm các cổng GPIO, Timer hoặc thuật toán quét LED. Việc điều khiển LED 7 đoạn cũng là một bước kiểm tra hữu ích đối với khả năng xuất dữ liệu số của vi điều khiển.

Trong phiên bản hệ thống hoàn chỉnh, LCD 16x2 đảm nhiệm vai trò hiển thị chính. Vì vậy, LED đơn và LED 7 đoạn chủ yếu đóng vai trò hỗ trợ kiểm thử hoặc mở rộng chức năng của bo mạch.

2.10.4 Các đầu nối mở rộng

Các đầu nối mở rộng được bố trí nhằm đưa một số đường nguồn và tín hiệu của ATmega16 ra ngoài bo mạch. Các đầu nối này hỗ trợ việc kết nối thêm cảm biến, thiết bị chấp hành hoặc các mô-đun ngoại vi trong quá trình kiểm thử và phát triển hệ thống.

Việc bố trí đầu nối mở rộng cũng giúp quá trình đo tín hiệu bằng đồng hồ đo hoặc máy hiện sóng thuận tiện hơn mà không cần tiếp xúc trực tiếp với chân vi điều khiển.

Các đầu nối được sử dụng trong báo cáo cần được đối chiếu với phiên bản schematic và PCB cuối cùng để bảo đảm nội dung mô tả thống nhất với bo mạch được nộp.

2.11 Công cụ thiết kế và phát triển

Quá trình thực hiện đồ án sử dụng nhiều công cụ phần mềm cho các giai đoạn khác nhau, từ thiết kế mạch, lập trình vi điều khiển đến mô phỏng và kiểm thử.

2.11.1 Altium Designer

Altium Designer được sử dụng để thiết kế phần cứng của hệ thống. Quá trình thiết kế bao gồm xây dựng thư viện linh kiện, vẽ sơ đồ nguyên lý, gán footprint, bố trí linh kiện và đi dây PCB.

Sau khi hoàn thiện thiết kế, công cụ Design Rule Check được sử dụng để kiểm tra các quy tắc của PCB như khoảng cách giữa các đối tượng, kết nối chưa hoàn thành và các vi phạm liên quan đến đường mạch.

Altium Designer cũng được sử dụng để xuất các tệp Gerber và NC Drill phục vụ quá trình gia công PCB.

2.11.2 Proteus Design Suite

Proteus Design Suite được sử dụng để mô phỏng hoạt động của hệ thống trước khi triển khai trên phần cứng thực tế. Mô hình mô phỏng bao gồm ATmega16, cảm biến DHT22, LCD 16x2, động cơ, các nút nhấn và các thiết bị hỗ trợ kiểm tra tín hiệu.

Tập chương trình sau khi biên dịch được nạp vào mô hình ATmega16 trong Proteus. Qua đó có thể kiểm tra quá trình đọc cảm biến, hiển thị LCD, thay đổi PWM, xử lý nút nhấn và giao tiếp UART.

Virtual Terminal được sử dụng để quan sát dữ liệu UART và gửi các ký tự điều khiển tới vi điều khiển. Ngoài ra, các công cụ đo tín hiệu trong Proteus có thể được sử dụng để quan sát dạng xung PWM và kiểm chứng hoạt động của các khối thời gian.

2.11.3 Atmel Studio/Microchip Studio và AVR-GCC

Chương trình điều khiển ATmega16 được phát triển bằng ngôn ngữ C với bộ công cụ dành cho vi điều khiển AVR. Atmel Studio, hiện được phát triển dưới tên Microchip Studio, cung cấp môi trường xây dựng dự án, viết chương trình, biên dịch và quản lý các tệp nguồn.

AVR-GCC được sử dụng làm trình biên dịch C cho kiến trúc AVR. Các thư viện chuẩn như:

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/wdt.h>
#include <util/delay.h>
```

được sử dụng để truy cập các thanh ghi của vi điều khiển, quản lý ngắt, Watchdog Timer và tạo các khoảng trễ cần thiết.

Sau khi biên dịch thành công, chương trình tạo ra tệp thực thi và tệp HEX. Tệp HEX có thể được nạp vào ATmega16 thực tế thông qua mạch nạp ISP hoặc được sử dụng trực tiếp trong mô hình ATmega16 trên Proteus để mô phỏng hoạt động của hệ thống.

CHƯƠNG 3. THIẾT KẾ PHẦN CỨNG VÀ MẠCH IN PCB

3.1 Phân rã hệ thống phần cứng

Từ sơ đồ khối tổng thể, hệ thống được phân chia thành các khối chức năng riêng biệt nhằm thuận lợi cho quá trình thiết kế, kiểm tra và tích hợp. ATmega16 đóng vai trò là bộ xử lý trung tâm, thực hiện thu nhận dữ liệu từ cảm biến, xử lý các tín hiệu vào, điều khiển các thiết bị ngoại vi và trao đổi dữ liệu với thiết bị bên ngoài.

Cấu trúc phần cứng của hệ thống bao gồm các khối chính sau:

- **Khối nguồn:** tiếp nhận nguồn điện một chiều từ bên ngoài, thực hiện ổn áp xuống mức 5 V và cung cấp nguồn cho ATmega16 cùng các linh kiện ngoại vi.
- **Khối vi điều khiển:** sử dụng ATmega16 làm bộ điều khiển trung tâm, thực hiện xử lý dữ liệu từ cảm biến DHT22, tạo tín hiệu PWM, điều khiển LCD, xử lý nút nhấn, giao tiếp UART và quản lý các tác vụ thời gian.
- **Khối cảm biến:** sử dụng cảm biến DHT22 để thu nhận nhiệt độ và độ ẩm môi trường. Dữ liệu được truyền tới ATmega16 dưới dạng tín hiệu số thông qua một chân GPIO.
- **Khối hiển thị:** sử dụng LCD 16x2 để hiển thị các thông số như nhiệt độ, độ ẩm, ngưỡng nhiệt độ, thời gian và trạng thái hoạt động của hệ thống.
- **Khối nút nhấn:** gồm các nút SET, UP, DOWN và RESET, cho phép người sử dụng thay đổi chế độ và điều chỉnh giá trị ngưỡng nhiệt độ.
- **Khối giao tiếp UART:** sử dụng các chân RXD và TXD của ATmega16 để truyền dữ liệu trạng thái và nhận lệnh điều khiển từ Virtual Terminal hoặc bộ chuyển đổi UART-USB.
- **Khối điều khiển động cơ:** nhận tín hiệu PWM từ chân OC0 của ATmega16 và điều khiển công suất cấp cho động cơ hoặc quạt. Giá trị duty cycle được xác định theo nhiệt độ đo được từ DHT22.
- **Khối nạp và Reset:** cung cấp khả năng nạp chương trình thông qua giao tiếp ISP và đưa vi điều khiển trở về trạng thái khởi tạo khi cần thiết.
- **Các đầu nối mở rộng:** đưa một số đường nguồn và tín hiệu của vi điều khiển ra ngoài bo mạch, phục vụ quá trình đo kiểm hoặc kết nối thêm các thiết bị ngoại vi.

Việc phân chia hệ thống thành các khối chức năng giúp quá trình thiết kế sơ đồ nguyên lý và PCB có tính hệ thống hơn. Mỗi khối có thể được kiểm tra riêng trước khi

tích hợp, nhờ đó giảm sai sót trong quá trình phát triển phần cứng.

3.2 Phân bố chân vi điều khiển

Việc phân bố chân của ATmega16 được thực hiện dựa trên yêu cầu của từng khối ngoại vi và chức năng phần cứng tích hợp bên trong vi điều khiển. Các chân có chức năng đặc biệt như OC0, RXD và TXD được ưu tiên sử dụng đúng với chức năng phần cứng tương ứng để giảm khối lượng xử lý bằng phần mềm.

Bảng 3.1 trình bày phân bố các chân chính của ATmega16 trong hệ thống. Bảng này cần được giữ thống nhất giữa sơ đồ nguyên lý, thiết kế PCB và chương trình điều khiển.

Bảng 3.1 Phân bổ chân ATmega16 trong hệ thống

STT	Chân/Port	Khối ngoại vi	Chức năng	Ghi chú
1	PAx	DHT22	Truyền và nhận dữ liệu nhiệt độ, độ ẩm	GPIO hai chiều; thay PAx bằng chân thực tế theo schematic cuối.
2	PB0	Nút nhấn	Nút UP	Đầu vào, sử dụng pull-up nội.
3	PB1	Nút nhấn	Nút DOWN	Đầu vào, sử dụng pull-up nội.
4	PB2	Nút nhấn	Nút SET	Đầu vào, sử dụng pull-up nội.
5	PB3/OC0	Điều khiển động cơ	Xuất tín hiệu PWM	Ngõ ra phần cứng của Timer0.
6	PB4	Nút nhấn	Nút RESET chức năng	Đầu vào, sử dụng pull-up nội.
7	PC0–PC7	LCD 16x2	Bus dữ liệu D0–D7	Giao tiếp LCD ở chế độ 8 bit.
8	PD0/RXD	UART	Nhận dữ liệu UART	Nhận lệnh điều khiển từ thiết bị ngoài.
9	PD1/TXD	UART	Truyền dữ liệu UART	Gửi nhiệt độ, độ ẩm, PWM và trạng thái.
10	PD5	LCD 16x2	Tín hiệu RW	Điều khiển chế độ đọc/ghi LCD.
11	PD6	LCD 16x2	Tín hiệu RS	Lựa chọn thanh ghi lệnh hoặc dữ liệu.
12	PD7	LCD 16x2	Tín hiệu EN	Tạo xung cho phép LCD chốt dữ liệu.

Trong bảng trên, chân kết nối với DATA của DHT22 được cấu hình linh hoạt giữa ngõ ra và ngõ vào. Khi bắt đầu quá trình đọc cảm biến, ATmega16 sử dụng chân này làm ngõ ra để phát tín hiệu Start. Sau đó chân được chuyển sang chế độ ngõ vào để nhận

tín hiệu phản hồi và 40 bit dữ liệu từ DHT22.

Các chân PB0, PB1, PB2 và PB4 được sử dụng cho nút nhấn và kích hoạt điện trở kéo lên nội. Chân PB3 được lựa chọn cho tín hiệu PWM do đây là chân OC0 của Timer0, cho phép sử dụng bộ tạo PWM phần cứng.

PORTC được sử dụng toàn bộ làm bus dữ liệu 8 bit cho LCD 16x2. Các tín hiệu điều khiển của LCD được bố trí tại PD5, PD6 và PD7. Hai chân PD0 và PD1 được giữ cho chức năng UART phần cứng của ATmega16.

Việc xây dựng bảng phân bổ chân trước khi hoàn thiện sơ đồ nguyên lý giúp hạn chế xung đột giữa các ngoại vi và là cơ sở để kiểm tra sự thống nhất giữa firmware, schematic và PCB.

3.3 Thiết kế khối nguồn

Khối nguồn có nhiệm vụ tạo điện áp một chiều ổn định để cung cấp cho toàn bộ hệ thống. Các mạch logic trong đồ án, bao gồm ATmega16, cảm biến DHT22 và LCD 16x2, sử dụng mức điện áp nguồn 5 V.

Khối nguồn được thiết kế theo chuỗi chức năng:

Nguồn vào → Khối bảo vệ → LM7805 → Lọc nguồn → Đường nguồn 5 V.

Điện áp sau khi ổn áp được phân phối tới vi điều khiển và các khối ngoại vi. Tại các vị trí cần thiết, tụ lọc và tụ decoupling được sử dụng nhằm giảm ảnh hưởng của nhiễu trên đường nguồn.

3.3.1 Nguồn vào và bảo vệ

Nguồn một chiều từ bên ngoài được đưa vào bo mạch thông qua đầu nối nguồn. Điện áp đầu vào cần lớn hơn điện áp đầu ra 5 V một khoảng đủ để LM7805 có thể hoạt động trong vùng ổn áp.

Khối đầu vào cần bảo đảm đúng cực tính của nguồn và hạn chế các trường hợp đầu nối sai gây ảnh hưởng đến các linh kiện trên bo mạch. Nếu trên phiên bản schematic cuối có sử dụng diode bảo vệ, diode được bố trí tại đầu vào nguồn nhằm hạn chế ảnh hưởng của hiện tượng đầu ngược cực hoặc dòng điện chạy theo chiều không mong muốn.

Ngoài yêu cầu về điện áp, nguồn đầu vào cần có khả năng cung cấp dòng điện đủ cho toàn bộ hệ thống, đặc biệt khi các khối ngoại vi cùng hoạt động. Động cơ là tải có khả năng tiêu thụ dòng lớn và tạo nhiễu, do đó đường cấp nguồn cho động cơ cần được xem xét riêng so với đường nguồn logic của vi điều khiển.

3.3.2 Ổn áp 5 V sử dụng LM7805

LM7805 là bộ ổn áp tuyến tính điện áp dương được sử dụng để tạo đường nguồn 5 V ổn định từ nguồn điện một chiều đầu vào. Điện áp 5 V sau ổn áp được sử dụng để cấp cho ATmega16 và các mạch logic của hệ thống.

LM7805 gồm ba chân cơ bản:

- **IN:** nhận điện áp một chiều đầu vào;
- **GND:** chân tham chiếu 0 V;
- **OUT:** cung cấp điện áp ổn áp 5 V.

Do LM7805 là bộ ổn áp tuyến tính, phần chênh lệch giữa điện áp đầu vào và điện áp đầu ra được tiêu tán dưới dạng nhiệt. Công suất tiêu tán trên LM7805 có thể ước lượng theo:

$$P_{LM7805} = (V_{in} - V_{out}) I_{load}, \quad (3.1)$$

với:

- V_{in} là điện áp đầu vào LM7805;
- $V_{out} = 5 \text{ V}$ là điện áp đầu ra;
- I_{load} là dòng điện tải.

Khi điện áp đầu vào hoặc dòng tải tăng, công suất tổn hao trên LM7805 cũng tăng. Vì vậy, trong quá trình thiết kế cần bảo đảm điều kiện tản nhiệt phù hợp, đặc biệt khi bo mạch phải cấp nguồn cho nhiều tải cùng lúc.

Nguồn sau LM7805 được đưa tới các chân VCC của ATmega16 và các ngoại vi. Các chân nguồn tương tự như AVCC của vi điều khiển cũng cần được cấp nguồn đúng theo yêu cầu của ATmega16, kể cả trong trường hợp chức năng ADC không được sử dụng để đọc cảm biến DHT22.

3.3.3 Tụ lọc và LED báo nguồn

Các tụ điện được bố trí ở đầu vào và đầu ra của LM7805 nhằm tăng độ ổn định của bộ nguồn và giảm thành phần nhiễu trên đường cấp điện. Ngoài các tụ lọc có giá trị tương đối lớn tại khối nguồn, các tụ decoupling có giá trị nhỏ được bố trí gần các chân nguồn của vi điều khiển và các linh kiện số.

Vai trò của các tụ này bao gồm:

- Giảm dao động điện áp khi tải thay đổi;
- Hạn chế nhiễu cao tần trên đường nguồn;
- Cung cấp dòng tức thời cho các linh kiện số khi chuyển trạng thái;
- Cải thiện độ ổn định chung của hệ thống.

LED báo nguồn được kết nối với đường nguồn 5 V thông qua điện trở hạn dòng. Khi nguồn 5 V hoạt động, LED sáng để người sử dụng có thể nhanh chóng xác nhận trạng thái cấp nguồn của bo mạch.

Dòng điện qua LED có thể được xác định gần đúng theo:

$$I_{LED} = \frac{V_{CC} - V_{LED}}{R_{LED}}, \quad (3.2)$$

trong đó V_{CC} là điện áp nguồn, V_{LED} là điện áp rơi trên LED và R_{LED} là điện trở hạn dòng.

Việc sử dụng LED báo nguồn tuy không ảnh hưởng trực tiếp tới thuật toán điều khiển nhưng giúp quá trình lắp ráp, kiểm tra và phát hiện lỗi nguồn thuận tiện hơn.

3.4 Thiết kế khối vi điều khiển ATmega16

ATmega16 là khối xử lý trung tâm của hệ thống, thực hiện giao tiếp với cảm biến DHT22, xử lý dữ liệu nhiệt độ và độ ẩm, tạo tín hiệu PWM điều khiển động cơ, điều khiển LCD 16x2, xử lý nút nhấn và truyền nhận dữ liệu qua UART. Vì vậy, thiết kế phần cứng xung quanh ATmega16 cần bảo đảm nguồn cấp ổn định, hệ thống xung nhịp phù hợp và khả năng nạp chương trình thuận tiện.

Các mạch phụ trợ chính của ATmega16 gồm khối cấp nguồn VCC/AVCC, mạch dao động, mạch Reset và đầu nối nạp chương trình ISP.

3.4.1 Nguồn VCC/AVCC và GND

ATmega16 sử dụng các chân VCC và GND để cấp nguồn cho phần mạch số. Trong hệ thống, điện áp cấp cho vi điều khiển là 5 V được tạo bởi khối ổn áp LM7805.

Ngoài chân VCC, ATmega16 còn có chân AVCC dùng để cấp nguồn cho PORTA và khối ADC bên trong vi điều khiển. Mặc dù cảm biến DHT22 sử dụng giao tiếp số và không yêu cầu bộ ADC, chân AVCC vẫn cần được cấp nguồn đúng theo yêu cầu của vi điều khiển để bảo đảm PORTA có thể hoạt động ổn định khi được sử dụng làm GPIO.

Các chân GND của ATmega16 được nối với mặt phẳng mass chung của hệ thống. Đường nguồn và đường mass cần có chiều rộng phù hợp và hạn chế đi vòng xa nhằm giảm sụt áp và nhiễu.

Tụ decoupling được bố trí gần các chân nguồn của ATmega16 nhằm lọc thành phần nhiễu cao tần và cung cấp dòng tức thời khi các khối logic bên trong vi điều khiển chuyển trạng thái. Việc đặt tụ gần chân nguồn giúp tăng độ ổn định của hệ thống, đặc biệt khi động cơ và các tải chuyển mạch hoạt động đồng thời.

3.4.2 Mạch dao động thạch anh

Hệ thống xung nhịp quyết định tốc độ thực thi chương trình và ảnh hưởng trực tiếp tới các chức năng UART, Timer, PWM cũng như quá trình xác định độ rộng xung khi giao tiếp với DHT22.

Trong chương trình, tần số CPU được khai báo:

```
#define F_CPU 8000000UL
```

Do đó, hệ thống phần cứng cần được cấu hình để ATmega16 hoạt động với tần số 8 MHz hoặc phải điều chỉnh lại giá trị F_CPU trong chương trình cho phù hợp với tần số thực tế.

Trong trường hợp sử dụng thạch anh ngoài, thạch anh được kết nối giữa hai chân XTAL1 và XTAL2 của ATmega16. Hai tụ tải được nối từ hai đầu thạch anh xuống GND để hỗ trợ quá trình dao động.

Thạch anh và các tụ tải cần được bố trí gần vi điều khiển, đường mạch ngắn và hạn chế chạy gần các đường tín hiệu chuyển mạch có nhiễu cao. Giá trị thạch anh và tụ tải trong báo cáo cần được ghi đúng theo phiên bản schematic cuối cùng của bo mạch.

3.4.3 Mạch Reset

Chân RESET của ATmega16 hoạt động ở mức tích cực thấp. Trong trạng thái làm việc bình thường, chân này được duy trì ở mức logic cao thông qua điện trở kéo lên. Khi chân RESET bị kéo xuống mức thấp trong khoảng thời gian đủ lớn, ATmega16 sẽ thực hiện khởi động lại chương trình.

Mạch Reset phần cứng cho phép người sử dụng đưa vi điều khiển trở lại trạng thái khởi tạo trong quá trình nạp chương trình hoặc kiểm thử. Nút Reset được nối sao cho khi nhấn, chân RESET được kéo xuống GND.

Cần phân biệt nút Reset phần cứng của ATmega16 với nút RESET chức năng được chương trình sử dụng để đưa các tham số cài đặt về giá trị mặc định. Hai chức năng này có mục đích khác nhau và cần được thể hiện rõ trên sơ đồ nguyên lý cũng như silkscreen của PCB.

3.4.4 Đầu nối nạp chương trình

ATmega16 hỗ trợ nạp chương trình trực tiếp trên bo mạch bằng giao tiếp ISP (*In-System Programming*). Nhờ đó, vi điều khiển không cần tháo khỏi PCB mỗi khi cập nhật chương trình.

Các tín hiệu chính của giao tiếp ISP gồm:

- MOSI;
- MISO;
- SCK;
- RESET;
- VCC;
- GND.

Các đường MOSI, MISO và SCK được kết nối với các chân SPI tương ứng của ATmega16. Chân RESET được sử dụng để đưa vi điều khiển vào chế độ lập trình.

Đầu nối ISP cần được bố trí tại vị trí dễ tiếp cận trên PCB và ghi rõ tên hoặc hướng chân trên silkscreen nhằm hạn chế nhầm lẫn khi kết nối mạch nạp.

3.5 Thiết kế khối cảm biến nhiệt độ và độ ẩm DHT22

DHT22 được sử dụng để thu nhận nhiệt độ và độ ẩm của môi trường. Đây là cảm biến số, do đó dữ liệu không được đưa vào bộ ADC của ATmega16 mà được truyền trực tiếp thông qua một chân GPIO.

DHT22 có các kết nối chính gồm VCC, DATA và GND. Chân VCC được cấp nguồn phù hợp với hệ thống, chân GND nối với mass chung và chân DATA được kết nối với chân GPIO đã phân bổ cho cảm biến.

Đường DATA cần có điện trở kéo lên để giữ tín hiệu ở mức logic cao khi bus ở trạng thái nghỉ. Khi bắt đầu quá trình đọc, ATmega16 thay đổi hướng của chân GPIO để phát tín hiệu Start, sau đó chuyển chân này sang chế độ đầu vào để nhận tín hiệu phản hồi và 40 bit dữ liệu từ DHT22.

Nếu cảm biến được đặt ngoài bo mạch, PCB cần bố trí đầu nối cho các đường VCC, DATA và GND. Các chân của đầu nối phải được đánh dấu rõ ràng để tránh đấu nhầm.

Một tụ lọc nguồn có thể được bố trí gần cảm biến hoặc đầu nối cảm biến nhằm hạn chế ảnh hưởng của nhiễu trên đường nguồn. Giá trị linh kiện cần được ghi theo đúng schematic của bo mạch cuối.

3.6 Thiết kế khối LCD 16x2

LCD 16x2 được sử dụng làm giao diện hiển thị chính của hệ thống. Màn hình hiển thị các thông tin như nhiệt độ, độ ẩm, ngưỡng nhiệt độ, thời gian và trạng thái hoạt động của động cơ.

Trong thiết kế hiện tại, LCD được điều khiển theo chế độ giao tiếp 8 bit. Các chân D0–D7 của LCD được kết nối với PORTC của ATmega16. Ba tín hiệu điều khiển RS, RW và EN được kết nối với các chân GPIO thuộc PORTD.

Ảnh xạ tín hiệu được sử dụng trong chương trình gồm:

- PC0–PC7: các đường dữ liệu D0–D7;
- PD5: tín hiệu RW;
- PD6: tín hiệu RS;
- PD7: tín hiệu EN.

Chân VSS của LCD được nối GND và chân VDD được nối nguồn 5 V. Chân VO được sử dụng để điều chỉnh độ tương phản. Một biến trở được mắc giữa nguồn và GND, trong đó chân giữa của biến trở được nối tới VO.

Việc sử dụng biến trở cho phép điều chỉnh điện áp tại chân VO, từ đó lựa chọn độ tương phản thích hợp cho màn hình.

Nếu LCD được kết nối với bo mạch thông qua header, thứ tự các chân trên đầu nối cần được kiểm tra cẩn thận để thống nhất với footprint và silkscreen.

3.7 Thiết kế khối nút nhấn

Các nút SET, UP, DOWN và RESET chức năng được sử dụng để người dùng tương tác trực tiếp với hệ thống.

Chức năng của từng nút được xác định như sau:

- **SET:** chuyển đổi giữa chế độ hoạt động bình thường và chế độ cài đặt;
- **UP:** tăng giá trị tham số đang được cài đặt;
- **DOWN:** giảm giá trị tham số đang được cài đặt;
- **RESET:** đưa giá trị cài đặt về giá trị mặc định theo chương trình.

Trong phương án sử dụng điện trở kéo lên nội của ATmega16, một đầu của nút nhấn được nối với chân GPIO và đầu còn lại nối xuống GND. Khi nút không được nhấn, chân GPIO ở mức logic cao. Khi nút được nhấn, chân GPIO được kéo xuống mức logic thấp.

Do nút nhấn cơ khí xuất hiện hiện tượng dội tiếp điểm, chương trình thực hiện chống dội bằng cách lấy mẫu trạng thái nút theo chu kỳ Timer2 và chỉ công nhận một lần nhấn sau khi mức tín hiệu ổn định trong số lần lấy mẫu quy định.

3.8 Thiết kế khối UART

Khối UART cho phép ATmega16 trao đổi dữ liệu với máy tính hoặc thiết bị ngoài thông qua bộ chuyển đổi UART–USB.

Hai tín hiệu phân cứng được sử dụng gồm:

- PD0/RXD: nhận dữ liệu;
- PD1/TXD: truyền dữ liệu.

Đường TX của ATmega16 cần được kết nối với RX của module UART–USB, trong khi RX của ATmega16 được kết nối với TX của module. Hai mạch đồng thời phải sử dụng chung GND.

Giao tiếp được cấu hình ở tốc độ 9600 baud và sử dụng mức logic tương thích với ATmega16. Khi lựa chọn module UART–USB cần bảo đảm mức tín hiệu UART phù hợp với mức điện áp logic của bo mạch, tránh đưa trực tiếp tín hiệu có mức điện áp không tương thích vào chân vi điều khiển.

Khối UART được sử dụng để gửi nhiệt độ, độ ẩm, giá trị PWM và trạng thái động cơ tới máy tính. Đồng thời, ATmega16 có thể nhận các ký tự điều khiển như '0' và '1' để dừng hoặc cho phép động cơ hoạt động.

3.9 Thiết kế ngõ ra PWM và giao tiếp động cơ

Tín hiệu PWM được tạo bởi Timer0 của ATmega16 và xuất tại chân PB3/OC0. Duty cycle của tín hiệu được thay đổi theo nhiệt độ thu nhận từ cảm biến DHT22.

Chân GPIO của ATmega16 chỉ có khả năng cung cấp dòng nhỏ và không được thiết kế để cấp trực tiếp cho động cơ. Vì vậy, trong phần cứng thực tế, tín hiệu PWM cần được đưa tới tầng công suất hoặc mạch driver trước khi điều khiển động cơ.

Cấu trúc tổng quát của khối điều khiển có dạng:

ATmega16/OC0 → Tầng công suất/Driver → Động cơ.

Tầng công suất có nhiệm vụ khuếch đại khả năng cấp dòng và cách ly tải công suất khỏi chân vi điều khiển. Đối với tải cảm có thành phần cuộn dây như động cơ, cần có giải pháp hạn chế sức điện động ngược sinh ra khi dòng điện bị ngắt, chẳng hạn diode bảo vệ nếu cấu trúc driver yêu cầu.

Trong mô phỏng Proteus, động cơ có thể được kết nối với mô hình driver tương ứng để quan sát ảnh hưởng của duty cycle tới tốc độ quay. Trong báo cáo cuối, tên linh kiện và cấu trúc tầng công suất phải được trình bày đúng với schematic và PCB thực tế; không mô tả động cơ được nối trực tiếp với ATmega16 nếu bo mạch thực tế sử dụng tầng driver.

3.10 Các khối LED và LED 7 đoạn mở rộng

Ngoài các khối chức năng chính, bo mạch có thể bố trí các LED đơn và LED 7 đoạn để phục vụ kiểm thử hoặc mở rộng chức năng hiển thị.

LED đơn là phương tiện đơn giản để quan sát trạng thái logic của một số tín hiệu. Trong quá trình phát triển phần cứng, LED có thể được sử dụng để kiểm tra hoạt động của các chân GPIO hoặc biểu diễn trạng thái của hệ thống.

Mỗi LED cần có điện trở hạn dòng mắc nối tiếp nhằm giới hạn dòng điện qua LED. Giá trị điện trở cần được lựa chọn phù hợp với điện áp nguồn và loại LED sử dụng.

LED 7 đoạn có thể được sử dụng để kiểm tra khả năng điều khiển GPIO, thực hiện quét hiển thị hoặc hiển thị các giá trị số đơn giản. Trong phiên bản hệ thống hoàn chỉnh, LCD 16x2 đóng vai trò hiển thị chính nên LED 7 đoạn chủ yếu phục vụ kiểm thử và phát triển.

Nếu phiên bản PCB nộp cuối không còn sử dụng LED 7 đoạn thì mục này cần được loại bỏ khỏi báo cáo để bảo đảm nội dung thống nhất với phần cứng thực tế.

3.11 Xây dựng thư viện linh kiện

Trong quá trình thiết kế trên Altium Designer, một số linh kiện được xây dựng thư viện riêng để bảo đảm ký hiệu nguyên lý và footprint phù hợp với linh kiện thực tế.

Thư viện linh kiện được chia thành hai phần chính gồm Schematic Library và PCB Library.

3.11.1 Schematic Library

Schematic Library chứa các ký hiệu dùng trong sơ đồ nguyên lý. Khi xây dựng một linh kiện mới, các thông tin quan trọng cần được kiểm tra gồm:

- Số thứ tự chân;
- Tên và chức năng của từng chân;
- Kiểu điện của chân;
- Ký hiệu đồ họa của linh kiện;

- Tên Designator và Comment;
- Mỗi liên hệ với footprint của linh kiện.

Số thứ tự chân của symbol phải trùng với số pad trên footprint. Đây là yêu cầu quan trọng vì nếu mapping giữa schematic và footprint bị sai, các đường mạch trên PCB có thể kết nối tới chân vật lý không đúng mặc dù sơ đồ nguyên lý vẫn hiển thị bình thường.

Sau khi tạo symbol, các chân cần được đối chiếu với datasheet của linh kiện trước khi sử dụng trong schematic.

3.11.2 PCB Library và footprint

PCB Library chứa footprint thể hiện hình dạng vật lý của linh kiện trên bo mạch. Quá trình xây dựng footprint cần dựa vào kích thước thực tế hoặc bản vẽ cơ khí trong datasheet của linh kiện.

Các thông số chính cần kiểm tra gồm:

- Khoảng cách giữa các chân;
- Kích thước và hình dạng pad;
- Đường kính lỗ khoan đối với linh kiện xuyên lỗ;
- Kích thước thân linh kiện;
- Ký hiệu chân số 1;
- Đường silkscreen;
- Khoảng hở phục vụ lắp ráp.

Đối với các linh kiện xuyên lỗ, đường kính lỗ khoan cần lớn hơn đường kính chân linh kiện một khoảng phù hợp để bảo đảm khả năng lắp ráp nhưng không làm giảm độ bền cơ khí của mối hàn.

Sau khi hoàn thành footprint, symbol trong Schematic Library được liên kết với footprint tương ứng. Quá trình mapping cần được kiểm tra lại trước khi cập nhật thiết kế từ schematic sang PCB.

3.12 Sơ đồ nguyên lý hoàn chỉnh

Sau khi hoàn thiện thiết kế từng khối chức năng, các khối được tích hợp thành sơ đồ nguyên lý hoàn chỉnh của hệ thống.

Sơ đồ nguyên lý tổng thể thể hiện các mối liên hệ giữa khối nguồn, ATmega16, DHT22, LCD 16x2, nút nhấn, UART, mạch PWM điều khiển động cơ, mạch Reset, đầu nối ISP và các khối mở rộng.

Trước khi chuyển sang thiết kế PCB, sơ đồ nguyên lý cần được kiểm tra về:

- Nguồn cấp và mass của tất cả linh kiện;
- Số thứ tự và chức năng chân của vi điều khiển;
- Tên các Net;
- Chiều của các tín hiệu UART;
- Kết nối tín hiệu DATA của DHT22;
- Chân OC0 dùng cho PWM;
- Các chân LCD;
- Các chân ISP;
- Các đầu nối ra ngoài bo mạch.

3.13 Thiết kế PCB

Sau khi sơ đồ nguyên lý được kiểm tra, các linh kiện và kết nối được chuyển sang PCB để thực hiện bố trí và đi dây. Quá trình thiết kế PCB cần đáp ứng đồng thời các yêu cầu về kết nối điện, khả năng chế tạo và thuận tiện khi lắp ráp.

3.13.1 Thiết lập Board Shape và Design Rules

Board Shape được thiết lập dựa trên kích thước mong muốn của bo mạch và vị trí dự kiến của các đầu nối.

Các Design Rules được thiết lập nhằm quy định các thông số như:

- Chiều rộng nhỏ nhất của đường mạch;
- Khoảng cách nhỏ nhất giữa các đường đồng;
- Kích thước via;
- Khoảng cách giữa pad và các đối tượng khác;
- Quy tắc liên quan đến vùng đổ đồng;
- Các yêu cầu công nghệ của đơn vị gia công PCB.

Việc thiết lập Design Rules trước khi đi dây giúp hạn chế việc phải chỉnh sửa lại PCB ở giai đoạn cuối và là cơ sở cho quá trình kiểm tra DRC.

3.13.2 Bố trí linh kiện

Linh kiện được bố trí theo từng khối chức năng để giảm chiều dài đường mạch và giúp PCB dễ kiểm tra.

ATmega16 được đặt ở khu vực trung tâm để thuận lợi cho việc kết nối tới các khối ngoại vi. Các tụ decoupling được đặt gần chân nguồn của vi điều khiển. Mạch dao động được bố trí gần XTAL1 và XTAL2.

Các đầu nối nguồn, UART, ISP, DHT22, LCD và động cơ nên được đặt gần mép bo mạch để thuận tiện khi kết nối với thiết bị bên ngoài.

Khối nguồn và khối công suất điều khiển động cơ cần được bố trí hợp lý để hạn chế ảnh hưởng của nhiễu tới khu vực vi điều khiển và cảm biến.

3.13.3 Đi dây nguồn và tín hiệu

Quá trình đi dây được thực hiện sau khi hoàn tất bố trí linh kiện. Các đường nguồn và đường cấp cho tải có dòng lớn được ưu tiên sử dụng chiều rộng lớn hơn các đường tín hiệu logic.

Các đường tín hiệu có yêu cầu đặc biệt như thạch anh, UART và DATA của DHT22 được giữ ngắn và hạn chế đi gần các đường có nhiều chuyển mạch lớn.

Đường PWM từ ATmega16 tới tầng công suất được đi hợp lý để hạn chế ghép nhiễu sang các tín hiệu cảm biến và xung nhịp.

Các đường nguồn và tín hiệu cần hạn chế tạo vòng dây không cần thiết và giảm số lượng via khi có thể.

3.13.4 Đổ đồng và xử lý mass

Sau khi hoàn thiện các đường tín hiệu chính, vùng đồng nối GND được đổ trên các lớp phù hợp nhằm tạo đường hồi dòng có trở kháng thấp và cải thiện khả năng chống nhiễu.

Các vùng đồng cần được kiểm tra để bảo đảm không tạo ra các vùng đồng cô lập không có kết nối điện. Những khu vực có dòng tải lớn cần được xem xét riêng để hạn chế dòng nhiễu đi qua vùng mass của các mạch tín hiệu nhạy.

Khi sử dụng PCB hai lớp, có thể sử dụng via để kết nối các vùng GND trên hai lớp nhằm cải thiện tính liên tục của mặt phẳng mass.

3.13.5 Silkscreen và ký hiệu đầu nối

Silkscreen được sử dụng để thể hiện các thông tin hỗ trợ quá trình lắp ráp và sử dụng bo mạch.

Các nội dung cần được thể hiện rõ gồm:

- Designator của linh kiện;
- Ký hiệu cực tính của diode, LED và tụ phân cực;
- Chân số 1 của IC và đầu nối;
- Tên các đầu nối nguồn;
- Tên các nút SET, UP, DOWN và RESET;
- Các tín hiệu TX, RX, VCC và GND;
- Hướng đầu nối ISP;
- Đầu nối DHT22, LCD và động cơ.

Chữ silkscreen không nên chồng lên pad hoặc lỗ khoan và cần có kích thước đủ lớn để có thể đọc được sau khi PCB được gia công.

3.14 Kiểm tra DRC

Sau khi hoàn tất bố trí và đi dây, PCB được kiểm tra bằng chức năng Design Rule Check của Altium Designer.

DRC giúp phát hiện các vi phạm liên quan đến các quy tắc đã thiết lập, chẳng hạn:

- Khoảng cách giữa các đường mạch không đạt yêu cầu;
- Đường mạch hoặc pad chưa được kết nối;
- Vi phạm chiều rộng đường mạch;
- Via hoặc lỗ khoan có kích thước không phù hợp;
- Vùng đồng vi phạm khoảng cách;
- Các đối tượng PCB nằm ngoài giới hạn thiết kế.

Sau mỗi lần kiểm tra, các lỗi được xem xét và chỉnh sửa trực tiếp trên PCB. Quá trình kiểm tra được lặp lại cho tới khi thiết kế đạt trạng thái phù hợp với các Design Rules đã đặt ra.

Kết quả kiểm tra của phiên bản PCB cuối cần được ghi theo báo cáo DRC thực tế. Có thể trình bày kết quả bằng Bảng 3.2.

Bảng 3.2 Kết quả kiểm tra DRC của PCB

Nội dung	Trước xử lý	Sau xử lý
Số lỗi DRC	<i>Điền theo báo cáo DRC</i>	<i>Điền theo báo cáo DRC cuối</i>
Số cảnh báo	<i>Điền theo báo cáo DRC</i>	<i>Điền theo báo cáo DRC cuối</i>

Chỉ có thể kết luận thiết kế không còn lỗi DRC khi báo cáo kiểm tra của phiên bản cuối xác nhận số vi phạm bằng 0.

3.15 Xuất file chế tạo Gerber và NC Drill

Sau khi PCB hoàn thiện và được kiểm tra, các tệp dữ liệu phục vụ chế tạo được xuất từ Altium Designer.

Bộ Gerber mô tả các lớp vật lý của PCB, trong khi tệp NC Drill chứa dữ liệu vị trí và kích thước các lỗ khoan.

Đối với PCB hai lớp, bộ dữ liệu chế tạo thường bao gồm:

- Top Copper: lớp đồng mặt trên;
- Bottom Copper: lớp đồng mặt dưới;
- Top Solder Mask: lớp chống hàn mặt trên;
- Bottom Solder Mask: lớp chống hàn mặt dưới;
- Top Overlay: lớp silkscreen mặt trên;
- Bottom Overlay nếu được sử dụng;
- Mechanical hoặc Board Outline: đường biên của bo mạch;
- NC Drill: dữ liệu các lỗ khoan và via.

Sau khi xuất, các tệp Gerber cần được mở bằng Gerber Viewer để kiểm tra trực quan trước khi gửi gia công. Các nội dung cần kiểm tra gồm kích thước board, đường đồng, solder mask, silkscreen, lỗ khoan và sự trùng khớp giữa các lớp.

3.16 Danh sách linh kiện BOM

Danh sách linh kiện BOM (*Bill of Materials*) được xây dựng từ schematic của phiên bản bo mạch cuối. BOM cung cấp thông tin cần thiết cho quá trình mua linh kiện và lắp ráp PCB.

Một BOM đầy đủ cần bao gồm Designator, tên hoặc loại linh kiện, giá trị, package, số lượng và các ghi chú liên quan.

Bảng 3.3 Danh sách linh kiện chính của hệ thống

STT	Designator	Linh kiện/Giá trị	Package	SL	Ghi chú
1	U1	ATmega16	<i>Theo PCB</i>	1	Vi điều khiển trung tâm
2	U2	LM7805	<i>Theo PCB</i>	1	Ổn áp 5 V
3	J...	DHT22/đầu nối DHT22	<i>Theo PCB</i>	1	Cảm biến nhiệt độ và độ ẩm
4	LCD1	LCD 16x2	Header	1	Màn hình hiển thị
5	Y1	Thạch anh	<i>Điền giá trị</i>	1	Xung nhịp MCU
6	SW...	Nút nhấn	<i>Theo PCB</i>	<i>Điền</i>	SET/UP/DOWN/RESET
7	R...	Điện trở	<i>Theo PCB</i>	<i>Điền</i>	Pull-up, hạn dòng, phân cực
8	C...	Tụ điện	<i>Theo PCB</i>	<i>Điền</i>	Lọc nguồn, decoupling, dao động
9	D...	LED/Diode	<i>Theo PCB</i>	<i>Điền</i>	Báo nguồn hoặc bảo vệ
10	J...	Header ISP	<i>Theo PCB</i>	1	Nạp chương trình
11	J...	Header UART	<i>Theo PCB</i>	1	Kết nối UART-USB
12	Q/U...	Tăng công suất động cơ	<i>Theo PCB</i>	<i>Điền</i>	Điều khiển tải từ PWM

Các trường chưa xác định trong Bảng 3.3 cần được hoàn thiện bằng cách xuất BOM trực tiếp từ Altium Designer hoặc đối chiếu với schematic của phiên bản bo mạch cuối. Việc sử dụng BOM được tạo từ chính thiết kế giúp hạn chế sai lệch giữa nội dung báo cáo, PCB và linh kiện thực tế.

CHƯƠNG 4. THIẾT KẾ PHẦN MỀM VÀ MÔ PHỎNG

Chương này trình bày quá trình xây dựng phần mềm điều khiển cho hệ thống sử dụng vi điều khiển ATmega16, từ kiến trúc tổng thể, cấu hình ngoại vi, thuật toán xử lý đến cách tổ chức các chương trình phục vụ hiển thị, giao tiếp và điều khiển động cơ. Nội dung được phân tích trực tiếp từ chương trình main.c của đồ án và đối chiếu với tài liệu kỹ thuật ATmega16, cảm biến DHT22/AM2302 và bộ điều khiển LCD HD44780U. ATmega16 cung cấp các khối Timer/Counter, USART, Watchdog, ngắt và các cổng I/O lập trình được, phù hợp với việc tổ chức hệ thống theo mô hình vòng lặp chính kết hợp các chương trình phục vụ ngắt [1].

Phiên bản phần mềm cuối sử dụng DHT22 theo giao tiếp số một dây tại chân PA0. Vì vậy, mặc dù PA0 đồng thời có chức năng ADC0 của ATmega16, bộ ADC không được sử dụng để đo nhiệt độ trong phiên bản này. Giá trị nhiệt độ được cảm biến số hóa và truyền trực tiếp về vi điều khiển dưới dạng khung dữ liệu 40 bit.

4.1 Yêu cầu phần mềm

Phần mềm được xây dựng để thực hiện đồng thời các nhiệm vụ sau:

- Đọc nhiệt độ từ cảm biến DHT22 và phát hiện trường hợp mất kết nối hoặc dữ liệu sai checksum.
- Tính mức điều khiển PWM theo nhiệt độ và xuất PWM bằng phần mềm tại chân PB4 để điều khiển tăng công suất động cơ.
- Hiển thị nhiệt độ, thời gian hoạt động, trạng thái cảnh báo và giao diện cài đặt trên LCD ký tự 16x2.
- Cho phép thay đổi ngưỡng cảnh báo bằng bốn nút nhấn SET, UP, DOWN và RESET.
- Chống dôi nút nhấn bằng cơ chế lấy mẫu định kỳ từ ngắt Timer2.
- Tạo bộ đếm thời gian giờ–phút–giây bằng Timer1.
- Gửi nhiệt độ và mức PWM qua UART; nhận lệnh ký tự '0' và '1' để dừng hoặc cho phép động cơ hoạt động.
- Sử dụng Watchdog Timer để tăng khả năng tự khôi phục khi chương trình bị treo.
- Khi cảm biến không phản hồi, đưa động cơ về trạng thái dừng an toàn và hiển thị trạng thái chờ kết nối.

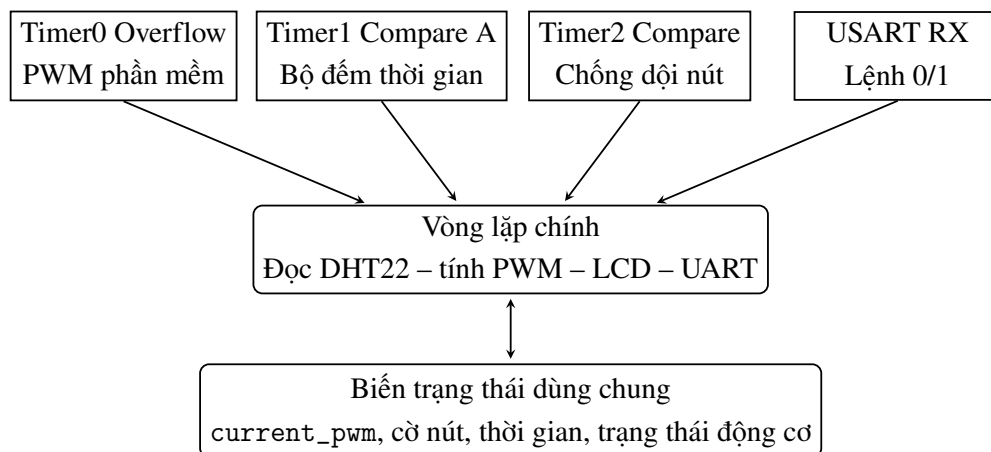
4.2 Kiến trúc chương trình

4.2.1 Mô hình vòng lặp chính kết hợp ngắt

Chương trình được tổ chức theo mô hình *super-loop*: sau khi khởi tạo phần cứng, hàm `main()` chạy liên tục trong vòng lặp `while(1)`. Các tác vụ cần độ chính xác thời gian hoặc cần phản ứng ngay với ngoại vi được chuyển sang các chương trình phục vụ ngắt (ISR). Cách tổ chức này giúp phần mềm đơn giản, phù hợp với quy mô của đề án và không cần hệ điều hành thời gian thực.

Bốn luồng ngắt được sử dụng gồm:

- `TIMER0_OVF_vect`: tạo PWM bằng phần mềm trên PB4.
- `TIMER1_COMP_A_vect`: tạo mốc thời gian một giây và cập nhật giờ–phút–giây.
- `TIMER2_COMP_vect`: lấy mẫu trạng thái nút nhấn và chống dội.
- `USART_RXC_vect`: nhận lệnh điều khiển động cơ từ UART.



Hình 4.1 Kiến trúc phần mềm và các luồng ngắt của chương trình

4.2.2 Phân bổ chân I/O theo chương trình

Ánh xạ chân được xác định trực tiếp từ các lệnh `#define` và phần cấu hình DDR trong `main.c`. Bảng 4.1 tổng hợp các chân mà phần mềm sử dụng.

Bảng 4.1 Phân bổ chân I/O trong chương trình

Chân/Port	Chức năng	Vai trò trong phần mềm
PC0–PC7	LCD D0–D7	Bus dữ liệu 8 bit cho LCD 16x2
PD6	LCD RS	Chọn thanh ghi lệnh hoặc dữ liệu
PD5	LCD R/W	Chọn đọc/ghi; chương trình luôn ghi
PD7	LCD EN	Tạo xung chốt dữ liệu/lệnh
PA0	DHT22 DATA	Giao tiếp số một dây với DHT22
PD0	RXD	Nhận lệnh UART
PD1	TXD	Truyền dữ liệu UART
PB0	UP	Nút tăng ngưỡng, tác động mức thấp
PB1	DOWN	Nút giảm ngưỡng, tác động mức thấp
PB2	SET	Vào/thoát chế độ cài đặt
PB3	RESET	Khôi phục giá trị ngưỡng trong chế độ cài đặt
PB4	PWM OUT	Tín hiệu PWM phần mềm đưa đến tăng công suất động cơ

Theo datasheet, PC2–PC5 còn được dùng cho giao tiếp JTAG. Do toàn bộ PORTC được dùng làm bus dữ liệu LCD, chương trình vô hiệu hóa JTAG bằng cách ghi bit JTD trong thanh ghi MCUCSR hai lần liên tiếp. Datasheet ATmega16 quy định bit JTD phải được ghi cùng một giá trị hai lần trong vòng bốn chu kỳ máy để tránh thay đổi ngoài ý muốn [1].

4.2.3 Các biến trạng thái dùng chung

Các biến có thể bị thay đổi trong ISR được khai báo với từ khóa `volatile`. Đây là yêu cầu quan trọng để trình biên dịch không tối ưu hóa việc đọc/ghi các biến mà không xét đến khả năng chúng thay đổi bất đồng bộ do ngắt.

```

1 volatile uint8_t trang_thai_dong_co = 1;
2 volatile uint8_t giay = 0, phut = 0, gio = 0;
3 volatile uint8_t c_btn_set = 0;
4 volatile uint8_t c_btn_up = 0;
5 volatile uint8_t c_btn_down = 0;
6 volatile uint8_t c_btn_reset = 0;
7 volatile uint8_t current_pwm = 0;
8

```

```

9 float nguong_nhiet_do = 30.0;
10 uint8_t che_do_cai_dat = 0;

```

Listing 1: Các biến trạng thái chính dùng giữa vòng lặp và ISR

Trong đó, `current_pwm` là biến trung gian giữa thuật toán tính tốc độ ở vòng lặp chính và ISR Timer0; các cờ `c_btn_*` là cầu nối giữa ISR Timer2 và phần xử lý giao diện; các biến giây, phút, giờ được cập nhật trong ISR Timer1 và đọc khi hiển thị lên LCD.

4.3 Tổ chức mã nguồn theo chức năng

Để thuận tiện cho việc kiểm tra và bảo trì, chương trình được chia thành các nhóm hàm theo ngoại vi. Bảng 4.2 mô tả vai trò của các hàm chính.

Bảng 4.2 Các hàm và chương trình phục vụ ngắt trong mã nguồn

Hàm/ISR	Chức năng
LCD_Command, LCD_Char	Ghi lệnh và ký tự lên LCD
LCD_Init, LCD_String, LCD_Gotoxy	Khởi tạo LCD, ghi chuỗi và đặt vị trí con trỏ
UART_Init	Cấu hình USART ở 9600 bit/s, 8 bit dữ liệu
UART_Transmit, UART_String	Truyền một ký tự hoặc một chuỗi
ISR(USART_RXC_vect)	Nhận lệnh dừng/chạy động cơ
DHT22_ReadData	Thực hiện giao thức DHT22, kiểm tra checksum và tách nhiệt độ
Timer1_Init	Tạo mốc thời gian một giây
ISR(TIMER1_COMPA_vect)	Cập nhật giờ–phút–giây
Timer2_Init	Tạo chu kỳ lấy mẫu nút khoảng 10,112 ms
ISR(TIMER2_COMP_vect)	Chống dội và sinh cờ sự kiện nút
Timer0_SoftPWM_Init	Khởi tạo Timer0 cho PWM phần mềm
ISR(TIMER0_OVF_vect)	Điều khiển mức PB4 theo <code>current_pwm</code>
main	Khởi tạo toàn hệ thống và thực hiện thuật toán điều khiển chính

4.4 Khởi tạo hệ thống

4.4.1 Vô hiệu hóa JTAG và cấu hình GPIO

Ngay đầu hàm `main()`, JTAG được vô hiệu hóa để giải phóng các chân PC2–PC5 cho LCD. Sau đó PB0–PB3 được đặt ở chế độ input và bật điện trở kéo lên nội; do đó các nút nhấn được xử lý theo logic tác động mức thấp. PB4 được đặt ở chế độ output để phát PWM.

```
1 MCUCSR |= (1<<JTD);
2 MCUCSR |= (1<<JTD);
3
4 DDRB &= ~( (1<<PB0) | (1<<PB1) | (1<<PB2) | (1<<PB3) );
5 PORTB |= (1<<PB0) | (1<<PB1) | (1<<PB2) | (1<<PB3);
6 DDRB |= (1 << PB4);
```

Listing 2: Khởi tạo JTAG và PORTB

Khi một nút không được nhấn, điện trở kéo lên giữ đầu vào ở mức logic 1. Khi nút nổi chân tương ứng xuống GND, giá trị đọc từ PINB trở thành 0. Cách cấu hình này giảm số điện trở kéo ngoài và phù hợp với cơ chế pull-up nội của các cổng I/O ATmega16 [1].

4.4.2 Trình tự khởi tạo ngoại vi

Sau khi cấu hình GPIO, chương trình lần lượt khởi tạo UART, LCD, Timer1, Timer2 và Timer0. Watchdog được đặt ở mức danh định 2 s, sau đó lệnh `sei()` cho phép ngắt toàn cục.

```
1 UART_Init(9600);
2 LCD_Init();
3 Timer1_Init();
4 Timer2_Init();
5 Timer0_SoftPWM_Init();
6
7 wdt_enable(WDTO_2S);
8 sei();
```

Listing 3: Trình tự khởi tạo các khối chức năng

Thư viện AVR-Libc định nghĩa WDTO_2S cho khoảng timeout Watchdog xấp xỉ 2 s [6]. Với ATmega16, datasheet cho giá trị điển hình khoảng 2,1 s ở 5 V khi bộ chia Watchdog ở mức lớn nhất [1].

4.4.3 Màn hình khởi động

Sau khởi tạo, hệ thống duy trì màn hình “Đang kết nối...” trong 40 vòng, mỗi vòng trễ 50 ms, tương đương khoảng 2 s. Trong thời gian này Watchdog vẫn được reset và

đồng hồ phần mềm vẫn có thể được cập nhật bởi Timer1.

4.5 Giao tiếp LCD 16x2

4.5.1 Nguyên tắc giao tiếp

Chương trình sử dụng LCD ở chế độ dữ liệu 8 bit. PORTC mang DB0–DB7, còn RS, R/W và EN lần lượt nối với PD6, PD5 và PD7. Với HD44780U, RS = 0 chọn thanh ghi lệnh, RS = 1 chọn thanh ghi dữ liệu; R/W = 0 tương ứng thao tác ghi [3].

Hàm LCD_Command() đặt RS = 0, R/W = 0 và tạo xung trên EN để ghi lệnh. Hàm LCD_Char() thực hiện tương tự nhưng đặt RS = 1 để ghi dữ liệu ký tự.

```
1 void LCD_Command(unsigned char cmd) {
2     LCD_DATA_PORT = cmd;
3     LCD_CTRL_PORT &= ~(1 << RS);
4     LCD_CTRL_PORT &= ~(1 << RW);
5     LCD_CTRL_PORT |= (1 << EN);
6     _delay_us(1);
7     LCD_CTRL_PORT &= ~(1 << EN);
8     _delay_ms(2);
9 }
10
11 void LCD_Char(unsigned char data) {
12     LCD_DATA_PORT = data;
13     LCD_CTRL_PORT |= (1 << RS);
14     LCD_CTRL_PORT &= ~(1 << RW);
15     LCD_CTRL_PORT |= (1 << EN);
16     _delay_us(1);
17     LCD_CTRL_PORT &= ~(1 << EN);
18     _delay_ms(1);
19 }
```

Listing 4: Ghi lệnh và dữ liệu lên LCD

Do chương trình không đọc cờ Busy Flag của LCD nên sử dụng thời gian chờ cố định. Datasheet HD44780U cho biết lệnh xóa màn hình có thời gian thực hiện lớn hơn đáng kể so với phần lớn lệnh thông thường; việc dùng thời gian chờ đến 2 ms sau lệnh giúp đảm bảo LCD hoàn thành thao tác trước khi nhận lệnh tiếp theo [3].

4.5.2 Chuỗi lệnh khởi tạo LCD

Hàm LCD_Init() gửi bốn lệnh chính:

- 0x38: giao tiếp 8 bit, 2 dòng, font ký tự 5x8.
- 0x0C: bật hiển thị, tắt con trỏ và tắt nhấp nháy.

- 0x06: tăng địa chỉ sau mỗi lần ghi, không dịch toàn màn hình.
- 0x01: xóa nội dung hiển thị và đưa địa chỉ DDRAM về đầu.

Các mã lệnh trên tương ứng với tập lệnh của bộ điều khiển HD44780U [3].

```

1 void LCD_Init() {
2     LCD_DATA_DDR = 0xFF;
3     LCD_CTRL_DDR |= (1<<RS) | (1<<RW) | (1<<EN);
4     _delay_ms(20);
5     LCD_Command(0x38);
6     LCD_Command(0x0C);
7     LCD_Command(0x06);
8     LCD_Command(0x01);
9     _delay_ms(2);
10 }
```

Listing 5: Khởi tạo LCD 16x2

Hàm LCD_Gotoxy() tạo địa chỉ DDRAM bằng $0x80 + x$ cho dòng thứ nhất và $0xC0 + x$ cho dòng thứ hai. Nhờ đó chương trình có thể cập nhật độc lập phần nhiệt độ, cảnh báo và thời gian mà không phải ghi lại toàn màn hình.

4.6 Giao tiếp với cảm biến DHT22

4.6.1 Đặc điểm giao thức sử dụng trong chương trình

DHT22, còn được gọi là AM2302 trong tài liệu của Aosong, truyền dữ liệu qua một đường DATA hai chiều. Sau tín hiệu bắt đầu của vi điều khiển, cảm biến phản hồi bằng một khoảng mức thấp khoảng $80\mu s$, tiếp theo mức cao khoảng $80\mu s$, rồi gửi 40 bit dữ liệu. Mỗi bit bắt đầu bằng khoảng mức thấp xấp xỉ $50\mu s$; bit 0 có khoảng mức cao khoảng $22\text{--}30\mu s$, còn bit 1 có khoảng mức cao khoảng $68\text{--}75\mu s$ [?].

Khung dữ liệu 40 bit gồm 5 byte theo thứ tự:

$$\underbrace{RH_H}_{8\text{ bit}} \underbrace{RH_L}_{8\text{ bit}} \underbrace{T_H}_{8\text{ bit}} \underbrace{T_L}_{8\text{ bit}} \underbrace{Checksum}_{8\text{ bit}} \quad (4.1)$$

trong đó chương trình hiện tại chỉ sử dụng hai byte nhiệt độ và byte checksum; hai byte độ ẩm được đọc để kiểm tra checksum nhưng không được hiển thị hoặc sử dụng trong thuật toán điều khiển.

4.6.2 Phát tín hiệu bắt đầu và kiểm tra phản hồi

Hàm DHT22_ReadData() tạm thời tắt ngắt bằng cli() để tránh một ISR xuất hiện giữa quá trình đo độ rộng xung. Chân PA0 được chuyển thành output, kéo xuống thấp 20 ms , sau đó lên cao $30\mu s$ và chuyển lại thành input.

```

1 cli();
2 DHT_DDR |= (1 << DHT_BIT);
3 DHT_PORT &= ~(1 << DHT_BIT);
4 _delay_ms(20);
5 DHT_PORT |= (1 << DHT_BIT);
6 _delay_us(30);
7 DHT_DDR &= ~(1 << DHT_BIT);
8 DHT_PORT |= (1 << DHT_BIT);

```

Listing 6: Phát tín hiệu bắt đầu cho DHT22

Tài liệu AM2302 cho phép thời gian kéo thấp tín hiệu START trong khoảng đến 20 ms; chương trình chọn đúng biên trên 20 ms, đồng thời phù hợp với mô hình DHT22 trong Proteus [?]. Sau khi nhả đường truyền, chương trình dùng ba vòng while có biến timeout để xác nhận chuỗi HIGH–LOW–HIGH của tín hiệu phản hồi. Biến timeout giúp tránh trường hợp chương trình bị kẹt vĩnh viễn khi cảm biến không kết nối.

4.6.3 Giải mã 40 bit dữ liệu

Sau tín hiệu phản hồi, hai vòng lặp lồng nhau thu năm byte dữ liệu. Chương trình đợi kết thúc khoảng LOW đầu bit rồi trễ 35 μ s. Thời điểm lấy mẫu này nằm sau độ rộng HIGH lớn nhất của bit 0 nhưng vẫn trước khi bit 1 kết thúc. Vì vậy:

- nếu PA0 đã về LOW sau 35 μ s thì bit nhận được là 0;
- nếu PA0 vẫn ở HIGH thì bit nhận được là 1.

Cách lấy mẫu phù hợp với cửa sổ thời gian của DHT22 [?].

```

1 _delay_us(35);
2 if (DHT_PIN & (1 << DHT_BIT)) {
3     data[i] = (data[i] << 1) | 1;
4     while ((DHT_PIN & (1 << DHT_BIT)) && timeout) timeout--;
5 } else {
6     data[i] = (data[i] << 1);
7 }

```

Listing 7: Nguyên tắc phân biệt bit 0 và bit 1 của DHT22

Sau khi đủ 40 bit, ngắt toàn cục được bật lại bằng sei().

4.6.4 Kiểm tra checksum và tính nhiệt độ

Checksum được tính bằng tổng bốn byte đầu và lấy 8 bit thấp:

$$Checksum = (RH_H + RH_L + T_H + T_L) \bmod 256. \quad (4.2)$$

Nếu kết quả không bằng byte thứ năm, hàm trả về 0 và vòng lặp chính coi cảm biến đang lỗi hoặc mất kết nối.

Hai byte nhiệt độ được ghép thành giá trị 16 bit. Bit 16 biểu diễn dấu, còn 15 bit thấp biểu diễn độ lớn theo đơn vị 0.1 °C [?]. Phần mã thực hiện đúng quy tắc này:

```

1 if ((uint8_t)(data[0] + data[1] + data[2] + data[3]) != data[4]) {
2     return 0;
3 }
4
5 uint16_t raw_temp = (data[2] << 8) | data[3];
6 if (raw_temp & 0x8000) {
7     *temperature = -((raw_temp & 0x7FFF) / 10.0);
8 } else {
9     *temperature = raw_temp / 10.0;
10 }
11 return 1;

```

Listing 8: Kiểm tra checksum và tách giá trị nhiệt độ

4.6.5 Chu kỳ đọc cảm biến

Vòng lặp chính đặt `dht_timer = 20`; cuối mỗi vòng có `_delay_ms(50)`. Do đó trong mô phỏng chương trình gọi hàm đọc DHT22 xấp xỉ mỗi 1 s. Tuy nhiên, tài liệu AM2302 khuyến nghị khoảng lấy mẫu dài hơn để hạn chế tự nung nóng của cảm biến; với phần cứng thực, có thể tăng `dht_timer` lên khoảng 40 để đạt xấp xỉ 2 s mà không thay đổi kiến trúc chương trình [?].

4.7 Thuật toán điều khiển tốc độ động cơ bằng PWM

4.7.1 Quan hệ giữa nhiệt độ và giá trị PWM

Giá trị điều khiển được lưu trong biến `duty_cycle` kiểu `uint8_t`. Chương trình chia nhiệt độ thành ba miền:

$$P(T) = \begin{cases} 80, & T < 20^{\circ}\text{C}, \\ 80 + \left\lfloor \frac{T - 20}{40} \times 175 \right\rfloor, & 20^{\circ}\text{C} \leq T \leq 60^{\circ}\text{C}, \\ 255, & T > 60^{\circ}\text{C}. \end{cases} \quad (4.3)$$

Trong khoảng 20–60 °C, PWM tăng tuyến tính từ 80 đến 255. Giá trị 80 tương ứng khoảng 31% toàn thang, giúp động cơ có một mức khởi động tối thiểu thay vì điều khiển từ 0. Khi vượt 60 °C, lệnh điều khiển được đưa lên mức cực đại.

```

1 if (nhiet_do < 20.0) {
2     duty_cycle = 80;
3 }
4 else if (nhiet_do <= 60.0) {

```

```

5     duty_cycle = 80 + (uint8_t)((nhiet_do - 20.0) / 40.0) *
      175.0);
6 }
7 else {
8     duty_cycle = 255;
9 }

```

Listing 9: Tính giá trị PWM theo nhiệt độ

Ngưỡng ngưỡng_nhiệt_do không tham gia vào công thức 4.3; nó chỉ dùng để phát cảnh báo trên LCD và UART. Đây là hai chức năng độc lập: nhiệt độ quyết định tốc độ theo đường đặc tính cố định, còn ngưỡng do người dùng cài đặt quyết định thời điểm xuất cảnh báo.

4.7.2 Điều kiện cho phép chạy động cơ

Biến trạng_thai_dong_co đóng vai trò cho phép/chặn lệnh PWM. Khi nhận lệnh UART '0', biến này bằng 0 và current_pwm bị ép về 0; khi nhận '1', PWM được khôi phục theo nhiệt độ hiện tại.

$$current_pwm = \begin{cases} P(T), & \text{trạng_thai_dong_co} = 1, \\ 0, & \text{trạng_thai_dong_co} = 0. \end{cases} \quad (4.4)$$

Ngoài ra, nếu DHT22 đọc lỗi, vòng lặp chính cũng gán current_pwm = 0. Đây là trạng thái an toàn của phần mềm khi không còn dữ liệu nhiệt độ tin cậy.

4.8 Tạo PWM bằng Timer0

4.8.1 Cấu hình Timer0

Timer0 được đặt ở chế độ Normal, dùng clock hệ thống không chia tần vì CS00 = 1. Theo datasheet, với CS02:0 = 001, Timer0 nhận trực tiếp clkI/O và cờ overflow xuất hiện khi bộ đếm 8 bit tràn từ 255 về 0 [1].

Với $F_{CPU} = 8\text{MHz}$, chu kỳ overflow là:

$$T_{OV0} = \frac{256}{8 \times 10^6} = 32\mu\text{s}. \quad (4.5)$$

```

1 void Timer0_SoftPWM_Init() {
2     TCCR0 = (1 << CS00);
3     TIMSK |= (1 << TOIE0);
4 }

```

Listing 10: Khởi tạo Timer0 cho PWM phần mềm

4.8.2 Nguyên lý PWM phần mềm

Trong mỗi ISR overflow, biến tĩnh `pwm_cnt` tăng một đơn vị và tự tràn sau 256 bước. PB4 được giữ mức cao khi `pwm_cnt < current_pwm`, sau đó chuyển xuống thấp cho đến hết chu kỳ. Một chu kỳ PWM hoàn chỉnh kéo dài:

$$T_{PWM} = 256 T_{OV0} = 8.192 \text{ ms}, \quad f_{PWM} \approx 122.07 \text{ Hz}. \quad (4.6)$$

Với $0 < P < 255$, hệ số lấp đầy thực tế xấp xỉ:

$$D \approx \frac{P}{256} \times 100\%. \quad (4.7)$$

Riêng hai trường hợp `current_pwm = 0` và `255` được xử lý riêng để đảm bảo PB4 luôn LOW hoặc luôn HIGH.

```
1 ISR(TIMERO_OVF_vect) {
2     static uint8_t pwm_cnt = 0;
3
4     if (current_pwm == 0) {
5         PORTB &= ~(1 << PB4);
6     } else if (current_pwm == 255) {
7         PORTB |= (1 << PB4);
8     } else {
9         if (pwm_cnt < current_pwm) PORTB |= (1 << PB4);
10        else PORTB &= ~(1 << PB4);
11    }
12    pwm_cnt++;
13 }
```

Listing 11: ISR tạo PWM phần mềm trên PB4

4.9 Xử lý nút nhấn và Timer2

4.9.1 Cấu hình Timer2 ở chế độ CTC

Timer2 được đặt ở chế độ CTC bằng bit `WGM21`. Ba bit `CS22:0` đều bằng 1 nên hệ số chia là 1024; `OCR2 = 78`. Datasheet mô tả ở chế độ CTC, bộ đếm được xóa về 0 khi `TCNT2` bằng `OCR2` và có thể tạo ngắt Compare Match [1].

Chu kỳ ngắt Timer2 được tính theo:

$$T_{T2} = \frac{(OCR2 + 1) \times 1024}{F_{CPU}} = \frac{79 \times 1024}{8 \times 10^6} \approx 10.112 \text{ ms}. \quad (4.8)$$

```
1 void Timer2_Init() {
2     TCCR2 = (1 << WGM21) | (1 << CS22) | (1 << CS21) | (1 << CS20)
3     ;
4 }
```

```

3   OCR2 = 78;
4   TIMSK |= (1 << OCIE2);
5 }

```

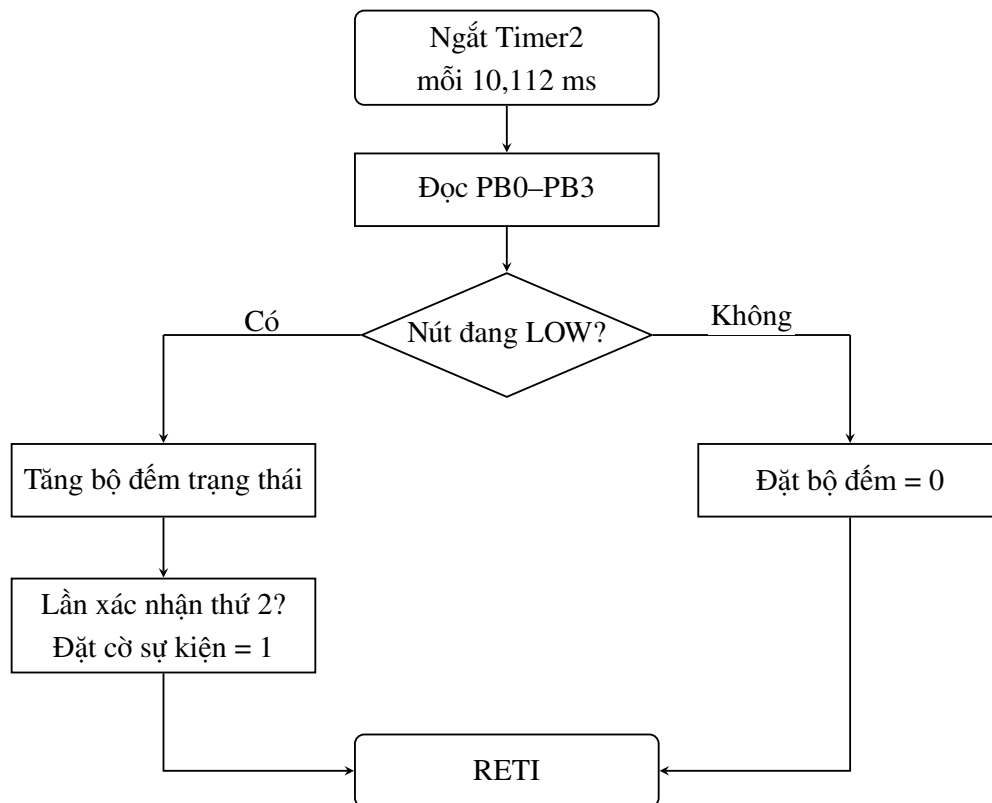
Listing 12: Khởi tạo Timer2 cho chống dội nút

4.9.2 Thuật toán chống dội

Mỗi nút có một biến trạng thái cục bộ tĩnh. Nếu nút ở mức thấp trong hai lần lấy mẫu liên tiếp, ISR đặt cờ sự kiện tương ứng. Như vậy thời gian xác nhận nhấn vào khoảng:

$$T_{debounce} \approx 2T_{T2} \approx 20.224 \text{ ms.} \quad (4.9)$$

Sau khi biến trạng thái đạt 3, ISR không phát thêm sự kiện khi người dùng tiếp tục giữ nút. Khi nút được thả, biến trạng thái trở về 0, sẵn sàng phát sự kiện ở lần nhấn tiếp theo.



Hình 4.2 Lưu đồ chống dội nút nhấn bằng Timer2

4.9.3 Chức năng các nút trong vòng lặp chính

Khi cờ SET xuất hiện, biến `che_do_cai_dat` được đảo trạng thái. Trong chế độ cài đặt:

- UP tăng ngưỡng 1 °C;

- DOWN giảm ngưỡng 1 °C;
- RESET đưa ngưỡng về giá trị mặc định 30 °C.

Giá trị khởi động của `nguonng_nhiet_do` là 30 °C. Khi người dùng nhấn RESET trong chế độ cài đặt, chương trình đưa ngưỡng trở lại đúng giá trị mặc định này.

4.10 Bộ đếm thời gian bằng Timer1

4.10.1 Tính toán chu kỳ Timer1

Timer1 hoạt động ở chế độ CTC với $WGM12 = 1$. Chương trình chỉ đặt $CS12 = 1$, tương ứng $CS12:0 = 100$; datasheet quy định cấu hình này sử dụng prescaler 256 [1]. OCR1A được đặt bằng 31249 nên:

$$T_{T1} = \frac{(31249 + 1) \times 256}{8 \times 10^6} = 1 \text{ s.} \quad (4.10)$$

Như vậy mỗi lần xảy ra Compare Match A, ISR tăng biến `giay` đúng một đơn vị theo cấu hình lý tưởng của xung hệ thống 8 MHz.

```

1 void Timer1_Init() {
2     TCCR1B |= (1 << WGM12);
3     TCCR1B |= (1 << CS12);
4     OCR1A = 31249;
5     TIMSK |= (1 << OCIE1A);
6 }
7
8 ISR(TIMER1_COMPA_vect) {
9     giay++;
10    if(giay > 59) { giay = 0; phut++; }
11    if(phut > 59) { phut = 0; gio++; }
12    if(gio > 23) { gio = 0; }
13 }
```

Listing 13: Cấu hình Timer1 và cập nhật thời gian

Bộ đếm này là thời gian chạy tương đối từ lúc vi điều khiển khởi động, không phải đồng hồ thời gian thực có khả năng lưu thời gian sau khi mất nguồn.

4.11 Giao tiếp UART và điều khiển từ xa

4.11.1 Cấu hình tốc độ truyền

Hàm `UART_Init(9600)` tính UBRR theo chế độ USART bất đồng bộ thông thường:

$$UBRR = \frac{F_{CPU}}{16BAUD} - 1. \quad (4.11)$$

Với $F_{CPU} = 8\text{MHz}$ và $BAUD = 9600$, giá trị tính toán là 51,083 và được lưu vào biến số nguyên thành 51. Baud thực tế là:

$$BAUD_{thuc} = \frac{8 \times 10^6}{16(51 + 1)} \approx 9615.38 \text{ bit/s}, \quad (4.12)$$

tương ứng sai số khoảng 0,16%. Công thức và cách sử dụng UBRR phù hợp với chế độ Asynchronous Normal Mode của USART ATmega16 [1].

4.11.2 Định dạng khung truyền

UCSRB bật bộ thu, bộ phát và ngắt khi nhận hoàn tất thông qua RXEN, TXEN và RXCIE. UCSRC đặt URSEL = 1 để chọn đúng thanh ghi UCSRC và đặt UCSZ1:0 = 11, tương ứng 8 bit dữ liệu. Các bit parity và stop giữ giá trị mặc định, do đó cấu hình thực tế là 9600–8N1 [1].

```

1 void UART_Init(unsigned long baud) {
2     unsigned int ubrr = (F_CPU / (16UL * baud)) - 1;
3     UBRRH = (unsigned char)(ubrr >> 8);
4     UBRRL = (unsigned char)(ubrr);
5     UCSRB = (1 << RXEN) | (1 << TXEN) | (1 << RXCIE);
6     UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0);
7 }

```

Listing 14: Khởi tạo USART

Hàm UART_Transmit() chờ cờ UDRE cho biết thanh ghi dữ liệu truyền sẵn sàng rồi ghi byte mới vào UDR. Datasheet USART mô tả việc ghi UDR là thao tác khởi đầu quá trình truyền khung dữ liệu [1].

4.11.3 Nhận lệnh dừng/chạy động cơ

Khi USART nhận đủ một ký tự, ngắt USART_RXC_vect xảy ra. ISR đọc UDR và xử lý hai lệnh:

- '0': đặt trang_thai_dong_co = 0;
- '1': đặt trang_thai_dong_co = 1.

Các ký tự khác bị bỏ qua.

```

1 ISR(USART_RXC_vect) {
2     char cmd = UDR;
3     if (cmd == '0') {
4         trang_thai_dong_co = 0;
5         UART_String("\r\n[HE THONG] Dong co da dung!\r\n");
6     } else if (cmd == '1') {
7         trang_thai_dong_co = 1;

```

```

8         UART_String("\r\n[HE THONG] Dong co hoat dong tro lai!\r\n
          ");
9     }
10 }

```

Listing 15: Xử lý lệnh UART trong ngắt nhận

4.11.4 Truyền dữ liệu trạng thái

Trong giao diện chính, biến `bo_dem_uart` tăng sau mỗi vòng lặp. Khi lớn hơn 10, hệ thống gửi chuỗi chứa nhiệt độ và PWM; nếu động cơ đang tạm dừng thì PWM truyền đi bằng 0 và thêm chuỗi “TAM DUNG”. Với thời gian trễ chính 50 ms, chu kỳ gửi danh định xấp xỉ $11 \times 50 = 550$ ms, chưa tính thời gian đọc DHT22 và thời gian truyền chuỗi.

Khi `nhiet_do >= nguong_nhiet_do`, một thông báo cảnh báo được gửi trước dữ liệu trạng thái. Việc truyền trạng thái chỉ được thực hiện khi hệ thống không ở chế độ cài đặt ngưỡng.

4.12 Hiển thị và cảnh báo trên LCD

4.12.1 Giao diện trạng thái chính

Ở chế độ bình thường, dòng thứ nhất hiển thị nhiệt độ theo định dạng một chữ số thập phân. Hàm `dtostrf()` chuyển số thực thành chuỗi, sau đó `sprintf()` tạo thông báo hoàn chỉnh. Dòng thứ hai hiển thị thời gian dạng `hh:mm:ss`.

Nếu nhiệt độ lớn hơn hoặc bằng ngưỡng, vị trí cột 13 của dòng đầu được ghi chuỗi `!W!`. Khi nhiệt độ trở xuống dưới ngưỡng, ba ký tự này được ghi bằng khoảng trắng để xóa cảnh báo cũ.

4.12.2 Giao diện cài đặt ngưỡng

Trong chế độ cài đặt, LCD hiển thị “SET NGUONG:” ở dòng đầu và giá trị `Max: xx.x C` ở dòng thứ hai. Các cờ nút được xử lý ở vòng lặp chính thay vì thực hiện trực tiếp trong ISR, giúp ISR Timer2 ngắn và dễ kiểm soát thời gian thực thi.

4.12.3 Trạng thái mất kết nối cảm biến

Nếu `DHT22_ReadData()` trả về 0, `trang_thai_cam_bien` được xóa. Vòng lặp chính khi đó:

- hiển thị “Dang ket noi...”;
- tiếp tục hiển thị thời gian;
- đặt `current_pwm = 0`;

- bỏ qua phần tính PWM, giao diện cài đặt và truyền trạng thái của vòng hiện tại bằng lệnh `continue`.

Cách xử lý này tránh điều khiển động cơ dựa trên giá trị nhiệt độ cũ khi cảm biến đã mất kết nối.

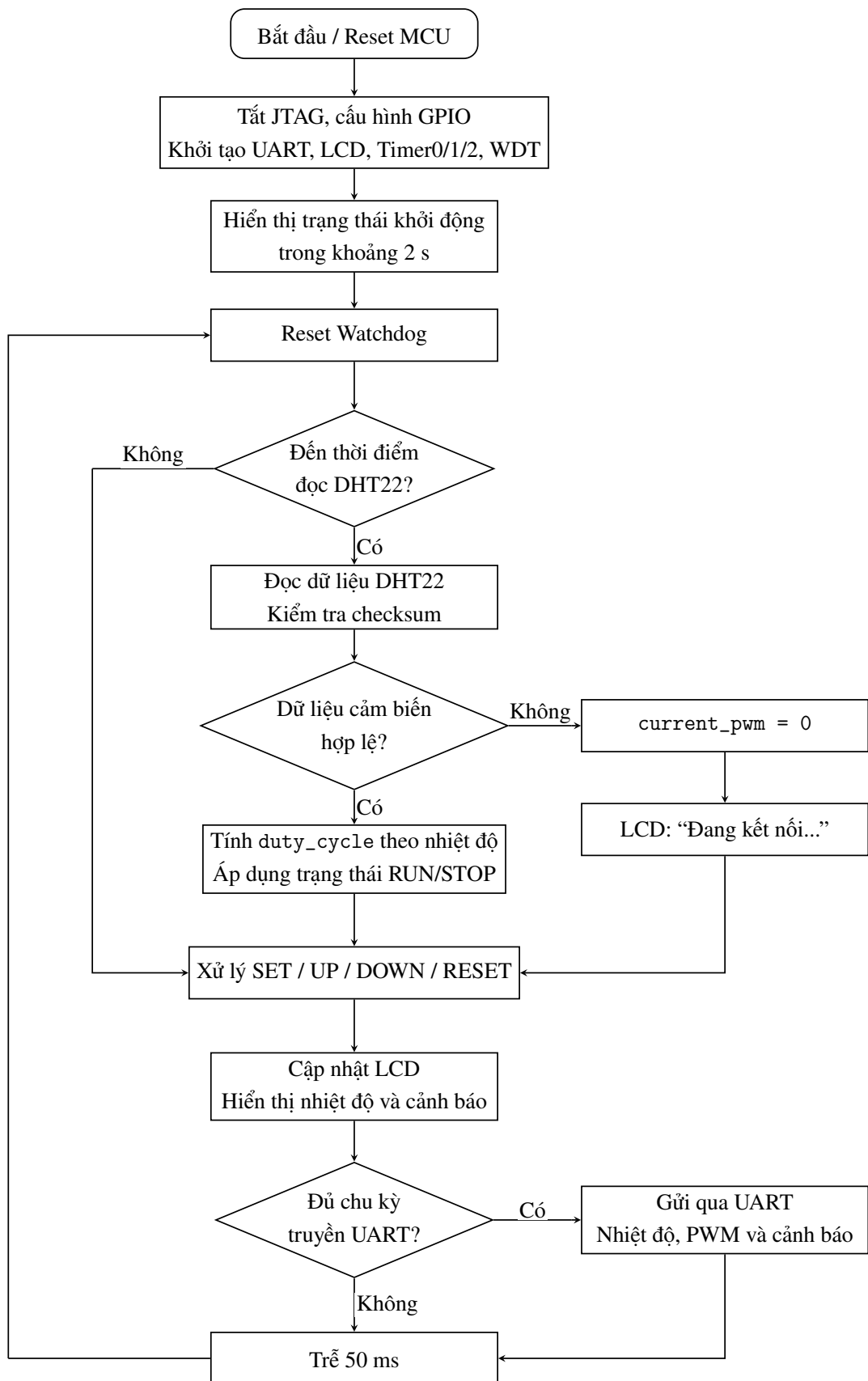
4.13 Cơ chế Watchdog

Watchdog được kích hoạt với mức timeout danh định 2 s. Hàm `wdt_reset()` được gọi ở mỗi vòng lặp chính và trong màn hình khởi động. Khi chương trình hoạt động bình thường, Watchdog liên tục được làm mới trước khi hết thời gian. Nếu CPU bị kẹt ở một trạng thái bất thường đủ lâu, Watchdog có thể tạo reset hệ thống. ATmega16 sử dụng một bộ dao động riêng cho Watchdog và cho phép chọn thời gian timeout qua các bit `WDP2:0` trong `WDTCR` [1]; AVR-Libc cung cấp macro `wdt_enable()` và `wdt_reset()` để thao tác thuận tiện với khối này [6].

4.14 Lưu đồ thuật toán chương trình

4.14.1 Lưu đồ chương trình chính

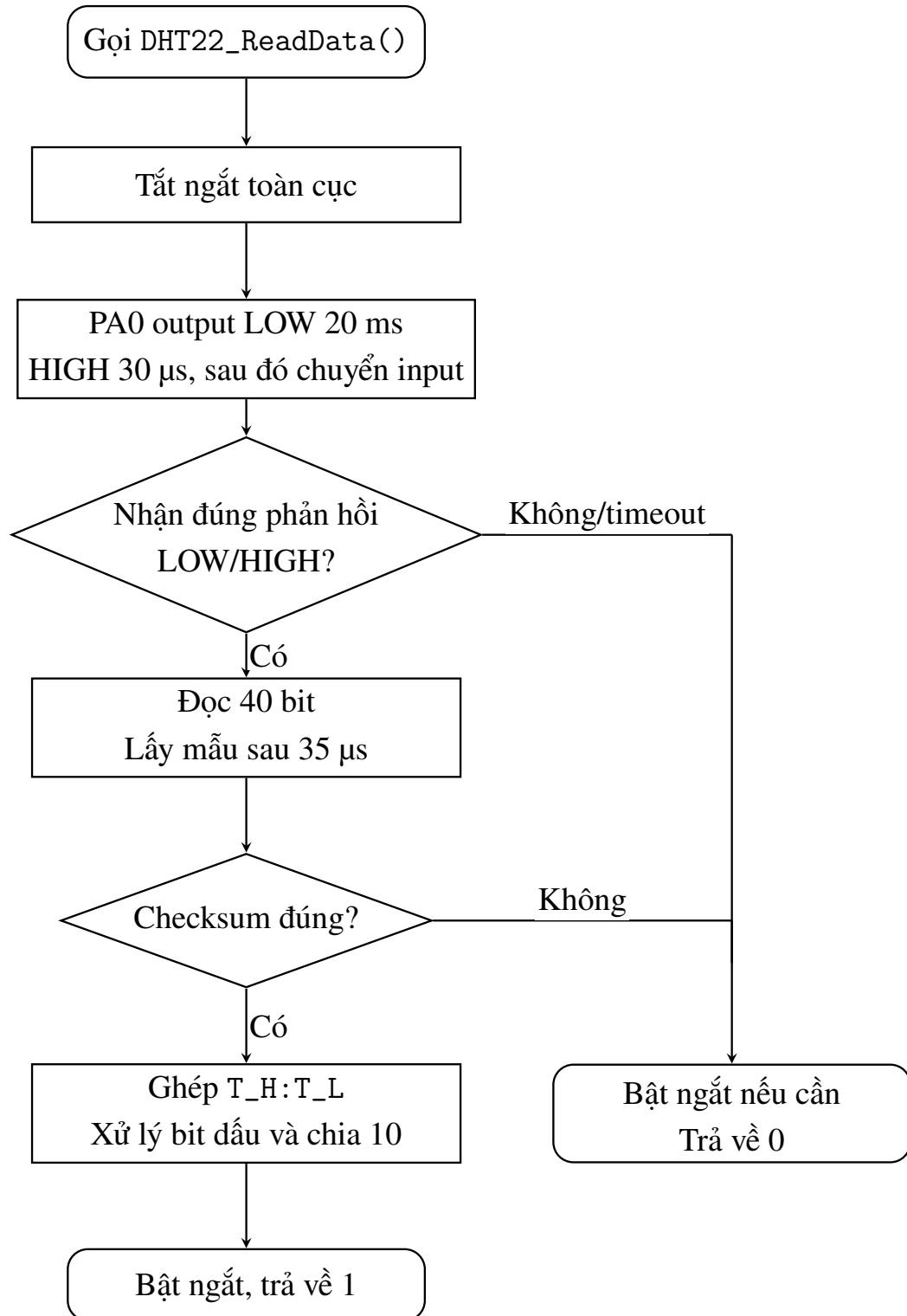
Hình 4.3 mô tả luồng tổng thể của chương trình từ khởi tạo đến vòng lặp xử lý liên tục. Luồng lỗi cảm biến được tách riêng vì đây là điều kiện làm động cơ dừng ngay và bỏ qua các bước điều khiển còn lại trong vòng hiện tại.



Hình 4.3 Lưu đồ thuật toán chương trình chính

4.14.2 Lưu đồ đọc DHT22

Hình 4.4 trình bày chi tiết hàm đọc cảm biến. Các nhánh lỗi timeout và checksum đều trả về 0, cho phép vòng lặp chính xử lý thống nhất như một trạng thái mất dữ liệu cảm biến.



Hình 4.4 Lưu đồ thuật toán đọc và kiểm tra dữ liệu DHT22

4.15 Phân tích vòng lặp chính từ tổng quát đến chi tiết

Phần trọng tâm của chương trình nằm trong `while(1)`. Mỗi vòng được thực hiện theo thứ tự sau:

1. Reset Watchdog.
2. Kiểm tra bộ đếm phần mềm của DHT22; khi bằng 0, gọi hàm đọc cảm biến và cập nhật cờ `trang_thai_cam_bien`.
3. Nếu cảm biến lỗi, dừng PWM, hiển thị trạng thái chờ và chuyển sang vòng tiếp theo.
4. Nếu cảm biến hợp lệ, tính `duty_cycle` theo phương trình 4.3.
5. Áp dụng trạng thái RUN/STOP do UART điều khiển để tạo `current_pwm`.
6. Xử lý cờ nút SET. Nếu đang ở chế độ cài đặt, xử lý UP/DOWN/RESET và hiển thị ngưỡng.
7. Nếu ở giao diện chính, hiển thị nhiệt độ, cảnh báo và thời gian.
8. Tăng bộ đếm UART; khi đủ chu kỳ, truyền trạng thái và cảnh báo.
9. Trễ 50 ms rồi lặp lại.

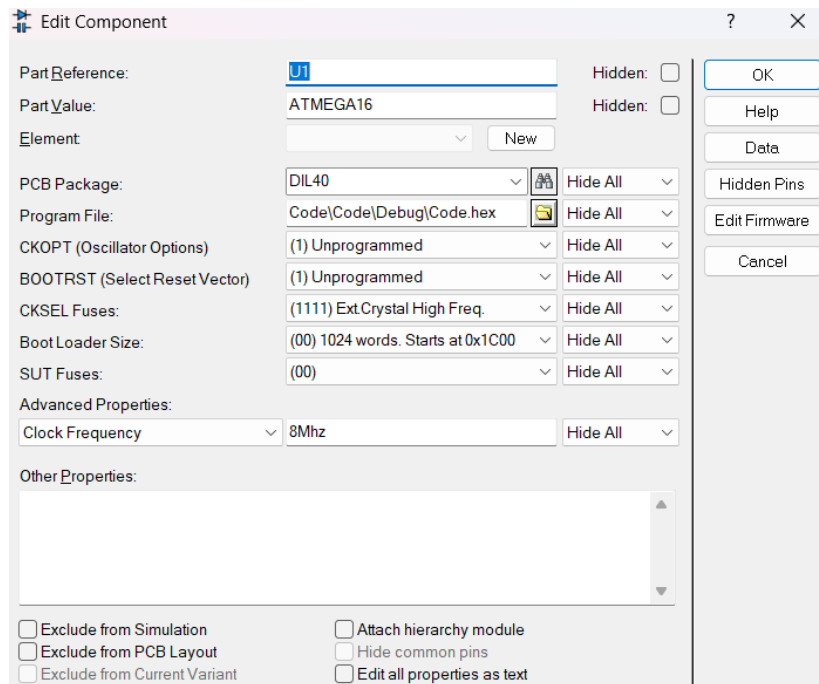
Cách sắp xếp trên bảo đảm dữ liệu cảm biến được kiểm tra trước khi dùng cho điều khiển. Các phần có yêu cầu thời gian nhanh hơn chu kỳ 50 ms của vòng lặp, gồm PWM, đồng hồ và chống dội nút, được thực hiện bằng ngắt Timer.

4.16 Xây dựng mô hình mô phỏng Proteus

Mô hình Proteus được xây dựng theo đúng ánh xạ chân và tham số sử dụng trong chương trình. Các hình dưới đây là các phần chính của mô hình mô phỏng, gồm vi điều khiển ATmega16, cảm biến DHT22, LCD 16x2, cụm nút nhấn, tăng công suất động cơ và Virtual Terminal. Việc tách thành từng khối giúp kiểm tra kết nối của từng ngoại vi trước khi chạy mô phỏng toàn hệ thống.

4.16.1 Thiết lập ATmega16 và nạp chương trình

ATmega16 được cấu hình với tần số clock 8 MHz, đúng với khai báo `F_CPU = 8000000UL` trong chương trình. File HEX sau khi biên dịch được chọn tại trường *Program File*. Việc đặt đúng tần số clock là cần thiết vì các hàm delay, Timer và tốc độ UART đều được tính dựa trên tần số này.

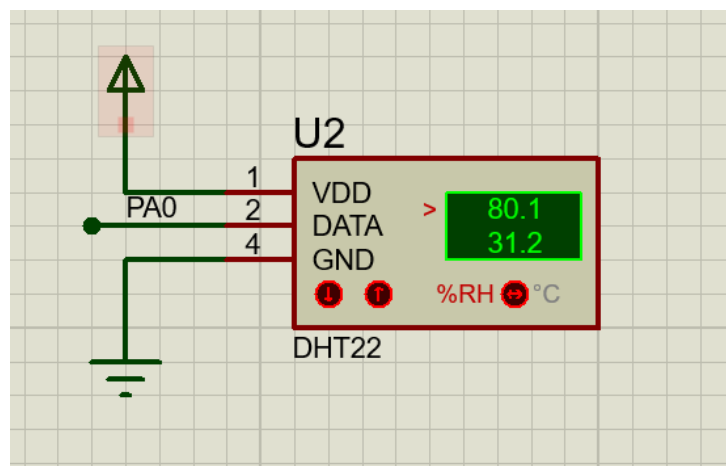


Hình 4.5 Thiết lập ATmega16 và file chương trình trong Proteus

Hình 4.5 cho thấy vi điều khiển U1 được chọn là ATmega16, file chương trình được nạp ở định dạng HEX và tần số mô phỏng được đặt bằng 8 MHz. Đây là bước cấu hình đầu tiên trước khi kiểm tra các khối ngoại vi.

4.16.2 Mô phỏng cảm biến DHT22

Cảm biến DHT22 sử dụng ba kết nối chính gồm VDD, DATA và GND. Trong mô hình, chân DATA được nối tới PA0 của ATmega16. Chương trình điều khiển PA0 luân phiên ở chế độ output và input để phát tín hiệu bắt đầu, nhận phản hồi và đọc 40 bit dữ liệu từ cảm biến.

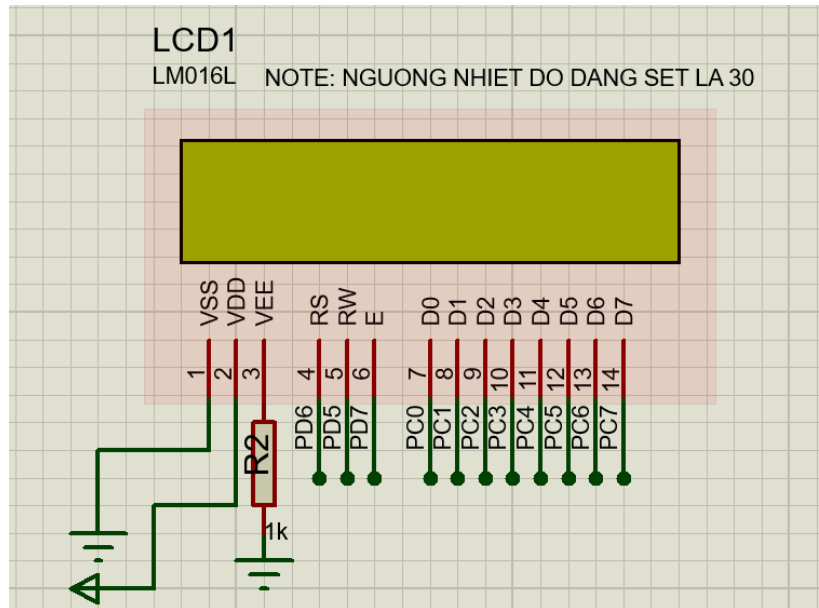


Hình 4.6 Kết nối cảm biến DHT22 với chân PA0 trong Proteus

Trong Hình 4.6, model DHT22 hiển thị đồng thời độ ẩm và nhiệt độ để phục vụ thay đổi điều kiện mô phỏng. Chương trình của đồ án đọc khung dữ liệu DHT22 nhưng sử dụng giá trị nhiệt độ làm đầu vào cho thuật toán điều khiển động cơ.

4.16.3 Mô phỏng LCD 16x2

LCD 16x2 được sử dụng ở chế độ dữ liệu 8 bit. Các chân D0–D7 nối với PC0–PC7; ba chân điều khiển RS, R/W và E lần lượt nối với PD6, PD5 và PD7. Cách nối này tương ứng trực tiếp với các khai báo chân trong chương trình.

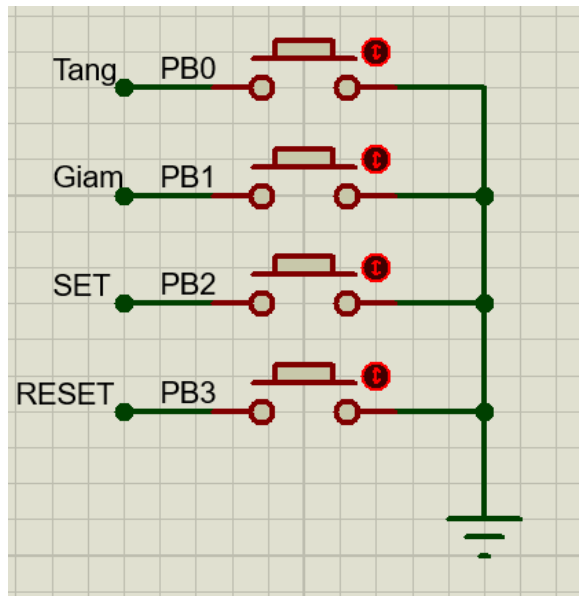


Hình 4.7 Kết nối LCD 16x2 trong mô hình Proteus

Hình 4.7 thể hiện đầy đủ bus dữ liệu 8 bit và các đường điều khiển của LCD. Trong quá trình mô phỏng, LCD được dùng để hiển thị nhiệt độ, thời gian, cảnh báo vượt ngưỡng và giao diện cài đặt ngưỡng.

4.16.4 Mô phỏng cụm nút nhấn

Bốn nút nhấn được bố trí theo chức năng UP, DOWN, SET và RESET, tương ứng với PB0, PB1, PB2 và PB3. Một đầu của mỗi nút nối GND. Chương trình bật điện trở kéo lên nội nên khi không nhấn, chân vào ở mức logic 1; khi nhấn, chân tương ứng được kéo xuống mức logic 0.

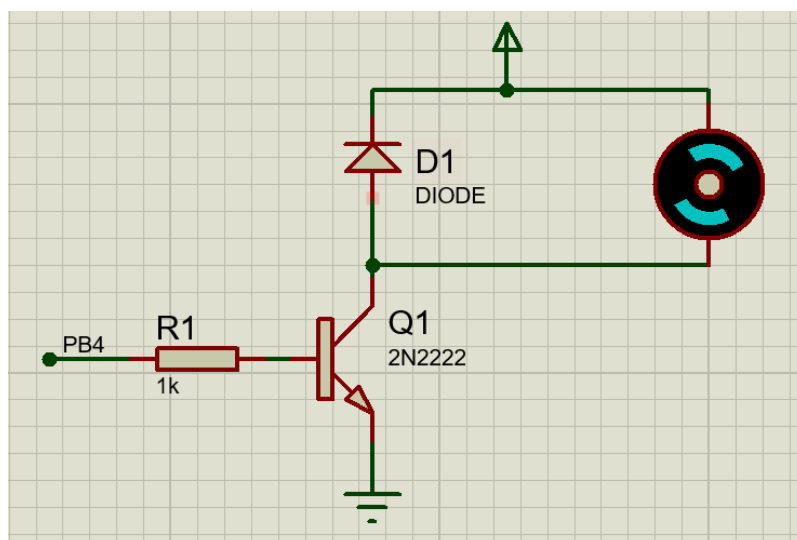


Hình 4.8 Kết nối các nút UP, DOWN, SET và RESET trong Proteus

Cấu hình active-low trong Hình 4.8 phù hợp với thuật toán chống dội bằng Timer2. UP và DOWN thay đổi ngưỡng, SET chuyển đổi chế độ cài đặt, còn RESET đưa ngưỡng về giá trị mặc định 30 °C.

4.16.5 Mô phỏng PWM và tăng công suất động cơ

PB4 là ngõ ra PWM phần mềm của hệ thống. Tín hiệu từ PB4 được đưa qua điện trở R1 tới transistor Q1 loại 2N2222 để đóng cắt dòng qua động cơ. Diode D1 mắc song song với tải động cơ có nhiệm vụ tạo đường thoát cho dòng cảm ứng khi transistor ngắt, qua đó bảo vệ phần tử đóng cắt khỏi điện áp ngược do tải cảm gây ra.

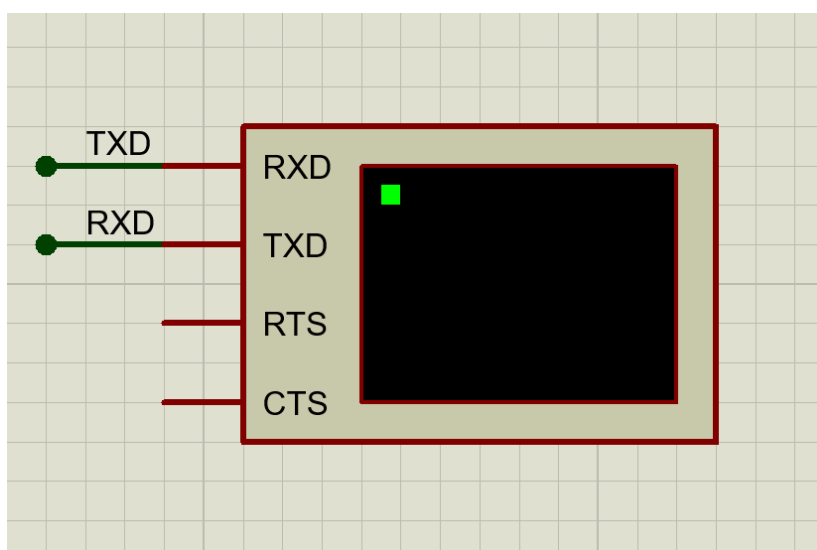


Hình 4.9 Tăng công suất điều khiển động cơ bằng tín hiệu PWM tại PB4

Theo Hình 4.9, vi điều khiển không cấp dòng trực tiếp cho động cơ mà chỉ tạo tín hiệu điều khiển cho transistor. Khi hệ số lấp đầy PWM tăng, thời gian Q1 dẫn trong mỗi chu kỳ tăng và công suất trung bình cấp cho động cơ tăng tương ứng.

4.16.6 Mô phỏng giao tiếp UART bằng Virtual Terminal

Virtual Terminal được dùng để theo dõi dữ liệu UART và gửi lệnh điều khiển ngược về ATmega16. Đường TXD của vi điều khiển nối tới RXD của Virtual Terminal, còn RXD của vi điều khiển nối tới TXD của Virtual Terminal. Giao tiếp được cấu hình ở tốc độ 9600 bit/s, 8 bit dữ liệu, không parity và 1 bit stop.



Hình 4.10 Kết nối UART giữa ATmega16 và Virtual Terminal trong Proteus

Thông qua Virtual Terminal ở Hình 4.10, hệ thống có thể gửi chuỗi trạng thái gồm nhiệt độ và mức PWM. Ký tự '0' được dùng để dừng động cơ, trong khi ký tự '1' cho phép động cơ tiếp tục hoạt động theo mức PWM được tính từ nhiệt độ.

4.16.7 Tổng hợp cấu hình mô phỏng

Các thông số và kết nối chính của mô hình Proteus được tổng hợp trong Bảng 4.3. Các giá trị này phải thống nhất với chương trình để kết quả mô phỏng phản ánh đúng hoạt động của hệ thống.

Bảng 4.3 Các thông số và kết nối chính trong mô hình Proteus

Khối	Thông số/kết nối
ATmega16	Clock 8 MHz, nạp file HEX sau khi biên dịch chương trình
DHT22	DATA nối PA0; VDD và GND nối nguồn tương ứng
LCD 16x2	D0–D7 = PC0–PC7; RS = PD6; R/W = PD5; E = PD7
Nút nhấn	UP = PB0; DOWN = PB1; SET = PB2; RESET = PB3; active-low
Động cơ	PWM tại PB4 qua R1 điều khiển Q1 2N2222; D1 bảo vệ tải cảm
UART	PD1/TXD nối RXD terminal; PD0/RXD nối TXD terminal; 9600–8N1

4.17 Các trường hợp kiểm thử phần mềm

Trước khi tích hợp với phần cứng thực, có thể kiểm tra các giá trị đầu ra dự kiến trong Proteus theo Bảng 4.4.

Bảng 4.4 Một số trường hợp kiểm thử chức năng phần mềm

Điều kiện	PWM dự kiến	Kết quả cần quan sát
DHT22 mất kết nối	0	LCD báo “Đang ket noi...”, động cơ dừng
$T = 15^{\circ}\text{C}$	80	Động cơ chạy mức tối thiểu nếu RUN
$T = 20^{\circ}\text{C}$	80	Bắt đầu miền tăng tuyến tính
$T = 30^{\circ}\text{C}$	123	PWM tăng theo nhiệt độ; nếu ngưỡng 30 thì xuất cảnh báo
$T = 40^{\circ}\text{C}$	167	PWM tiếp tục tăng tuyến tính
$T = 60^{\circ}\text{C}$	255	Ngõ PB4 luôn HIGH trong mã hiện tại
$T > 60^{\circ}\text{C}$	255	Duy trì mức cực đại
UART gửi 0	0	Động cơ dừng, terminal nhận phản hồi
UART gửi 1	Theo $P(T)$	Động cơ hoạt động lại theo nhiệt độ
Nhấn SET	–	Chuyển qua lại giữa giao diện chính và cài đặt
Nhấn UP/DOWN	–	Ngưỡng tăng/giảm từng 1°C
Nhấn RESET	–	Ngưỡng được gán về 45°C theo mã hiện tại

Các giá trị PWM ở miền tuyến tính chịu phép ép kiểu sang uint8_t, vì vậy phần thập phân bị loại bỏ. Ví dụ tại 30°C , giá trị lý thuyết là $80 + 43.75$, sau khi ép kiểu thu được 123.

4.18 Đánh giá và các lưu ý đối với phiên bản phần mềm hiện tại

Phần mềm đã tích hợp được nhiều ngoại vi của ATmega16 trong cùng một chương trình, đồng thời có cơ chế xử lý lỗi cảm biến và Watchdog. Tuy nhiên, từ việc phân tích mã nguồn có thể rút ra một số điểm cần lưu ý khi chuyển từ mô phỏng sang phần cứng thực:

- Hàm `DHT22_ReadData()` tắt toàn bộ ngắt trong khoảng thời gian giao tiếp cảm biến. Trong thời gian đó, các ISR Timer0, Timer1, Timer2 và USART không được thực thi. Do Timer0 dùng để tạo PWM bằng phần mềm, xung PWM có thể bị kéo dài bất thường trong lúc đọc DHT22. Đây là giới hạn của cách tạo PWM bằng phần mềm.
- ISR nhận UART hiện gửi cả chuỗi phản hồi bằng hàm chờ `UDRE`. Ở 9600 bit/s, việc truyền nhiều ký tự ngay trong ISR làm ISR kéo dài và trì hoãn các ngắt khác. Khi tối ưu, ISR nên chỉ đọc `UDR`, cập nhật trạng thái hoặc đặt cờ; phần truyền chuỗi nên chuyển ra vòng lặp chính.
- DHT22 được lấy mẫu khoảng 1 s trong mã hiện tại. Khi triển khai cảm biến thực nên kiểm tra yêu cầu chu kỳ lấy mẫu của đúng module DHT22 sử dụng và có thể tăng chu kỳ lên khoảng 2 s theo khuyến nghị của tài liệu AM2302 [?].
- Ngưỡng ban đầu là 30 °C nhưng nút RESET đưa về 45 °C; cần thống nhất nếu hai giá trị được hiểu là cùng một “ngưỡng mặc định”.
- Nếu cần PWM ổn định hơn, có thể chuyển sang một kênh PWM phần cứng của Timer/Counter và thay đổi ánh xạ chân. ATmega16 hỗ trợ các kênh PWM phần cứng và nhiều Timer/Counter [1].

Các điểm trên không thay đổi nguyên lý hoạt động của chương trình đang mô phỏng, nhưng là cơ sở để đánh giá sai số thời gian và cải thiện độ ổn định ở phiên bản phần cứng tiếp theo.

4.19 Kết luận chương

Chương đã trình bày toàn bộ cấu trúc phần mềm của hệ thống từ mức tổng quát đến các khối chức năng cụ thể. Chương trình sử dụng DHT22 để đo nhiệt độ, Timer0 để tạo PWM phần mềm, Timer1 để tạo cơ sở thời gian, Timer2 để chống dội nút, USART để giám sát và điều khiển từ xa, LCD 16x2 để hiển thị và Watchdog để tăng khả năng tự phục hồi. Các thông số Timer và UART đã được tính lại từ tần số hệ thống 8 MHz, qua đó xác định được chu kỳ Timer1 là 1 s, chu kỳ lấy mẫu nút khoảng 10.112 ms và tần số PWM phần mềm khoảng 122.07 Hz. Các lưu đồ thuật toán ở Hình 4.3, Hình 4.4 và Hình 4.2 thể hiện quan hệ giữa vòng lặp chính và các tác vụ thời gian thực, làm cơ sở cho quá trình mô phỏng và kiểm thử ở các bước tiếp theo.

CHƯƠNG 5. THỰC NGHIỆM, KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày phương pháp kiểm thử và đánh giá phiên bản phần mềm cuối của hệ thống sử dụng ATmega16. Nội dung kiểm thử được xây dựng trực tiếp từ chương trình `main.c` và các thông số ngoại vi đã phân tích ở Chương 4. Mục tiêu của chương là kiểm tra tính đúng đắn của các khối đọc cảm biến, tạo PWM, hiển thị LCD, xử lý nút nhấn, UART, Timer và Watchdog, đồng thời đánh giá các giới hạn còn tồn tại của chương trình.

Trong mã nguồn cuối, cảm biến nhiệt độ được sử dụng là DHT22 nối với PA0 theo giao tiếp số. Chương trình không cấu hình và không đọc bộ ADC của ATmega16. Vì vậy, phần kiểm thử ADC/LM35 trong khung báo cáo ban đầu không còn phù hợp và được thay bằng kiểm thử giao tiếp DHT22. PA0 vẫn có chức năng phụ ADC0 theo datasheet ATmega16, nhưng ở chương trình hiện tại chân này được sử dụng như một chân I/O số [1].

Các kết quả trong chương được chia thành hai nhóm. Nhóm thứ nhất là các thông số có thể xác minh trực tiếp từ mã nguồn và datasheet, ví dụ chu kỳ Timer, tốc độ UART và quan hệ nhiệt độ–PWM. Nhóm thứ hai là các kết quả cần quan sát trên Proteus hoặc phần cứng thật, ví dụ dạng sóng PWM, ảnh LCD, Virtual Terminal và phản ứng của động cơ. Đối với nhóm thứ hai, báo cáo chỉ kết luận theo hành vi được định nghĩa trong chương trình; các ảnh minh chứng cần được chèn từ mô phỏng hoặc thực nghiệm thực tế để tránh đưa vào số liệu không có cơ sở.

5.1 Mục tiêu và phương pháp kiểm thử

Quá trình kiểm thử được tổ chức theo hướng từ từng khối chức năng đến toàn hệ thống. Trước tiên kiểm tra các ngoại vi riêng lẻ, sau đó kiểm tra sự tương tác giữa các ngoại vi khi chương trình chạy đồng thời. Các tiêu chí chính gồm:

- DHT22 trả về nhiệt độ hợp lệ và hệ thống phát hiện được lỗi truyền hoặc sai checksum.
- Giá trị PWM thay đổi đúng theo đặc tính nhiệt độ được cài đặt trong chương trình.
- Khi mất cảm biến hoặc nhận lệnh dừng qua UART, ngõ ra điều khiển động cơ phải về 0.
- LCD hiển thị đúng nhiệt độ, thời gian, ngưỡng cài đặt và trạng thái cảnh báo.
- Bốn nút SET, UP, DOWN, RESET chỉ tạo một sự kiện cho mỗi lần nhấn thông thường.

- UART làm việc ở cấu hình 9600–8N1, truyền được dữ liệu trạng thái và nhận được lệnh '0', '1'.
- Timer1 tạo mốc thời gian xấp xỉ một giây; Timer2 tạo chu kỳ lấy mẫu phím xấp xỉ 10 ms; Timer0 tạo PWM phần mềm.
- Watchdog có khả năng reset vi điều khiển khi chương trình không được làm mới trong thời gian cho phép.

Bảng 5.1 tổng hợp kế hoạch kiểm thử chính.

Bảng 5.1 Ma trận kiểm thử các chức năng chính

Mã	Nội dung kiểm thử	Tiêu chí đạt	Cơ sở đánh giá
TC01	Đọc nhiệt độ DHT22	Nhận đủ 40 bit, checksum đúng, tách được nhiệt độ	Mã nguồn + mô phỏng
TC02	Mất kết nối DHT22	LCD báo chờ kết nối, PWM về 0	Mã nguồn
TC03	Quan hệ nhiệt độ–PWM	PWM đúng ba miền nhiệt độ	Tính toán từ mã nguồn
TC04	LCD và cảnh báo	Hiển thị đúng nhiệt độ, thời gian, ngưỡng và !W!	Mã nguồn + mô phỏng
TC05	Nút nhấn	Một lần nhấn tạo một sự kiện, có chống dôi	Timer2 + mô phỏng
TC06	UART	Truyền trạng thái, nhận '0'/'1'	Mã nguồn + Virtual Terminal
TC07	Timer	Chu kỳ phù hợp với giá trị OCR và prescaler	Datasheet + tính toán
TC08	Watchdog	Reset khi không gọi wdt_reset() đủ lâu	Datasheet + thực nghiệm

5.2 Kiểm thử cảm biến DHT22

5.2.1 Kiểm tra quá trình nhận dữ liệu

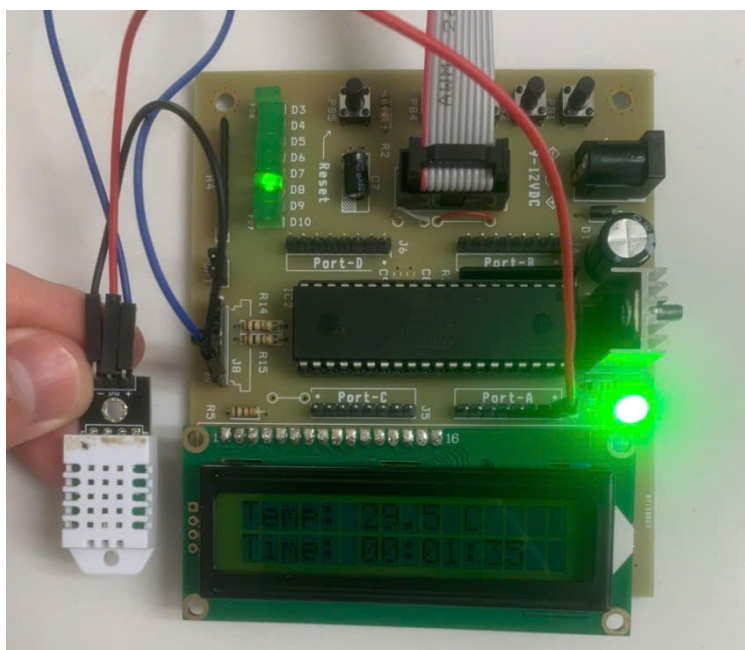
Hàm `DHT22_ReadData()` thực hiện đầy đủ các bước phát tín hiệu bắt đầu, nhận phản hồi, thu 40 bit, kiểm tra checksum và tách nhiệt độ. Mỗi lần đọc, năm byte dữ liệu được lưu trong mảng `data[5]`. Hai byte đầu là dữ liệu độ ẩm, hai byte tiếp theo là nhiệt độ và byte cuối là checksum. Mặc dù chương trình không sử dụng giá trị độ ẩm, hai byte này vẫn được đưa vào phép tính checksum.

Điều kiện chấp nhận dữ liệu được biểu diễn bởi:

$$(data_0 + data_1 + data_2 + data_3) \bmod 256 = data_4. \quad (5.1)$$

Nếu điều kiện 5.1 sai hoặc một vòng chờ tín hiệu bị timeout, hàm trả về 0. Nếu dữ liệu hợp lệ, hàm trả về 1 và cập nhật biến nhiệt độ. Cơ chế này giúp vòng lặp chính phân biệt được hai trạng thái “có dữ liệu tin cậy” và “không có dữ liệu tin cậy”.

Để kiểm tra trên Proteus, có thể lần lượt đặt nhiệt độ của DHT22 tại các mức 15 °C, 30 °C, 45 °C, 60 °C và 65 °C. Giá trị hiển thị trên LCD phải thay đổi theo nhiệt độ đặt với một chữ số thập phân.



Hình 5.1 Kiểm thử hoạt động của cảm biến DHT22 trên mô hình thực tế

5.2.2 Kiểm tra trạng thái mất kết nối

Khi `DHT22_ReadData()` trả về 0, chương trình gán `trang_thai_cam_bien = 0`. Ở vòng lặp chính, hệ thống thực hiện ba hành động quan trọng:

- hiển thị chuỗi Dang ket noi... trên LCD;
- tiếp tục hiển thị bộ đếm thời gian ở dòng thứ hai;
- gán `current_pwm = 0` và bỏ qua phần tính PWM còn lại của vòng lặp.

Như vậy, khi dữ liệu nhiệt độ không còn tin cậy, động cơ bị đưa về trạng thái dừng thay vì tiếp tục sử dụng giá trị nhiệt độ cũ. Đây là lựa chọn an toàn phù hợp với mục tiêu của hệ thống.

Một phép kiểm thử mô phỏng có thể thực hiện bằng cách ngắt dây DATA của DHT22 hoặc làm cảm biến không phản hồi. Kết quả mong đợi là LCD chuyển sang trạng thái chờ kết nối và chân PB4 không còn phát lệnh PWM điều khiển động cơ.

5.3 Kiểm thử quan hệ nhiệt độ–PWM

5.3.1 Đối chiếu giá trị PWM theo mã nguồn

Thuật toán điều khiển tốc độ chia miền nhiệt độ thành ba khoảng:

$$P(T) = \begin{cases} 80, & T < 20^{\circ}\text{C}, \\ 80 + \left\lfloor \frac{T - 20}{40} \times 175 \right\rfloor, & 20^{\circ}\text{C} \leq T \leq 60^{\circ}\text{C}, \\ 255, & T > 60^{\circ}\text{C}. \end{cases} \quad (5.2)$$

Dấu lấy phần nguyên trong 5.2 xuất hiện do kết quả biểu thức số thực được ép về kiểu `uint8_t`. Bảng 5.2 cho thấy một số điểm kiểm thử đại diện.

Bảng 5.2 Giá trị PWM tính từ thuật toán điều khiển nhiệt độ

Nhiệt độ	Giá trị PWM	Duty xấp xỉ
15 °C	80	31.25%
20 °C	80	31.25%
30 °C	123	48.05%
40 °C	167	65.23%
45 °C	189	73.83%
50 °C	211	82.42%
60 °C	255	100%
65 °C	255	100%

Với các giá trị từ 1 đến 254, PWM phần mềm có duty xấp xỉ $\frac{P}{255} \times 100\%$. Riêng

trường hợp $\text{current_pwm} = 255$ được xử lý trực tiếp trong ISR bằng cách giữ PB4 luôn ở mức HIGH, do đó được coi là 100%.

Kết quả tính toán cho thấy đặc tính điều khiển đúng với mục tiêu của chương trình: động cơ duy trì một mức tối thiểu ở vùng nhiệt độ thấp, tăng gần tuyến tính trong khoảng 20–60 °C và đạt mức cực đại từ 60 °C trở lên.

5.3.2 Kiểm tra dạng sóng PWM

Timer0 chạy với clock hệ thống không chia tần. Với $F_{CPU} = 8\text{MHz}$, mỗi lần overflow xảy ra sau:

$$T_{OV0} = \frac{256}{8 \times 10^6} = 32\mu\text{s}. \quad (5.3)$$

Biến pwm_cnt tăng qua 256 trạng thái nên chu kỳ PWM lý tưởng là:

$$T_{PWM} = 256 \times 32\mu\text{s} = 8.192\text{ms}, \quad f_{PWM} \approx 122.07\text{Hz}. \quad (5.4)$$

Trên Proteus, có thể đặt Oscilloscope hoặc Logic Analyzer tại PB4. Khi thay đổi nhiệt độ, tần số cơ bản theo 5.4 phải gần như không đổi, trong khi độ rộng xung HIGH thay đổi theo Bảng 5.2.

5.4 Kiểm thử LCD và chức năng cảnh báo

5.4.1 Màn hình trạng thái chính

Ở chế độ bình thường, dòng đầu LCD hiển thị nhiệt độ theo dạng Temp: xx.x C; dòng thứ hai hiển thị thời gian Time: hh:mm:ss. Nếu:

$$T \geq T_{ngưỡng}, \quad (5.5)$$

chương trình ghi chuỗi !W! tại cột 13 của dòng đầu. Khi nhiệt độ giảm xuống dưới ngưỡng, ba vị trí này được ghi khoảng trắng để xóa cảnh báo.

Giá trị ngưỡng khi khởi động được đặt bằng 30 °C. Do đó, nếu chưa thay đổi ngưỡng, nhiệt độ từ 30 °C trở lên phải làm xuất hiện cảnh báo. Cần lưu ý rằng ngưỡng cảnh báo không tham gia vào công thức điều khiển PWM; tốc độ động cơ vẫn được tính theo 5.2.



Hình 5.2 Trạng thái cảnh báo trên màn hình LCD khi nhiệt độ vượt ngưỡng

5.4.2 *Màn hình cài đặt ngưỡng*

Khi nhấn SET, biến `che_do_cai_dat` đảo trạng thái. Trong chế độ cài đặt:

- LCD dòng đầu hiển thị SET NGUONG::;
- LCD dòng thứ hai hiển thị Max: xx.x C;
- UP làm tăng ngưỡng 1 °C;
- DOWN làm giảm ngưỡng 1 °C;
- RESET gán ngưỡng bằng 45 °C.



Hình 5.3 Màn hình LCD ở chế độ cài đặt ngưỡng nhiệt độ



Hình 5.4 Kết quả thay đổi giá trị ngưỡng nhiệt độ trên LCD

Bảng 5.3 là bộ tình huống kiểm tra giao diện đề xuất.

Bảng 5.3 Kiểm thử các trạng thái hiển thị LCD

Tình huống	Thao tác/đầu vào	Kết quả mong đợi
Bình thường dưới ngưỡng	$T < T_{nguong}$	Hiển thị nhiệt độ và thời gian, không có !W!
Vượt ngưỡng	$T \geq T_{nguong}$	Hiển thị !W! trên dòng nhiệt độ
Cài đặt	Nhấn SET	Hiển thị SET NGUONG và giá trị Max
Tăng/giảm ngưỡng	Nhấn UP/DOWN	Giá trị Max thay đổi từng 1 °C
Reset ngưỡng	Nhấn RESET khi đang SET	Giá trị Max trở về 45 °C
Mất cảm biến	DHT22 không phản hồi	Hiển thị Dang ket noi...

5.5 Kiểm thử nút nhấn và chống dội

Timer2 được cấu hình ở chế độ CTC với prescaler 1024 và OCR2 = 78. Theo datasheet ATmega16, cấu hình CS22:0 = 111 chọn hệ số chia 1024 và CTC cho phép tạo ngắt khi bộ đếm bằng OCR2 [1]. Chu kỳ ngắt là:

$$T_{T2} = \frac{(78 + 1) \times 1024}{8 \times 10^6} = 10.112 \text{ ms.} \quad (5.6)$$

Một nút chỉ được xác nhận khi ở mức LOW trong hai lần lấy mẫu liên tiếp, nên thời gian xác nhận danh định khoảng:

$$T_{xac_nhan} \approx 2T_{T2} = 20.224 \text{ ms.} \quad (5.7)$$

Sau lần xác nhận thứ hai, cờ sự kiện được đặt bằng 1. Nếu tiếp tục giữ nút, bộ đếm trạng thái đạt 3 và không tạo thêm sự kiện. Khi nhả nút, bộ đếm trở về 0. Vì vậy, thuật toán có tác dụng vừa chống dội, vừa ngăn một lần giữ nút tạo ra nhiều lần tăng/giảm liên tục.

Các tình huống nên kiểm tra gồm:

- nhấn và nhả SET một lần: chỉ chuyển chế độ một lần;
- nhấn UP một lần trong chế độ SET: ngưỡng tăng đúng 1 °C;
- giữ UP: chương trình hiện tại không tự lặp tăng liên tục;

- nhấn DOWN một lần: ngưỡng giảm đúng 1 °C;
- nhấn RESET: ngưỡng được đặt thành 45 °C;
- tạo xung nút rất ngắn dưới thời gian xác nhận: không nên tạo sự kiện.

Một giới hạn của mã nguồn hiện tại là chưa có ràng buộc giá trị nhỏ nhất/lớn nhất cho `nguon_nhiet_do`. Nếu nhấn UP hoặc DOWN nhiều lần, ngưỡng có thể vượt ra ngoài vùng nhiệt độ có ý nghĩa của ứng dụng. Phiên bản sau nên giới hạn ngưỡng, ví dụ trong khoảng thiết kế đã lựa chọn.

5.6 Kiểm thử Timer1 và bộ đếm thời gian

Timer1 được cấu hình ở chế độ CTC với $WGM12 = 1$. Chương trình đặt $CS12 = 1$, $CS11 = CS10 = 0$; theo datasheet, cấu hình $CS12:0 = 100$ tương ứng prescaler 256 [1]. Với $OCR1A = 31249$, chu kỳ Compare Match A là:

$$T_{T1} = \frac{(31249 + 1) \times 256}{8 \times 10^6} = 1 \text{ s.} \quad (5.8)$$

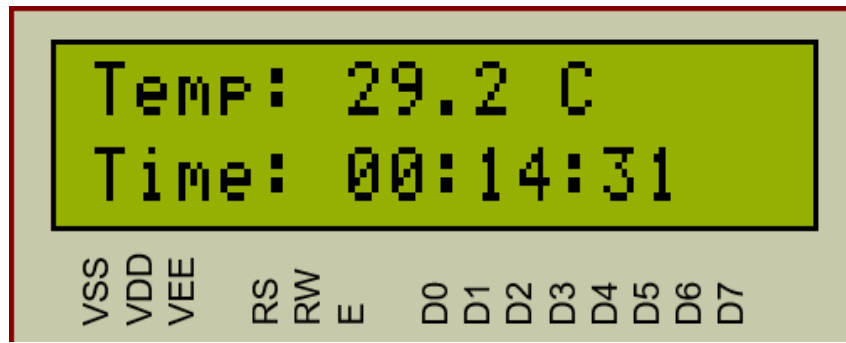
Mỗi ngắt Timer1 tăng giây; sau 60 giây tăng phút; sau 60 phút tăng giờ; sau 24 giờ đưa giờ về 0. Về mặt cấu hình thanh ghi, kết quả tính toán xác nhận mốc thời gian một giây là đúng khi clock hệ thống thực tế bằng 8 MHz.

Bảng 5.4 đưa ra các mốc kiểm tra.

Bảng 5.4 Các mốc kiểm thử bộ đếm thời gian

Trạng thái trước ngắt	Trạng thái sau ngắt
00:00:00	00:00:01
00:00:59	00:01:00
00:59:59	01:00:00
23:59:59	00:00:00

Khi kiểm thử thực tế trong thời gian dài, sai số còn phụ thuộc vào sai số nguồn clock 8 MHz. Bộ đếm này là thời gian chạy tương đối kể từ lúc khởi động, không phải đồng hồ thời gian thực và không giữ thời gian khi mất nguồn.



Hình 5.5 Kết quả kiểm thử bộ đếm thời gian sử dụng Timer1

5.7 Kiểm thử giao tiếp UART

5.7.1 Kiểm tra cấu hình tốc độ truyền

Hàm `UART_Init(9600)` sử dụng chế độ bất đồng bộ thông thường. Theo công thức của datasheet ATmega16 [1]:

$$UBRR = \frac{F_{CPU}}{16BAUD} - 1. \quad (5.9)$$

Với $F_{CPU} = 8\text{MHz}$ và $BAUD = 9600$, chương trình lưu $UBRR = 51$. Tốc độ thực tế là:

$$BAUD_{thuc} = \frac{8 \times 10^6}{16(51 + 1)} \approx 9615.38 \text{ bit/s}. \quad (5.10)$$

Sai số so với 9600 bit/s vào khoảng 0,16%, đủ nhỏ cho giao tiếp UART thông thường. UCSRC chọn 8 bit dữ liệu, không parity và một stop bit, vì vậy Virtual Terminal phải được cấu hình 9600–8N1.

5.7.2 Kiểm tra dữ liệu truyền

Ở chế độ hiển thị bình thường, hệ thống định kỳ gửi chuỗi nhiệt độ và giá trị PWM. Với động cơ đang hoạt động, định dạng có dạng:

Nhiet do: xx.x C | PWM: xxx

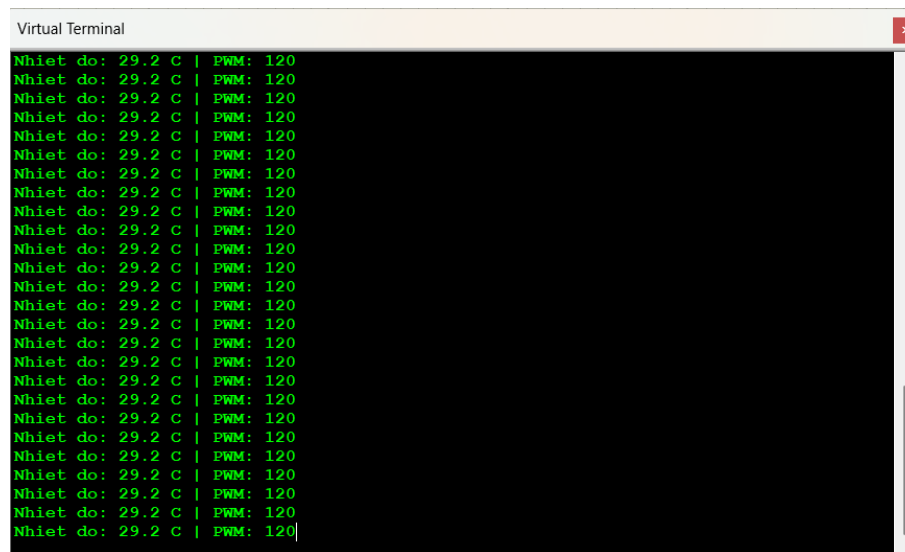
Nếu `trang_thai_dong_co = 0`, chuỗi truyền chuyển thành:

Nhiet do: xx.x C | PWM: 0 (TAM DUNG)

Khi nhiệt độ lớn hơn hoặc bằng ngưỡng, chương trình gửi thêm:

[CANH BA0] Nhiet do vuot nguong!

Biến `bo_dem_uart` phát dữ liệu sau mỗi 11 vòng lặp bình thường. Nếu chỉ xét lệnh trễ 50 ms, chu kỳ truyền danh định xấp xỉ 550 ms. Trong thực tế, chu kỳ này dài hơn do còn thời gian truy cập DHT22, LCD và các hàm xử lý chuỗi.



Hình 5.6 Dữ liệu trạng thái được truyền qua UART trên Virtual Terminal

5.7.3 Kiểm tra lệnh điều khiển từ UART

Ngắt nhân USART xử lý hai ký tự:

- gửi '0': `trang_thai_dong_co = 0`, `current_pwm` về 0 ở vòng lặp chính;
- gửi '1': `trang_thai_dong_co = 1`, PWM được khôi phục theo nhiệt độ hiện tại.

Bảng 5.5 tổng hợp kết quả logic.

Bảng 5.5 Kiểm thử lệnh điều khiển qua UART

Ký tự nhận	Trạng thái động cơ	PWM điều khiển
'0'	Dừng	0
'1'	Cho phép chạy	Theo nhiệt độ
Ký tự khác	Không đổi	Không đổi

```
Virtual Terminal
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
[HE THONG] Động cơ đã dừng!

Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
```

Hình 5.7 Kết quả gửi lệnh '0' để dừng động cơ qua UART

```
Virtual Terminal
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
Nhiệt độ: 29.2 C | PWM: 0 (TAM DUNG)
[HE THONG] Động cơ hoạt động trở lại!

Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
Nhiệt độ: 29.2 C | PWM: 120
```

Hình 5.8 Kết quả gửi lệnh '1' để cho phép động cơ hoạt động trở lại

5.8 Kiểm thử Watchdog

Chương trình kích hoạt Watchdog bằng:

```
wdt_enable(WDTO_2S);
```

và gọi `wdt_reset()` trong màn hình khởi động cũng như mỗi vòng lặp chính. Datasheet ATmega16 cho biết Watchdog sử dụng bộ dao động riêng; với mức chia lớn nhất, thời gian timeout điển hình ở 5 V khoảng 2,1 s [1].

Có thể kiểm tra chức năng Watchdog bằng một phiên bản chương trình thử nghiệm trong đó tạm thời bỏ `wdt_reset()` hoặc cố ý tạo một vòng lặp vô hạn sau khi Watchdog

đã được bật. Nếu hệ thống tự quay lại màn hình khởi động sau thời gian timeout thì cơ chế Watchdog hoạt động đúng.

Việc tạo trạng thái treo chỉ nên thực hiện trong mô phỏng hoặc trên phần cứng khi bảo đảm không gây trạng thái nguy hiểm cho tăng công suất.

5.9 Kiểm thử tích hợp toàn hệ thống

Sau khi từng khối riêng lẻ hoạt động, hệ thống được kiểm tra theo một chuỗi tình huống liên tục. Một kịch bản kiểm thử tích hợp phù hợp với chương trình hiện tại gồm:

1. Cấp nguồn hoặc reset vi điều khiển. LCD hiển thị `Dang ket noi...` trong khoảng 2 s.
2. DHT22 trả về dữ liệu hợp lệ. LCD chuyển sang giao diện nhiệt độ và thời gian.
3. Đặt nhiệt độ dưới 20 °C. Động cơ chạy với PWM tối thiểu bằng 80.
4. Tăng nhiệt độ lần lượt qua 30, 45 và 60 °C. PWM tăng theo Bảng 5.2.
5. Với ngưỡng mặc định 30 °C, khi nhiệt độ đạt hoặc vượt 30 °C, LCD xuất hiện `!W!` và UART gửi cảnh báo.
6. Nhấn SET, dùng UP/DOWN thay đổi ngưỡng, sau đó nhấn SET để trở lại giao diện chính.
7. Gửi '0' từ Virtual Terminal. Động cơ dừng và dữ liệu UART thể hiện trạng thái TAM DUNG.
8. Gửi '1'. Động cơ hoạt động lại với PWM tương ứng nhiệt độ hiện tại.
9. Làm DHT22 mất phản hồi. PWM phải về 0 và LCD trở lại trạng thái chờ kết nối.

Bảng 5.6 tổng hợp các quan hệ chức năng có thể xác minh trực tiếp từ mã nguồn.

Bảng 5.6 Tổng hợp kết quả đối chiếu chức năng của phần mềm

Chức năng	Kết quả đối chiếu	Đánh giá
Đọc nhiệt độ DHT22	Có timeout, checksum và giải mã dấu/độ phân giải 0,1°C	Phù hợp
Điều khiển PWM theo nhiệt độ	Đúng ba miền, 80–255	Phù hợp
An toàn khi mất cảm biến	Ép <code>current_pwm = 0</code>	Phù hợp
LCD 16x2	Có giao diện chính, cài đặt và cảnh báo	Phù hợp
Nút nhấn	Lấy mẫu Timer2 và chống dội hai mẫu	Phù hợp
Bộ đếm thời gian	Timer1 CTC tạo mốc lý tưởng 1 s	Phù hợp
UART	9600–8N1, truyền trạng thái, nhận lệnh chạy/dừng	Phù hợp
Watchdog	Bật khoảng 2 s và được làm mới định kỳ	Phù hợp

5.10 Các vấn đề và giới hạn phát hiện

Bên cạnh các chức năng đã triển khai, quá trình phân tích và kiểm thử cho thấy một số điểm cần lưu ý.

5.10.1 Thời gian đọc DHT22 làm gián đoạn các ngắt

Hàm `DHT22_ReadData()` gọi `cli()` trước khi phát tín hiệu START và chỉ gọi `sei()` sau khi đã đọc xong 40 bit. Trong thời gian này, các ISR Timer0, Timer1, Timer2 và USART không được thực thi. Riêng tín hiệu START đã kéo LOW 20 ms, chưa kể thời gian phản hồi và truyền dữ liệu của DHT22.

Do Timer0 tạo PWM hoàn toàn bằng ISR overflow, việc tắt ngắt trong thời gian đọc DHT22 làm PWM phần mềm không được cập nhật liên tục. Trong mô phỏng hệ thống vẫn có thể biểu hiện đúng về xu hướng tốc độ, nhưng khi triển khai phần cứng thật, dạng sóng PWM có thể xuất hiện khoảng gián đoạn theo chu kỳ đọc cảm biến.

Một hướng cải tiến là sử dụng PWM phần cứng trên chân OCx hoặc xây dựng bộ đọc DHT22 dựa trên Timer/Input Capture thay vì khóa toàn bộ ngắt trong thời gian dài.

5.10.2 Truyền chuỗi UART bên trong ISR nhận

Trong USART_RXC_vect, sau khi nhận '0' hoặc '1', chương trình gọi trực tiếp UART_String(). Hàm truyền sử dụng polling chờ UDRE nên ISR có thể kéo dài trong thời gian gửi toàn bộ thông báo. Trong lúc đang ở ISR, các ngắt khác chưa được phục vụ, vì vậy có khả năng tạo jitter cho Timer và PWM.

Phương án tốt hơn là ISR chỉ đọc ký tự và đặt một cờ; phần gửi chuỗi phản hồi được thực hiện ở vòng lặp chính.

5.10.3 Chu kỳ đọc DHT22

Biến dht_timer được đặt bằng 20 và vòng lặp có trễ 50 ms, nên chu kỳ gọi cảm biến danh định khoảng 1 s. Tài liệu DHT22/AM2302 khuyến nghị khoảng lấy mẫu đủ lớn để bảo đảm độ ổn định của phép đo [?]. Nếu triển khai mạch thật, nên tăng chu kỳ đọc lên khoảng 2 s và kiểm tra lại đáp ứng điều khiển.

5.10.4 Giá trị ngưỡng mặc định chưa thống nhất

Biến nguonng_nhiet_do được khởi tạo bằng 30 °C, trong khi thao tác RESET ở chế độ cài đặt gán ngưỡng bằng 45 °C. Nếu RESET được hiểu là “khôi phục mặc định”, hai giá trị này cần được thống nhất.

Ngoài ra, chương trình chưa giới hạn biên cho ngưỡng. Đây là điểm nên bổ sung để tránh người dùng cài giá trị quá thấp hoặc quá cao.

5.11 Đối chiếu với yêu cầu của hệ thống

Theo phiên bản phần mềm cuối, các chức năng chính gồm đọc nhiệt độ DHT22, điều khiển PWM, hiển thị LCD, thiết lập ngưỡng, UART, Timer, chống dội nút và Watchdog đều đã có phần triển khai cụ thể trong mã nguồn. Bảng 5.7 trình bày mức độ đáp ứng theo chương trình hiện tại.

Bảng 5.7 Đối chiếu yêu cầu và chức năng đã triển khai

Yêu cầu	Cách thực hiện	Mức độ
Đo nhiệt độ	DHT22 tại PA0, giao tiếp số 40 bit	Đã triển khai
Điều khiển tốc độ động cơ	PWM phần mềm PB4 theo nhiệt độ	Đã triển khai
Hiển thị thông tin	LCD 16x2, giao tiếp 8 bit	Đã triển khai
Thiết lập ngưỡng	SET/UP/DOWN/RESET	Đã triển khai
Cảnh báo quá nhiệt	!W! trên LCD và chuỗi UART	Đã triển khai
Giao tiếp máy tính	USART 9600–8N1, lệnh '0'/'1'	Đã triển khai
Tác vụ thời gian	Timer1, Timer2, Timer0	Đã triển khai
Tự khôi phục phần mềm	Watchdog khoảng 2 s	Đã triển khai

Cần thống nhất lại các chương trước của báo cáo nếu vẫn còn mô tả cảm biến LM35 và ADC. Phiên bản `main.c` dùng trong Chương 4 và Chương 5 đã chuyển sang DHT22; do đó báo cáo cuối nên sử dụng cùng một phương án cảm biến ở tất cả các chương để tránh mâu thuẫn giữa yêu cầu, thiết kế phần cứng và phần mềm.

5.12 Đánh giá chung

Qua phân tích cấu hình ngoại vi và các tình huống kiểm thử, chương trình đã hình thành được một hệ thống điều khiển nhiệt độ tương đối đầy đủ trên ATmega16. Các khối chức năng được phân tách rõ: DHT22 cung cấp dữ liệu nhiệt độ; Timer0 tạo PWM phần mềm; Timer1 quản lý thời gian; Timer2 xử lý phím; UART đảm nhiệm truyền thông; LCD cung cấp giao diện tại chỗ; Watchdog tăng khả năng phục hồi.

Điểm tích cực của thiết kế là có xử lý mất cảm biến, có checksum cho DHT22, có chống dột nút và có cơ chế dừng động cơ từ UART. Các thông số Timer1, Timer2 và UART phù hợp với tần số hệ thống 8 MHz khi đối chiếu với datasheet ATmega16 [1].

Hạn chế chính nằm ở kiến trúc thời gian thực: quá trình đọc DHT22 tắt toàn bộ ngắt trong thời gian tương đối dài, trong khi PWM lại phụ thuộc vào ngắt Timer0; đồng thời ISR nhận UART thực hiện truyền chuỗi trực tiếp. Hai điểm này có thể làm tăng jitter khi chạy trên phần cứng thật. Vì vậy, bước phát triển tiếp theo nên ưu tiên chuyển PWM sang ngoại vi PWM phần cứng, rút ngắn ISR và điều chỉnh chu kỳ đọc DHT22. Sau khi bổ sung ảnh mô phỏng Proteus, ảnh Virtual Terminal và dạng sóng PB4, Chương 5 sẽ

có đầy đủ minh chứng trực quan cho các kết quả đã phân tích.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

Trong quá trình thực hiện Đồ án II, hệ thống giám sát nhiệt độ, độ ẩm và điều khiển tốc độ động cơ đã được xây dựng trên nền vi điều khiển ATmega16. Các khối chức năng chính của hệ thống được thiết kế và tích hợp theo hướng kết hợp giữa phần cứng và phần mềm.

Về phần cứng, đồ án đã thực hiện thiết kế các khối nguồn, vi điều khiển ATmega16, cảm biến DHT22, LCD 16x2, nút nhấn, giao tiếp UART, ngõ ra PWM điều khiển động cơ và các mạch phụ trợ phục vụ nạp chương trình, Reset và kết nối ngoại vi. Sơ đồ nguyên lý và mạch in PCB được xây dựng trên Altium Designer, đồng thời các tệp phục vụ chế tạo như Gerber và NC Drill được chuẩn bị từ thiết kế PCB.

Về phần mềm, chương trình điều khiển được phát triển bằng ngôn ngữ C cho ATmega16. Hệ thống thực hiện thu nhận dữ liệu nhiệt độ và độ ẩm từ DHT22, hiển thị các thông số trên LCD 16x2, tạo tín hiệu PWM thay đổi theo nhiệt độ, xử lý các nút nhấn, truyền nhận dữ liệu qua UART, sử dụng Timer và ngắt để thực hiện các tác vụ định thời, đồng thời sử dụng Watchdog Timer nhằm tăng khả năng phục hồi của chương trình.

Các chức năng của hệ thống được kiểm tra thông qua mô phỏng trên Proteus. Quá trình mô phỏng cho phép theo dõi dữ liệu cảm biến, nội dung hiển thị LCD, tín hiệu PWM, trạng thái nút nhấn và dữ liệu truyền nhận UART. Qua đó, hoạt động của các khối chức năng được kiểm tra trước khi triển khai hoặc hoàn thiện trên phần cứng thực tế.

Nhìn chung, kết quả của đồ án đã hình thành được một hệ thống nhúng tương đối hoàn chỉnh, từ quá trình thiết kế nguyên lý, lập trình vi điều khiển, mô phỏng đến thiết kế PCB và chuẩn bị dữ liệu chế tạo.

6.2 Mức độ hoàn thành mục tiêu

So với các mục tiêu đã đặt ra ở Chương 1, đồ án đã thực hiện được phần lớn các chức năng chính của hệ thống.

ATmega16 đã được sử dụng làm bộ xử lý trung tâm và có khả năng phối hợp với các ngoại vi cần thiết. Dữ liệu nhiệt độ và độ ẩm được thu nhận từ cảm biến DHT22 thông qua giao tiếp số. Các thông số của hệ thống được hiển thị trực tiếp trên LCD 16x2, đồng thời dữ liệu trạng thái có thể được truyền tới thiết bị bên ngoài qua giao tiếp UART.

Chức năng điều khiển động cơ được thực hiện bằng tín hiệu PWM từ Timer0. Giá

trị duty cycle được thay đổi theo nhiệt độ đo được, qua đó tạo khả năng điều chỉnh mức hoạt động của động cơ theo điều kiện môi trường. Người sử dụng có thể tương tác với hệ thống thông qua các nút nhấn để thay đổi tham số cài đặt. Các bộ Timer, ngắt và Watchdog Timer cũng được tích hợp nhằm hỗ trợ việc xử lý thời gian, quét phím và tăng độ ổn định của chương trình.

Về phần cứng, sơ đồ nguyên lý và PCB đã được xây dựng trên Altium Designer, đồng thời dữ liệu Gerber và NC Drill được xuất để phục vụ chế tạo. Các chức năng chính của hệ thống đã được mô phỏng trên Proteus để đánh giá khả năng hoạt động và sự phối hợp giữa phần mềm với các khối ngoại vi.

Tuy nhiên, mức độ hoàn thành của một số nội dung còn phụ thuộc vào phạm vi kiểm thử phần cứng thực tế. Những kết quả chưa được xác nhận bằng đo đạc trên bo mạch thật cần tiếp tục được kiểm chứng trước khi đánh giá đầy đủ độ chính xác, độ ổn định và khả năng làm việc lâu dài của hệ thống.

6.3 Hạn chế của hệ thống

Mặc dù các chức năng cơ bản đã được xây dựng, hệ thống vẫn còn một số hạn chế cần được xem xét.

Trước hết, độ chính xác của phép đo nhiệt độ và độ ẩm phụ thuộc vào đặc tính của cảm biến DHT22, điều kiện môi trường và cách bố trí cảm biến trên hệ thống. Dữ liệu thu được phù hợp cho các ứng dụng giám sát và điều khiển thông thường, nhưng chưa được hiệu chuẩn với thiết bị đo chuẩn để đánh giá sai số thực tế của toàn bộ hệ thống.

Thuật toán điều khiển động cơ hiện chủ yếu dựa trên giá trị nhiệt độ và thay đổi duty cycle PWM theo các vùng đã xác định. Hệ thống chưa sử dụng tín hiệu phản hồi tốc độ từ động cơ, vì vậy giá trị PWM chỉ thể hiện mức điều khiển đầu vào chứ chưa phản ánh chính xác tốc độ quay thực tế khi tải thay đổi.

Các tác vụ thời gian được xây dựng từ Timer bên trong ATmega16 và phần mềm điều khiển. Độ chính xác của bộ đếm thời gian phụ thuộc vào tần số xung nhịp, prescaler và giá trị thanh ghi so sánh. Do đó, các thông số Timer cần được kiểm chứng kỹ khi yêu cầu độ chính xác thời gian cao.

Tăng công suất điều khiển động cơ cũng là một nội dung cần được hoàn thiện dựa trên dòng điện và điện áp của tải thực tế. Việc lựa chọn linh kiện công suất, mạch bảo vệ và phương án tản nhiệt cần được thực hiện phù hợp khi chuyển từ mô phỏng sang sản phẩm phần cứng.

Ngoài ra, hệ thống chưa được đánh giá đầy đủ về khả năng chống nhiễu điện từ, ảnh hưởng của động cơ tới nguồn và tín hiệu cảm biến, cũng như độ ổn định khi hoạt động liên tục trong thời gian dài. Đây là những yếu tố cần được kiểm tra thêm trong quá

trình hoàn thiện phần cứng.

6.4 Bài học kinh nghiệm

Quá trình thực hiện đồ án giúp củng cố kiến thức và kỹ năng trong cả thiết kế phần cứng lẫn phát triển phần mềm cho hệ thống nhúng.

Trong quá trình sử dụng Altium Designer, việc xây dựng thư viện linh kiện cho thấy tầm quan trọng của việc kiểm tra kỹ số chân, chức năng chân và mối liên hệ giữa symbol với footprint. Một sai lệch nhỏ trong quá trình mapping giữa schematic và footprint có thể dẫn tới kết nối sai trên PCB, vì vậy các thông số cần được đối chiếu với datasheet trước khi hoàn thiện thiết kế.

Việc lập bảng phân bổ chân của ATmega16 trước khi thiết kế schematic cũng là một bước quan trọng. Bảng pin mapping giúp duy trì sự thống nhất giữa chương trình, sơ đồ nguyên lý, PCB và mô hình Proteus, đồng thời hạn chế việc các ngoại vi sử dụng trùng hoặc sai chức năng của chân vi điều khiển.

Trong thiết kế PCB, quá trình bố trí linh kiện, đi dây nguồn, xử lý mass và kiểm tra Design Rule Check cho thấy thiết kế mạch in không chỉ yêu cầu kết nối đúng về mặt logic mà còn phải quan tâm đến khả năng chế tạo, lắp ráp và chống nhiễu.

Về phần mềm, việc sử dụng Timer, ngắt và các cờ trạng thái giúp chương trình có thể thực hiện nhiều chức năng mà không phải xử lý tuần tự hoàn toàn trong vòng lặp chính. Qua đó, đồ án giúp củng cố cách tổ chức chương trình nhúng theo hướng phân chia tác vụ và hạn chế thời gian xử lý trong các hàm phục vụ ngắt.

Mô phỏng trên Proteus hỗ trợ đáng kể trong quá trình kiểm tra thuật toán và gỡ lỗi trước khi sử dụng bo mạch thực tế. Đặc biệt, Virtual Terminal và các công cụ quan sát tín hiệu giúp kiểm tra UART, PWM và trạng thái ngoại vi một cách trực quan.

Qua quá trình thực hiện, một kinh nghiệm quan trọng được rút ra là phần cứng, firmware và mô phỏng cần được phát triển đồng thời và thường xuyên đối chiếu với nhau. Việc thay đổi một chân kết nối hoặc một chức năng trong firmware cần được cập nhật tương ứng trên schematic, PCB và tài liệu báo cáo để tránh sai lệch giữa các phiên bản.

6.5 Hướng phát triển

Trên cơ sở hệ thống đã xây dựng, một số hướng phát triển có thể được xem xét trong tương lai:

- Hoàn thiện tăng công suất điều khiển động cơ hoặc quạt dựa trên điện áp, dòng điện và công suất của tải thực tế; bổ sung các mạch bảo vệ cần thiết để tăng độ an toàn

và độ tin cậy.

- Thực hiện hiệu chuẩn cảm biến DHT22 bằng thiết bị đo tham chiếu hoặc sử dụng cảm biến có độ chính xác cao hơn trong các ứng dụng yêu cầu sai số nhỏ.
- Bổ sung cảm biến tốc độ động cơ để xây dựng hệ thống điều khiển vòng kín. Khi đó, tốc độ thực tế có thể được phản hồi về vi điều khiển để hiệu chỉnh duty cycle PWM phù hợp với tải.
- Sử dụng IC thời gian thực RTC khi hệ thống cần duy trì thông tin ngày, giờ với độ chính xác cao và tiếp tục lưu thời gian khi nguồn chính bị ngắt.
- Sử dụng bộ nhớ EEPROM của ATmega16 để lưu các giá trị ngưỡng và tham số cài đặt, giúp các thông số không bị mất sau khi hệ thống Reset hoặc mất nguồn.
- Bổ sung các giao tiếp không dây như Bluetooth, Wi-Fi hoặc các chuẩn truyền thông phù hợp để theo dõi và điều khiển hệ thống từ xa.
- Xây dựng giao diện giám sát trên máy tính hoặc thiết bị di động để hiển thị nhiệt độ, độ ẩm, trạng thái động cơ và lưu trữ lịch sử dữ liệu.
- Tối ưu lại PCB về bố trí nguồn, mặt phẳng mass, khoảng cách giữa khối công suất và khối tín hiệu, khả năng chống nhiễu điện từ và tính thuận tiện khi lắp ráp, kiểm tra.
- Nghiên cứu các thuật toán điều khiển nâng cao hơn thay cho luật điều khiển tuyến tính hiện tại nhằm cải thiện chất lượng điều khiển trong các ứng dụng có yêu cầu cao hơn.

6.6 Kết luận chung

Đồ án đã giúp xây dựng và kiểm chứng các khối cơ bản của một hệ thống điều khiển nhúng sử dụng vi điều khiển ATmega16. Hệ thống có khả năng thu nhận nhiệt độ và độ ẩm từ DHT22, hiển thị thông tin, giao tiếp UART và tạo tín hiệu PWM để điều khiển động cơ theo nhiệt độ. Bên cạnh phần lập trình, quá trình thiết kế sơ đồ nguyên lý, xây dựng PCB và chuẩn bị dữ liệu chế tạo đã giúp kết nối kiến thức lý thuyết với quy trình phát triển một hệ thống điện tử thực tế. Quá trình mô phỏng và kiểm thử cũng cho thấy tầm quan trọng của việc kiểm tra sự thống nhất giữa phần cứng và firmware trong từng giai đoạn thiết kế. Một số nội dung của hệ thống vẫn cần tiếp tục được kiểm chứng và tối ưu trên phần cứng thực tế, đặc biệt về độ chính xác, khả năng chống nhiễu và tăng công suất điều khiển tải. Qua đồ án, các kiến thức về vi điều khiển, thiết kế PCB, lập trình nhúng và kiểm thử hệ thống đã được củng cố và vận dụng trong một bài toán cụ thể. Kết quả đạt được là cơ sở để tiếp tục hoàn thiện hệ thống theo hướng ổn định, chính xác và có khả năng ứng dụng cao hơn trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] Microchip Technology / Atmel, *ATmega16/ATmega16A 8-bit AVR Microcontroller Datasheet*.
- [2] Texas Instruments, *LM35 Precision Centigrade Temperature Sensors Datasheet*.
- [3] Tài liệu kỹ thuật bộ điều khiển HD44780 và module LCD ký tự 16x2.
- [4] Altium, *Altium Designer Documentation*.
- [5] Labcenter Electronics, *Proteus Design Suite Documentation*.
- [6] AVR Libc Project, *AVR Libc User Manual*.